

LEVERAGING SYNTHETIC DATA FROM LARGE LANGUAGE MODELS
TO STEER AND ENHANCE MODEL LEARNING

Ajay Patel

A DISSERTATION

in

Department of Computer and Information Science

Presented to the Faculties of the University of Pennsylvania

in

Partial Fulfillment of the Requirements for the

Degree of Doctor of Philosophy

2025

Supervisor of Dissertation

Chris Callison-Burch, Professor, University of Pennsylvania

Graduate Group Chairperson

Anindya De, Associate Professor, University of Pennsylvania

Dissertation Committee

Dan Roth, Eduardo D. Glandt Distinguished Professor, University of Pennsylvania

Lyle Ungar, Professor, University of Pennsylvania

Duncan Watts, Stevens University Professor, University of Pennsylvania

Hannaneh Hajishirzi, Professor, University of Washington

Colin Raffel, Associate Professor, University of Toronto

LEVERAGING SYNTHETIC DATA FROM LARGE LANGUAGE MODELS
TO STEER AND ENHANCE MODEL LEARNING

© COPYRIGHT

2025

Ajay Patel

This work is licensed under the

Creative Commons

Attribution-NonCommercial-ShareAlike 4.0 International

License

To view a copy of this license, visit

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

ABSTRACT

LEVERAGING SYNTHETIC DATA FROM LARGE LANGUAGE MODELS TO STEER AND ENHANCE MODEL LEARNING

Ajay Patel

Chris Callison-Burch

Synthetic data, training data generated by large language models (LLMs), has enabled a promising direction for model improvement. LLMs can generate synthetic data to improve their own performance or enhance smaller models on specific tasks, sometimes even enabling these models to surpass the larger LLM’s capabilities. Despite these promises, using synthetic data for model enhancement is counterintuitive and challenging to implement in practice. The characteristics of synthetic data that lead to improved model performance—and the mechanisms by which models learn from it—are poorly understood and vary by task.

In this dissertation, I contend that synthetic data’s effectiveness hinges on using it as a method of encoding prior knowledge about a task. From this perspective, I reframe synthetic data as an *inductive bias*—a mechanism through which human practitioners equip models with helpful learning biases and assumptions to improve their performance. LLMs enable practitioners to, at scale, steer and enhance the learning of neural models by allowing them to shape the training examples fed into a model with their prior knowledge about a task. Practitioners can do this by influencing the synthetic data generation, filtering, and selection process. The mechanism for model learning after training on synthetic data derives from this prior knowledge.

I first demonstrate this in an in-context learning setting, using a self-consistency prior to bootstrap synthetic examples to improve the machine translation performance of a LLM. Next, I introduce a framework, DataDreamer, to make synthetic data generation and training work-

flows easier to implement. Using DataDreamer, I demonstrate in a fine-tuning setting how to encode task-specific learning biases in synthetic datasets to enable training stronger embedding models, agent models, and vision-language models. Finally, I use synthetic data to impose an inductive bias during LLM pre-training, the fundamental process that enables LLMs and their emergent general capabilities. This work restructures LLM pre-training documents into synthetic instruction-answer pairs to better align pre-training with the distribution of downstream, real-world usage. Reformulating the data and objective into a supervised, instruction-tuning objective at pre-training scale instead of the typical, self-supervised next token prediction objective achieves stronger performing general models with an equivalent training compute budget.

TABLE OF CONTENTS

Abstract	iii
List of Tables	viii
List of Figures	xi
Chapter 1 Introduction	1
1.1 Challenges with Using Synthetic Data	4
1.2 When and Why Does Synthetic Data Improve Model Performance?	6
1.3 Thesis Statement	13
1.4 Dissertation Outline	13
1.5 Relevant Publications Presented	15
Chapter 2 Background	18
2.1 Early Applications of Synthetic Data	18
2.2 Limitations of Synthetic Data	21
2.3 Large Language Models	22
2.4 Synthetic Data Approaches with Large Language Models	25
2.5 Tasks Discussed in this Dissertation	27
Chapter 3 Enhancing Model Performance using Synthetic Data and In-Context Learning . .	31
3.1 Experimental Setup	32
3.2 Results	35
3.3 Discussion	38

3.4	Conclusion	40
3.5	Summary of Contributions	41
Chapter 4	DataDreamer: A Framework For Synthetic Data Generation and Training Workflows	42
4.1	Motivation	43
4.2	Current Challenges with LLM Workflows	44
4.3	DataDreamer	46
4.4	Reproducible LLM Research and Workflows	58
4.5	Examples	60
4.6	Impact and Case Studies	65
4.7	Conclusion	65
4.8	Summary of Contributions	66
Chapter 5	Enabling New Model Capabilities by Fine-Tuning on Synthetic Data	67
5.1	Example 1: Training Style Embedding Models	67
5.2	Example 2: Making Agent Models More Capable	80
5.3	Example 3: Enabling Vision Language Models to Understand Text-Rich Figures . .	100
5.4	Conclusion	124
Chapter 6	Advancing Large Language Model Pre-Training with Synthetic Data	126
6.1	Motivation	127
6.2	FineInstructions	129
6.3	Experimental Setup	139
6.4	Results	142
6.5	Discussion	144
6.6	Conclusion	148
6.7	Summary of Contributions	149
Chapter 7	Conclusion and Future Directions	150

7.1	Considerations for Applying Synthetic Data	153
7.2	Future Directions	154
	Bibliography	159

LIST OF TABLES

Table 3.1	Abbreviated machine translation results on FLORES-101 devtest (sp-BLEU).	36
Table 3.2	Full machine translation results on FLORES-101 devtest (sp-BLEU).	37
Table 3.3	Results from ablating the use of mT5Score as a filter to select the high-quality, self-consistent synthetic examples during bootstrapping.	38
Table 3.4	Results with prompt ensembles of size 4, 8, and 16.	39
Table 3.5	Results from ablating the use of English-centric bootstrapping.	39
Table 4.1	We compare feature coverage between other popular libraries and solutions available to researchers today that target similar workflows. DataDreamer integrates these features into a single library with a standardized interface making experimentation and chaining data between tasks simple. (✗= No; ✓= Yes; ○ = Partial Support)	49
Table 4.2	A few examples of built-in steps, models, and trainers available in DataDreamer.	50
Table 4.3	Information automatically recorded in a synthetic data card or synthetic model card.	55
Table 5.1	Accuracy on the STEL/STEL-or-Content (S-o-C) tasks. STYLEDISTANCE leads on both tasks among representations trained for content-independence, and STYLEDISTANCE _{SYNTH} generalizes remarkably well to real text data despite being trained only on synthetic data.	75
Table 5.2	We evaluate how well STYLEDISTANCE _{SYNTH} embeddings generalize to unseen style features by ablating features from the synthetic training dataset under three conditions: In-Domain, Out-of-Domain, Out-of-Distribution. We evaluate their performance on the STEL and STEL-or-Content (S-o-C) tasks.	76
Table 5.3	We reproduce the automatic formality transfer evaluation procedure from TinyStyler [187] on the GYAFC dataset [188]. $\rightarrow F$ corresponds to formal transfer, and $\rightarrow I$ corresponds to informal transfer.	79
Table 5.4	Outputs from TinyStyler, conditioned on Wegmann embeddings and STYLEDISTANCE embeddings.	80

Table 5.5	Metrics on the proportion of trajectories that successfully completed the task in the set of plausible trajectories kept in $\mathcal{P}$ after filtering out low-quality trajectories. Approximately 43% of trajectories in $\mathcal{P}$ successfully completed the task, up from ~7% with no filtering, indicating useful learning signal is introduced by filtering using self-critiques and information from the environment.	86
Table 5.6	A sample of the novel objectives generated compared with the objectives found in WebArena.	87
Table 5.7	Evaluation metrics on WebArena for baseline agents and self-improved agent models.	90
Table 5.8	Capabilities acquired and lost compared to the base agent model $\mathcal{M}$, along with the net change in the total number of capabilities demonstrated, for each self-improved fine-tuned agent model.	97
Table 5.9	Variants of the VERTEX _{DTW} score metric: 1) computed over all trajectories 2) weighting the trajectories by capability 3) weighting the trajectories by capability and filtering out trajectories for trivial tasks.	99
Table 5.10	Metrics on the proportion of trajectories that successfully completed the task in the set of plausible trajectories kept in $\mathcal{P}$ after filtering out low-quality trajectories for each iterative round of self-improvement. On the second round of self-improvement, we keep 131 plausible trajectories making our potential synthetic training dataset larger, however, the accuracy and P/R/F1 metrics indicate it would be a lower quality dataset to fine-tune on.	99
Table 5.11	Results on 7 text-rich benchmarks. The result of the best-performing open-source model is bold , and the second-best is <u>underlined</u> . Models with [†] stand for open data and code for multimodal training. Models with * are zero-shot models, which means the models are not trained on instances from any of the evaluation datasets.	111
Table 5.12	Results on human and machine-generated questions of ChartQA. The pie charts above display the percentage distribution of two question types in training and testing. Δ ($\downarrow$ lower is better) denotes the performance gap between human and machine questions.	117
Table 5.13	Compare image and text diversity across different chart datasets. We randomly sample 10K instances from each dataset to compute the results.	118
Table 5.14	Single vs. Multiple Rendering Tools for Data Generation. Each row uses the same number of 45K synthetic images. SINGLE only uses Matplotlib, while MULTIPLE involves four other rendering tools: HTML, LaTeX, Plotly, and VegaLite.	118

Table 5.15	Compare the LLMs used for synthetic data generation. For both LLMs, we create 100K synthetic charts for fine-tuning the VLMs. We report the zero-shot evaluation results on ChartQA.	120
Table 5.16	Click accuracy on ScreenSpot. We report our models trained on different pointing data. Human stands for using the human-annotated data from PixMo-point [177]. Combined means combining human-annotated data with our synthetic pointing data.	123
Table 6.1	Benchmark performance of FineInstructions and various baselines. We bold the strongest method and bold the win rate % if FineInstructions wins.	143
Table 6.2	1B+ FineInstructions classified into various categories.	145
Table 6.3	Incorporating a judging and filtering stage on top of FineInstructions leads to further improvements in performance.	146

LIST OF FIGURES

Figure 1.1	Classical AI vs. Machine Learning	2
Figure 1.2	LLMs can generate labeled training datasets that can then be used to help models learn with in-context learning, fine-tuning, or pre-training.	3
Figure 1.3	Inductive biases were applied by practioners in the form of synthetic examples in image classification datasets used to improve Convolutional Neural Network performance.	7
Figure 1.4	Human practitioners can apply useful inductive biases to steer the model toward the right task-specific patterns and reasoning. . .	9
Figure 1.5	A taxonomy of broad inductive biases and sources of knowledge that will be used to categorize various synthetic data approaches in this dissertation.	10
Figure 3.1	A visualization of the bootstrapping process.	33
Figure 3.2	Inductive biases and sources of knowledge highlighted by the work on improving the zero-shot machine translation performance of a model with synthetic data.	40
Figure 4.1	DataDreamer helps researchers implement many types of LLM workflows easier and makes reproducibility automatic and simple. These workflows often involve synthetic data generation with a LLM-in-the-loop and/or fine-tuning, aligning, and distilling models.	43
Figure 4.2	Various ways LLMs are used in synthetic data workflows.	44
Figure 4.3	Synthetic data workflows are becoming increasingly complex, where LLMs are often chained to process data over multiple stages or are used to judge and filter generated outputs. The resulting data from these stages are also used sometimes to perform fine-tuning, distillation, or self-distillation.	45
Figure 4.4	DataDreamer logs produced by the workflow in Example 1 when resuming from a prior interrupted run.	51
Figure 4.5	Inductive biases and sources of knowledge can be easily applied through complex synthetic data workflows that DataDreamer enables.	66
Figure 5.1	STYLEDISTANCE embeddings are trained using contrastive learning from synthetic parallel (positive and negative) examples representing 40 style features. The illustrated example is for the “Usage of Active Voice” feature.	69

Figure 5.2	We generate synthetic parallel examples to train STYLEDISTANCE for a wide range of style features in seven linguistic and stylistic categories.	70
Figure 5.3	A visualization of the STEL and STEL-or-Content tasks.	74
Figure 5.4	We generate synthetic data to fine-tune LLM agents to accomplish WebArena tasks such as “Add this product to my wishlist”. Step 1: We first collect an initial set of trajectories, filter out low-quality trajectories in an unsupervised fashion, and keep the remainder as synthetic in-domain examples. We prompt our base LLM to generate novel out-of-domain tasks along with hypothetical solution trajectories by providing a few in-domain examples. Step 2: We then fine-tune our base LLM agent on each of the three distinct synthetic training data mixtures and evaluate performance.	82
Figure 5.5	A selected full sample of a generated novel, out-of-domain synthetic training example in $\mathcal{D}_{\text{OUT-OF-DOMAIN}}$	88
Figure 5.6	Given a novel task (e.g., answering questions about nutrition facts), our code-guided generation system can produce targeted synthetic data to enhance the performance of VLMs on that specific task. .	101
Figure 5.7	The overview of our Code Guided Synthetic data generation system (CoSyn), which has 20 generation pipelines based on 11 render tools. Given a user query, e.g., “book cover,” CoSyn selects the appropriate pipelines and starts by generating diverse topics conditioned on personas, then synthesizes detailed data for code generation. The code renders the image and is also fed as context for an LLM to construct instruction-tuning data.	102
Figure 5.8	Our CoSyn-400K dataset consists of 9 categories of text-rich images with 2.7M instruction-tuning data.	107
Figure 5.9	An ablation on training data selection. Aux, Syn, and Eval stand for auxiliary, synthetic, and evaluation datasets, respectively. We report the average score on seven benchmarks.	112
Figure 5.10	Zero-shot performance on NutritionQA. The x-axis denotes the number of training examples used for the instruction-tuning stage. The models on the upper left side demonstrate better data efficiency. .	113
Figure 5.11	Examples from our newly collected NutritionQA dataset.	114
Figure 5.12	An ablation of using Chain-of-Thought reasoning. Short Answer represents prompting model to output the answer as short as possible. + CoT stands for providing Chain-of-Thought reasoning before giving the final answer.	116
Figure 5.13	Performance while scaling the size of synthetic data. We evaluate the <i>zero-shot</i> performance on ChartQA of models fine-tuned on increasing numbers of synthetic images.	119

Figure 5.14	An overview of enabling VLMs to point through synthetic data. (a) We synthesize pointing data by prompting an LLM to generate pointing questions and edit the code to draw the answer points explicitly. (b) We demonstrate that the VLM trained on synthetic pointing data can be generalized to real agentic tasks. 120
Figure 5.15	Randomly selected examples from our synthetic pointing data. 122
Figure 5.16	Inductive biases and sources of knowledge highlighted by the three examples discussed in this chapter that fine-tune on synthetic data to enhance a model’s performance. 124
Figure 6.1	The FineInstructions pipeline for efficiently generating diverse, pre-training scale, synthetic instruction-answer pairs. 131
Figure 6.2	An example of a prompt used to create silver-standard outputs to distill the Instantiator Model. 132
Figure 6.3	Illustrative example from the FineInstructions pipeline. 137
Figure 6.4	A visualization of the task diversity in FineInstructions. We also preview domain-specific diversity, comparing with a subset that was instantiated from medicine-related templates. 146
Figure 6.5	Inductive biases and sources of knowledge highlighted by the work on improving the broad general capabilities and performance of a model by applying synthetic data to the pre-training stage of LLM training. 148
Figure 7.1	The works presented in this dissertation demonstrated applying synthetic data at three levels of LLM learning: 1) In-Context Learning 2) Fine-Tuning 3) Pre-Training. 151

1 | INTRODUCTION

The evolution of machine learning (ML) over the past several decades has seen a transformation from rule-based expert systems to data-driven neural models [1]. Early ML systems relied on manually crafted rules and heuristics, which required significant domain knowledge and were limited in their flexibility. The advent of large datasets and powerful computational resources introduced a paradigm shift: instead of hardcoding knowledge, models could "learn" patterns directly from data. Neural networks, especially deep learning architectures, capitalized on this shift, allowing for complex representations that dramatically advanced performance across tasks from image recognition to natural language understanding. This revolution marked a shift away from handcrafted features and hardcoded rules toward automated feature learning, enabling the field to tackle previously intractable problems. Today, the reliance on data as the primary source of knowledge has cemented data-centric machine learning as the dominant paradigm in artificial intelligence.

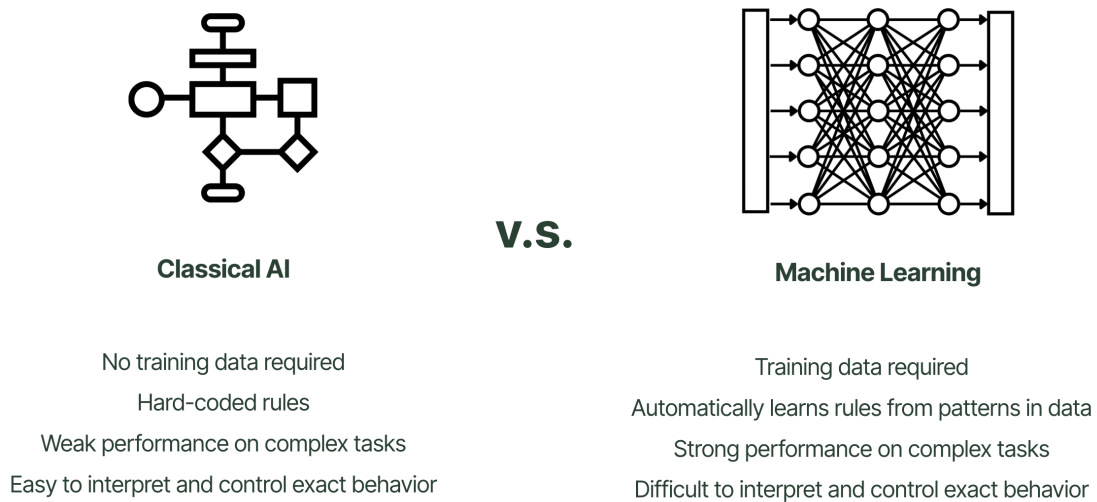


Figure 1.1: Classical AI vs. Machine Learning

An example of the success of data-centric machine learning are large language models (LLMs) [2, 3, 4, 5]. Large language models are trained in an unsupervised manner on vast amount of text data (books, web pages, documents, etc.) where the training objective is predicting the next word in a sentence given the prior context or in-filling words in a sentence that have been masked or hidden from the model in a stage known as “pre-training”. This stage allows models to learn the structure of language and form neural representations within the weights of the model. While these models trained to perform text-completion can complete some tasks like question-answering directly with a careful “prompt” to be completed (e.g. “*Question: Who is the president of the United States? Answer:*”), their primary utility is being used as a base model that is further “fine-tuned” on smaller, supervised datasets with input and output examples of task(s) of interest. When these models are “fine-tuned” specifically on a broad range of instruction-and-answer pairs in a stage referred to as “instruction-tuning” [6, 7, 8], they acquire broad, general capabilities leading to models that power chatbot-style assistant platforms like OpenAI ChatGPT or Google Gemini.

Synthetic data, in the context of LLMs, refers to training data generated by a language model that can serve as substitutes for real-world datasets [9, 10, 11, 12, 13, 14, 15, *inter alia*]. This data can be used for various purposes, such as annotation, evaluation, and full-generation of datasets for tasks like classification, generation, or reasoning [16]. Unlike traditional data collected from human input, synthetic data is generated automatically by LLMs and can be tailored to specific tasks or goals. This data generation capability allows researchers to create annotated examples at scale, bypassing the constraints and costs associated with human annotation [17, 18]. Furthermore, synthetic data is particularly valuable for tasks facing data scarcity, enabling the creation of large, diverse datasets where real-world examples may be limited [19]. In essence, synthetic data in the LLM era provides a flexible and scalable method of data generation that can be tuned to meet the requirements of different machine learning tasks, enabling experiments that would otherwise be prohibitive or impossible.

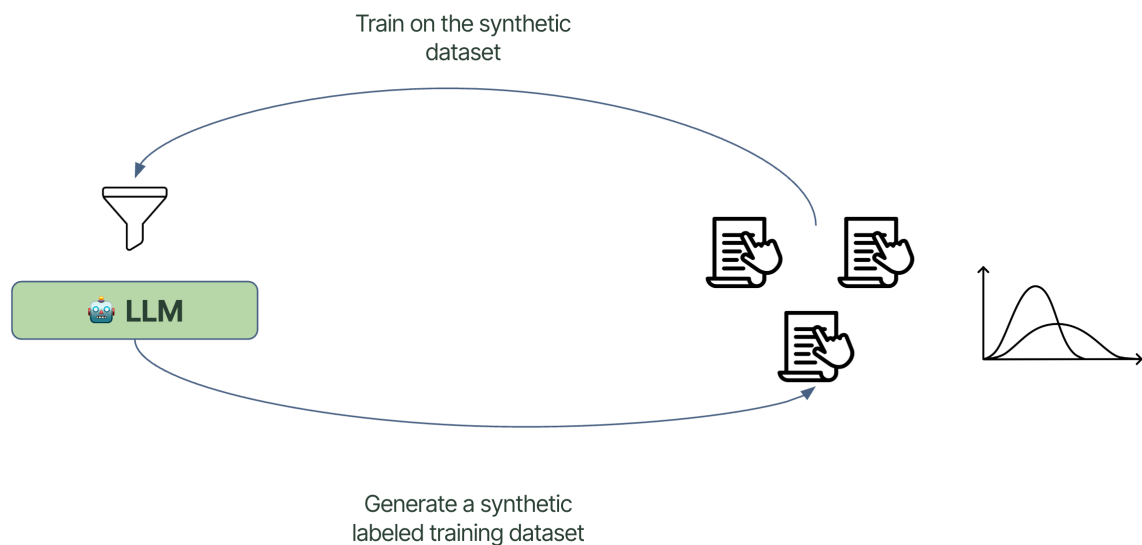


Figure 1.2: LLMs can generate labeled training datasets that can then be used to help models learn with in-context learning, fine-tuning, or pre-training.

Even before the advent of LLMs, synthetic data was used to enhance machine learning models, albeit in more limited ways. One early example is backtranslation, a technique where new unlabeled monolingual data in the target language is acquired and translated into the source language using a model trained in the reverse direction [20]. This creates synthetic parallel pairs that can be used for training in the forward translation direction. This approach has been widely applied in machine translation, where labeled data is often scarce for many language pairs. Another notable method was data augmentation in computer vision, where images were rotated, flipped, or distorted to artificially increase the size of training datasets [21]. While effective, these methods were constrained in their flexibility and scalability. The synthetic data created through these methods typically relied on straightforward transformations rather than the generation of entirely new, contextually coherent examples—a limitation that recent advances in language models have overcome.

With the development of large language models, synthetic data generation has entered a new era. LLMs can now generate coherent, contextually relevant data in massive quantities and at remarkable speed [22]. This capacity represents a pivotal moment in machine learning, as it enables practitioners to encode task-specific knowledge at scale, ultimately enabling new model capabilities through synthetic data in ways that were previously impossible. However, this ability also brings new challenges, as the synthetic data generated by LLMs is inherently shaped by the biases present in the models themselves [9].

1.1 CHALLENGES WITH USING SYNTHETIC DATA

A fundamental challenge with using synthetic data generated by a large language model (LLM) lies in the inherent tautology of the process: the model is effectively sampling from its own

learned distribution to improve itself or other models trained on similar distributions [23]. This setup can be problematic, as it raises questions about whether the generated data truly brings in "new" information or merely reinforces the model's existing biases and limitations. When a model generates data, it does so based on the patterns and distributions it has already learned, which means that any synthetic data produced is inherently constrained by those same learned representations. As a result, synthetic data risks falling into a closed feedback loop, where models trained on this data may be limited to the same biases, gaps, and misconceptions embedded in the original model's training distribution, rather than expanding or diversifying their understanding of the task at hand [24, 25].

DIVERSITY Another critical challenge in synthetic data lies in achieving adequate diversity within generated samples. LLMs, despite their scale and generative power, tend to produce outputs that reflect dominant patterns from their training data [9]. This can result in a lack of representation for rarer or less conventional examples, which are often crucial for robust model learning. If synthetic data fails to include a broad range of variations and edge cases, it risks overfitting the model to common patterns while neglecting the subtleties that enable long-tail task performance and generalization. Ensuring diversity in synthetic data is particularly challenging since models are more likely to default to high-probability sequences, leading to synthetic datasets that lack the necessary breadth to support models in learning from a true distribution.

MODEL COLLAPSE Synthetic data generation also risks contributing to model collapse, where the iterative generation of training data leads to a concentration of certain patterns and a corresponding erosion of long-tail knowledge [25]. Large language models possess extensive knowledge, including rare or niche information that may appear infrequently in natural data.

However, synthetic data generation, especially when applied repeatedly, may favor commonly seen or high-probability outputs over rare instances, effectively collapsing the distribution toward more "generic" knowledge. This collapse could lead to a gradual loss of diversity in the model's internal representations, where less frequently encountered, yet valuable, knowledge is progressively weakened. Consequently, the richness of long-tail knowledge—critical for tasks requiring nuanced or domain-specific understanding—may be diminished, reducing the overall effectiveness of the model when faced with complex or specialized tasks.

1.2 WHEN AND WHY DOES SYNTHETIC DATA IMPROVE MODEL PERFORMANCE?

Inductive biases refer to assumptions about a task that guide a learning algorithm in making predictions or decisions. Inductive biases are particularly helpful to deploy when there is limited task-specific data available. A practitioner may have prior knowledge about the distribution of inputs the model will likely encounter, the outputs it will need to produce, the general procedure to reach the prediction or decision for a task, or heuristics that help optimize task performance. By equipping a model with such biases, and this prior knowledge, the learning process is guided to ensure the model's learning aligns with the expected nature of the task [26].

Inductive biases can be introduced in two primary ways: through data transformations or by modifying the model's architecture or optimization routine. In computer vision tasks like image classification, certain image properties—such as scale, rotation, and perspective—are not relevant for distinguishing between objects. Practitioners can introduce knowledge about these properties by applying random transformations like rotation, scaling, or cropping to the

training images, ensuring the model learns invariant features [21]. Alternatively, biases can be encoded directly into the model's architecture. For example, convolutions in Convolutional Neural Networks (CNNs) are designed to capture spatial hierarchies, effectively encoding the assumption that local patterns (such as edges or textures) are invariant to transformations like translation or scaling [27]. In natural language processing (NLP), tasks often involve complex reasoning and understanding, which makes automatic data augmentation more challenging [28]. Instead, the pretrain-then-finetune paradigm has become dominant, where large language models (LLMs) are pre-trained on vast amounts of unlabeled data using self-supervised learning, then fine-tuned on a smaller, more representative set of labeled examples for the target task [2]. This paradigm leverages the knowledge encoded during pre-training to perform well across a wide range of tasks without needing task-specific manual data manipulation or architectural changes.

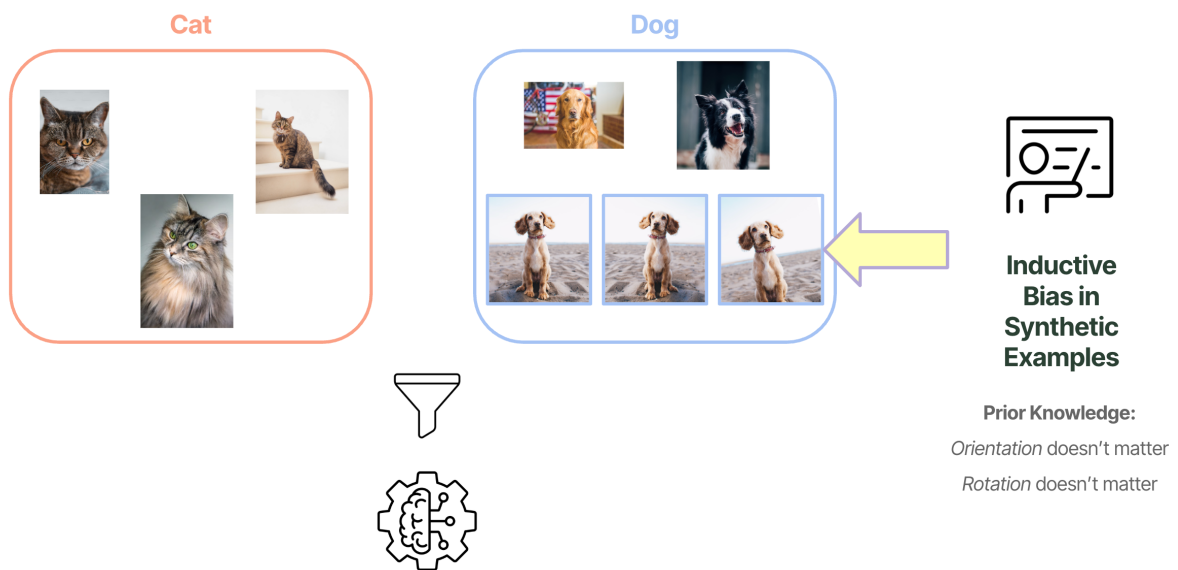


Figure 1.3: Inductive biases were applied by practioners in the form of synthetic examples in image classification datasets used to improve Convolutional Neural Network performance.

Although naively generating synthetic task data from large language models (LLMs) may lead to some of the challenges discussed earlier, such as degradation of performance on long-tail knowledge or model collapse, LLMs' strong instruction-following and generation capabilities have enabled practitioners to apply helpful inductive biases in synthetic data workflows. By carefully guiding synthetic data generation and filtering and selecting specific synthetic examples, practitioners can use LLMs to encode prior assumptions and task-specific knowledge, improving model performance at scale. This allows models to learn from synthetic data that encodes a helpful learning bias, steering the model toward the right task-specific patterns and reasoning. Uniquely, using synthetic data in neural model training allows these models to harness one of the main advantages of expert systems and other classical AI approaches: the ability to direct a model's predictions and decision-making processes. However, unlike expert systems, neural models are not bound to rigidly follow these rules; instead, they are free to incorporate and learn from other patterns available in the data. Synthetic data, therefore, can act as a guiding "hint" that complements natural data or the model's pre-trained knowledge. For the first time, synthetic data generated by LLMs allows practitioners to easily influence the learning of these black-box, data-centric neural models at the source of where they absorb and integrate their knowledge.

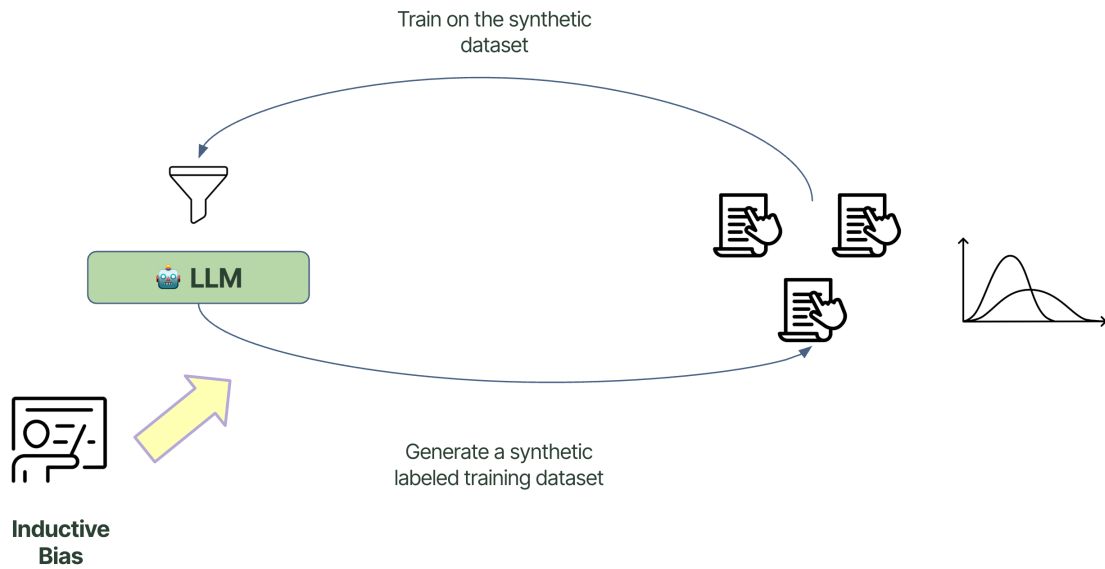


Figure 1.4: Human practitioners can apply useful inductive biases to steer the model toward the right task-specific patterns and reasoning.

In this section, I will introduce a taxonomy to broadly capture the emerging and common **inductive biases** used in synthetic data workflows in my own research and the field more broadly. When applied appropriately, these biases are effective in steering model learning and enhancing model performance. To better understand why synthetic data works, I will also attempt to explicitly identify the **source of knowledge** from which the model learns when these biases are applied. This exploration will clarify how learning with synthetic data can lead to the expansion of a model’s capabilities and knowledge, rather than causing collapse.

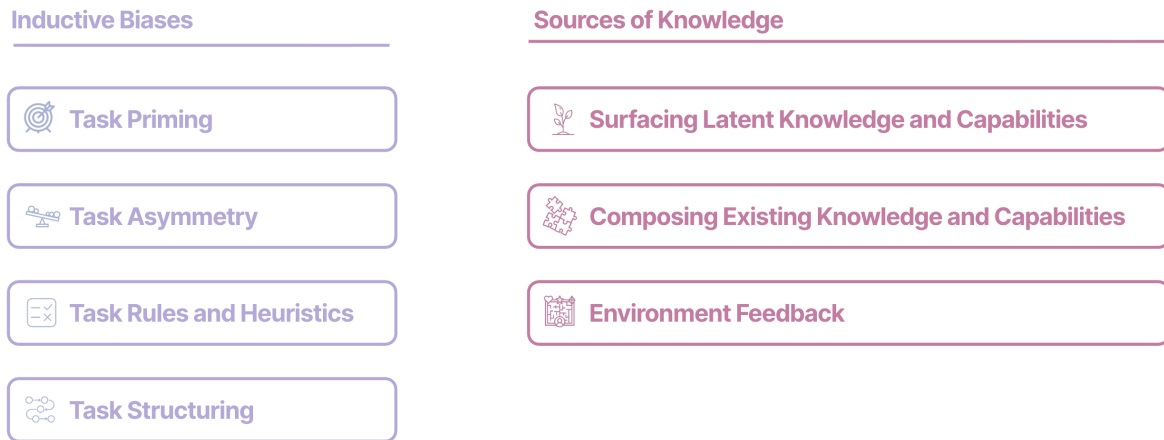


Figure 1.5: A taxonomy of broad inductive biases and sources of knowledge that will be used to categorize various synthetic data approaches in this dissertation.

1.2.1 Inductive Biases

I outline inductive biases that broadly categorize synthetic data approaches used in my research and other work in the field. In Chapters 3, 5, and 6, I concretely demonstrate applying these inductive biases in synthetic datasets used to train and enhance models.

1. **Task Priming:** Task priming is the inductive bias that applies prior knowledge of the specific task a model is expected to complete, conditioning a general model for more specialized performance. This is often achieved through in-context examples or fine-tuning on task-specific data, which signal the type of task being asked of the model, enhancing its focus and aligning its computation toward effectively performing that task [29, 30, 19, 31]. By narrowing the potential range of outputs to those most relevant to the task, task priming can also constrain the outputs of a more general model, leading to better performance and improved task alignment.

2. **Task Asymmetry:** Task asymmetry is an inductive bias that is indirectly applied to learning during the synthetic data generation or synthetic data filtering stages. Recognizing inherent or practical differences in the complexity of task subcomponents or directions or leveraging prior knowledge that one part of a task may be easier than another [32, 33], it becomes possible to generate synthetic examples or filter synthetic examples to a high-quality subset, both of which improve learning. For example, validating the quality of a task outcome (e.g., assessing a summary) might be simpler than generating the outcome itself (e.g., writing a summary). This insight can be used to transfer knowledge from the easier subcomponent to guide learning in the more challenging one [34].
3. **Task Rules and Heuristics:** This is an inductive bias that incorporates prior knowledge of simplified principles or strategies that can guide effective task performance. By embedding these rules or heuristics into synthetic data, models are exposed to structured examples that enhance their ability to execute and generalize effectively [21]. When new examples are encountered where such rules or heuristics may be beneficial, the model can leverage this embedded knowledge to navigate the task more adeptly.
4. **Task Structuring:** Task structuring is the inductive bias that leverages prior knowledge of useful intermediate steps or systematic approaches that contribute to optimal performance on a task [35]. By incorporating this process structure into learning, models are encouraged to adopt a more organized method of problem-solving and decision-making, thereby enhancing their overall execution capabilities [36].

1.2.2 Sources of Knowledge and Learning

One significant source of model learning when training on synthetic datasets carefully crafted to elicit a helpful learning bias originates from the task-specific assumptions embedded in their creation. I outline additional avenues of knowledge and learning that models can leverage when trained on synthetic datasets.

1. **Surfacing Latent Knowledge and Capabilities:** Large language models (LLMs) may already possess knowledge or capabilities stored as latent information from their pre-training on vast amounts of data [2]. This pre-training allows LLMs to memorize a wide range of uncured, scattered examples, accessible through strong retrieval capabilities [37]. In situations with limited high-quality labeled data, LLMs can surface useful, one-off examples from their pre-training [19]. While these examples may not fully elicit task-specific capabilities by pre-training on them, they can be synthesized into curated synthetic datasets [38, 39, 40]. These datasets can then better guide model performance through in-context learning or fine-tuning, which offer a more effective approach to task-specific learning than relying solely on the original language modeling task during pre-training.
2. **Composing Existing Knowledge and Capabilities:** Models might excel at certain sub-skills or subcomponents of a task, which can be orchestrated and combined to form a capability for a new, more complex task [35, 41]. By generating synthetic data that requires the integration of these distinct subskills or two pieces of existing knowledge, models can be trained to compose their existing abilities into a cohesive performance, enhancing their adaptability and task execution in novel scenarios.

3. **Environment Feedback:** The generation and refinement of synthetic examples can be informed by feedback from the model’s interaction with an environment [42]. This feedback loop allows models to learn from past successes and failures, adapting their behavior based on observed outcomes to iteratively improve their performance on target tasks [43, 44].

1.3 THESIS STATEMENT

This thesis argues synthetic training data is most effective at enhancing models when it is used as an inductive bias. My research demonstrates that when synthetic training data is carefully constructed to incorporate useful learning biases relevant to a given task, model performance can significantly improve, enabling new capabilities. I demonstrate that this approach is not limited to enhancing performance on a single task; rather, applying an inductive bias through synthetic data in the pre-training phase can enrich the general knowledge and skill acquisition of LLMs as they undergo their foundational learning process.

1.4 DISSERTATION OUTLINE

This dissertation is structured as follows:

Chapter 3 presents an initial case study demonstrating how synthetic data can improve model performance in an in-context learning setting. Leveraging a self-consistency prior, I bootstrap synthetic machine translation examples to use as few-shot examples to enhance the

machine translation performance of a LLM, showcasing the potential of synthetic data in driving targeted improvements with a simple procedure.

Chapter 4 introduces DataDreamer, a Python library designed to simplify multi-stage LLM-based synthetic data generation and training workflows. By reducing the technical expertise required, DataDreamer makes it easier for practitioners to implement and reproduce synthetic data procedures, enabling them to generate customized datasets and apply inductive biases through training to improve model performance.

Chapter 5 showcases a range of synthetic data workflows built with DataDreamer, focusing on how task-specific learning biases can be embedded through fine-tuning. This chapter explores larger-scale synthetic data generation, demonstrating how synthetic data can be used to train stronger embedding models, agent models, and vision-language models—capabilities that would be challenging to develop given the limited availability of curated natural datasets for these tasks. In Section 5.1, I generate synthetic contrastive triplets used to train style embeddings (text embeddings that embed two texts of the same style closely regardless of content) to create stronger, content-independent style embeddings. Without synthetic data, contrastive triplets for training such text style embeddings previously had to be mined in imperfect ways leading to weaker style embeddings that mixed style and content representations. In Section 5.2, we generate a synthetic dataset of actions sequences (clicking, scrolling, typing) to self-improve an LLM agent that performs assistant-style tasks on websites (“*Add this product to my wishlist*”). In Section 5.3, we generate synthetic text-rich images (charts, figures, tables, diagrams, documents) along with questions and answers about those images to train stronger vision language models that can assist users with analyzing such images that might appear in scientific publications or news articles.

Chapter 6 demonstrates applying synthetic data to enhance LLM pre-training. This approach demonstrates that synthetic data can move beyond task-specific improvements, support-

ing a more generalized enhancement of LLM capabilities. I develop and benchmark a technique where I use a LLM to optimize and restructure pre-training data from raw documents to instruction-answer pairs in order to be more closely aligned with the expected distribution downstream usage (questions, instructions, and tasks users may ask). This restructuring of the pre-training data and objective allows “pre-training” in a supervised fashion on instructions and answers instead of the typical, self-supervised next token prediction objective. This yields broad performance gains across multiple tasks and benchmarks.

1.5 RELEVANT PUBLICATIONS PRESENTED

This dissertation will present my relevant research publications published at a number of conferences. I am a primary author on these publications completed at the University of Pennsylvania in collaboration with university colleagues and, at times, external collaborators. Detailed summaries of my contributions are provided at the end of each relevant chapter or section.

1. (Patel et al., 2023) Ajay Patel, Bryan Li, Mohammad Sadegh Rasooli, Noah Constant, Colin Raffel, and Chris Callison-Burch. Bidirectional Language Models Are Also Few-shot Learners. *In ICLR 2023*.
2. (Patel et al., 2024) Ajay Patel, Colin Raffel, and Chris Callison-Burch. DataDreamer: A Tool for Synthetic Data Generation and Reproducible LLM Workflows. *In ACL 2024*.
3. (Patel et al., 2023) Ajay Patel, Delip Rao, Ansh Kothary, Kathleen McKeown, and Chris Callison-Burch. Learning Interpretable Style Embeddings via Prompting LLMs. *In EMNLP 2023 (Findings)*.

* denotes co-first authorship

4. (Patel et al., 2024) Ajay Patel, Markus Hofmarcher, Claudiu Leoveanu-Condrei, Marius-Constantin Dinu, Chris Callison-Burch, and Sepp Hochreiter. Large Language Models Can Self-Improve At Web Agent Tasks. *Manuscript*.
5. (Patel et al., 2024) Ajay Patel*, Jiacheng Zhu*, Justin Qiu, Zachary Horvitz, Marianna Apidianaki, Kathleen McKeown, and Chris Callison-Burch. StyleDistance: Stronger Content-Independent Style Embeddings with Synthetic Parallel Examples. *In NAACL 2025*.
6. (Yang et al., 2025) Yue Yang*, Ajay Patel*, Matt Deitke, Tanmay Gupta, Luca Weihs, Andrew Head, Mark Yatskar, Chris Callison-Burch, Ranjay Krishna, Aniruddha Kembhavi, Christopher Clark. Scaling Text-Rich Image Understanding via Code-Guided Synthetic Multimodal Data Generation. *In ACL 2025*.
7. (Patel et al., 2025) Ajay Patel, Colin Raffel, Chris Callison-Burch. FineInstructions: A Web-Scale Instructions Dataset. *Working manuscript*.

In this dissertation, I will also briefly touch upon a few publications that I have co-authored and contributed to, but that are not my primary work.

1. (Horvitz et al., 2024) Zachary Horvitz, **Ajay Patel**, Kanishk Singh, Chris Callison-Burch, Kathleen McKeown, and Zhou Yu. TinyStyler: Efficient Few-Shot Text Style Transfer with Authorship Embeddings. *In EMNLP 2024 (Findings)*.
2. (Deitke et al., 2024) Matt Deitke, Christopher Clark, Sangho Lee, Rohun Tripathi, Yue Yang, Jae Sung Park, Mohammadreza Salehi, Niklas Muennighoff, Kyle Lo, Luca Soldaini, Jiasen Lu, Erin Bransom, Kiana Ehsani, Huong Ngo, **Ajay Patel**, Mark Yatskar, Chris Callison-Burch, Andrew Head, Rose Hendrix, Favyen Bastani, Eli Vanderbilt,

* denotes co-first authorship

Nathan Lambert, Jenna Sparks, Sam Skjonsberg, Evan Pete Walsh, Piper Wolters, Tanmay Gupta, Kuo-Hao Zeng, Dirk Groeneveld, Crystal Nam, Carissa Schoenick, Oscar Michel, Ranjay Krishna, Luca Weihs, Noah A. Smith, Hannaneh Hajishirzi, Ross Girshick, Ali Farhadi, Aniruddha Kembhavi. Molmo and PixMo: Open Weights and Open Data for State-of-the-Art Multimodal Models. *In CVPR 2025*.

These publications are licensed under Creative Commons 4.0 BY, listed below. Therefore, parts of the relevant discussions are quoted directly from said publications, with the explicit approval of all primary co-authors, my thesis committee, and the Graduate Group Chair. Only one of these of these publications have been or will be extensively discussed in any of my collaborators' theses as a subsection within a larger chapter. All work was completed jointly with collaborators at the University of Pennsylvania, University of North Carolina at Chapel Hill, HuggingFace, Google, Columbia University, ExtensityAI, Johannes Kepler University Linz, NXAI, and the Allen Institute for Artificial Intelligence.

2 | BACKGROUND

In this chapter, I will introduce some relevant background and highlight prior and current approaches to contextualize the work presented in later chapters. I will first discuss early applications of synthetic data in machine learning and natural language processing. I then connect how the particular limitations of earlier synthetic data methods renewed interest in synthetic data strategies when more advanced, large language model (LLM) generation approaches became available. Following this, I will introduce large language models in-depth, the main models the work in this dissertation will focus on, and how they are used and trained. With that background, I will discuss prior and current synthetic data approaches using LLMs. Finally, I will provide background on some of the various machine learning tasks and problems discussed in this dissertation.

2.1 EARLY APPLICATIONS OF SYNTHETIC DATA

Prior to LLMs, machine learning primarily focused supervised learning with curated or mined datasets. Some early approaches, however, did utilize procedurally-generated data from programs or model-generated data. These approaches were early applications of synthetic data to machine learning that often were devised to overcome limited data resources.

2.1.1 Automatic Dataset Augmentation

In Chapter 1, we discussed how the field of natural language processing used backtranslation, a technique where new unlabeled monolingual data in the target language is acquired and translated into the source language using a model trained in the reverse direction [20]. This approach is a form of dataset augmentation with model-generated translation pairs. On another sequence-to-sequence task similar to machine translation, text style transfer, a lack of parallel examples can be overcome by reformulating the style transfer task as a paraphrasing task whereby a model-generated “neutral paraphrase” of a styled text can serve as the input in input/output training pairs [45].

A more sophisticated direction of dataset augmentation involves using seed training data and model predictions to mine and collect further examples for learning in a bootstrapping manner in a process broadly referred to as “weak supervision”. Early work in this direction includes Yarowsky [46], who iteratively expanded labeled examples for word sense disambiguation by adding high-confidence model predictions to the training set. More recent frameworks, such as Snorkel [47], generalize this idea by combining multiple noisy labeling functions to generate large-scale training data, enabling models to learn effectively from imperfect supervision.

2.1.2 Programmatically Generated Datasets

Distantly related to synthetic data, several early NLP benchmarks were constructed by programmatically generating examples from templates or rules, offering controlled alternatives to natural corpora. The Facebook bAbI dataset [48] generates synthetic stories and questions to target distinct reasoning skills, SCAN [49] pairs navigation commands with action sequences to study compositionality, and CLUTRR [50] creates family relation stories governed by log-

ical rules. These resources illustrate how rule-based generation provided scalable datasets for probing reasoning and generalization.

2.1.3 Environment Simulators

Interactive simulators use programs to generate dynamic interactive environments and task trajectories. SHRDLU [51] demonstrated early grounding of language in a blocks-world simulation, while TextWorld [52] and ALFWorld [53] generate text-based environments for sequential decision-making and planning. Such simulators show how synthetic interaction data has been used to study reasoning and generalization in multi-step tasks.

2.1.4 Knowledge Transfer and Distillation

Model-generated data can also be used to train new models as a form knowledge transfer referred to as “distillation” [54], where a teacher model is used to generate training data for a student model. Model distillation can be an efficient way to train smaller, more efficient models while maintaining similar or approximate levels of performance. Some works have explored “self-distillation” where model-generated data is used to train the generating model itself and they find this can lead to performance improvements for some tasks [29, 30].

2.2 LIMITATIONS OF SYNTHETIC DATA

Early synthetic data approaches faced significant limitations in their representativeness and generalizability. Datasets created through programmatic rules or small grammars such as bAbI, SCAN, or CLUTRR provided precise control over linguistic and logical structures but lacked the richness and diversity of natural data. The resulting models often learned narrow heuristics tied to the generator’s design rather than acquiring robust, generalizable reasoning or language understanding skills [23, 24]. This gap meant systems that performed well on synthetic benchmarks often failed to transfer their capabilities to natural settings.

Another major concern in synthetic data generation, particularly when models are used to create their own training inputs, is model collapse [25], which was previously discussed in Section 1.1. When synthetic data is iteratively generated from a model and reused for training without sufficient diversity or external grounding, the model may converge toward degenerate distributions that reinforce its own errors and biases. Such collapse can manifest as reduced output diversity, loss of factual accuracy, or the amplification of spurious correlations. It can also propagate and amplify pre-existing biases present in either the generating model or the rules governing the generation process [25]. For example, if a model reflects gender, cultural, or topical biases in its outputs, synthetic data derived from it may further entrench these biases in downstream systems. Models trained exclusively on such data may overfit to such simplified or idealized distributions, leading to unrealistic confidence and poor calibration when confronted with real-world data [55].

These limitations motivated later research on generating higher-fidelity synthetic data that better mirrors the diversity, ambiguity, and contextual depth of real human examples. The emergence of large language models, capable of producing increasingly natural and semanti-

cally coherent text, has renewed interest in synthetic data approaches which we turn to in the following section to discuss in-depth.

2.3 LARGE LANGUAGE MODELS

Large Language Models (LLMs) are large-scale neural networks trained to model the statistical structure of natural language. They are typically built on the transformer architecture, introduced by Vaswani et al. [56], which relies on self-attention mechanisms to capture long-range dependencies between words in text more effectively than earlier recurrent or convolutional architectures. LLMs are trained on large text corpora containing hundreds of billions of tokens and are drawn from sources like web pages, books, and social media. The training objective is usually autoregressive language modeling or “next token prediction”, where the model predicts the next token in a sentence given the prior context [4]. Masked language modeling and other variants of this task are also sometimes used [2, 3]. Scaling model size, dataset size, and compute has been shown to yield predictable improvements in performance across tasks, a phenomenon formalized in neural scaling laws [57].

The development of LLMs has progressed through various stages of scale and compute. There were early contextual word embeddings such as ELMo [58], bidirectional transformers such as BERT [2], autoregressive models such as GPT-2 [4], and sequence-to-sequence transformers such as T5 [3]. Later, with more data and compute, came larger models such as GPT-3 [5], PaLM [59], and GPT-4 [60]. These models demonstrated emergent capabilities not present in smaller models, including multi-step reasoning, code generation, and few-shot task generalization. As a result, LLMs have become foundational to modern natural language processing and broader AI applications.

2.3.1 Pre-Training

Pre-training refers to the initial large-scale training phase where the model learns general-purpose language representations. For autoregressive LLMs, the pre-training objective is to maximize the likelihood of the next token in a sequence, enabling the model to capture linguistic patterns, factual knowledge, and semantic relationships. Masked language modeling, as used in BERT [2], instead trains the model to reconstruct randomly masked tokens from their surrounding context. These objectives require no explicit human labeling and thus allow models to leverage enormous web-scale corpora. The pre-training process is computationally expensive, often requiring thousands of GPUs over weeks or months. The scale of pre-training has proven central to downstream performance as demonstrated by Kaplan et al. [57] as increasing model size and dataset size in tandem yields smooth power-law improvements in accuracy, perplexity, and generalization.

2.3.2 Prompting and In-Context Learning

Prompting is a technique of conditioning a pre-trained LLM to complete a certain context carefully constructed to yield a desired answer such as “*Question: Who is the president of the United States? Answer:*”. GPT-2 [4] and GPT-3 [5] demonstrated that large models could perform new tasks with little or no fine-tuning through in-context learning. In-context learning involves placing task descriptions and a few input–output examples into the prompt. GPT-3 demonstrated strong in-context learning capabilities and the ability to generalize to unseen task inputs without parameter updates. This demonstrated that LLMs encode a form of meta-learning, using the prompt as an implicit training signal [61, 62, 63]. Prompting strategies have since expanded to include chain-of-thought prompting [64], which elicits step-by-step rea-

soning before an answer, and retrieval-augmented prompting [65], which supplements model knowledge with external sources.

2.3.3 Fine-Tuning

Fine-tuning adapts a pre-trained LLM to narrower domains or specific downstream tasks by continuing training on a curated dataset. Traditional supervised fine-tuning updates all parameters of the model, often yielding state-of-the-art results on benchmarks such as question answering, summarization, and classification [2, 3, 4, 5]. However, full fine-tuning of billion-parameter models is resource-intensive, leading to the development of parameter-efficient fine-tuning methods such as adapters [66], prefix-tuning [67], and LoRA (Low-Rank Adaptation) [68]. These approaches allow models to specialize with significantly lower computational cost while retaining general-purpose capabilities.

2.3.4 Instruction-Tuning

Instruction-tuning is a form of fine-tuning designed to improve a model's ability to follow natural language instructions. Instead of being trained on a single downstream dataset, the model is fine-tuned on a mixture of tasks presented in the form of instruction–response pairs. This was pioneered in T5 [3], which reframed multiple NLP benchmarks into a unified text-to-text format. Later, InstructGPT, FLAN, and others [6, 8, 7] demonstrated that instruction-tuning on diverse collections of tasks significantly improved zero-shot and few-shot generalization. By aligning the training format with typical user interactions, instruction-tuned models are better able to handle open-ended tasks in prompts.

2.3.5 Alignment and Preference-Tuning

Although instruction-tuned models are more usable, they may still produce unhelpful, biased, or unsafe outputs. Alignment methods address this by steering models toward outputs that better reflect human preferences. Reinforcement learning from human feedback (RLHF) has become the dominant approach [69, 6]. In RLHF: 1) human annotators rank multiple sampled outputs from a model, 2) a reward model is trained to predict these human preferences, and 3) the LLM is fine-tuned with reinforcement learning to maximize the reward assigned by the reward model. This procedure was used in InstructGPT [6] and later in ChatGPT, establishing a commonly adopted standard for aligning LLMs. More recently, direct preference optimization (DPO) [70] and related methods have been introduced as simpler and more stable alternatives to RLHF. Alignment and preference-tuning remain active areas of research, as models are increasingly deployed in real-world contexts.

2.4 SYNTHETIC DATA APPROACHES WITH LARGE LANGUAGE MODELS

Recent work has explored using LLMs to create synthetic data for tasks or to augment existing datasets to boost task performance [9, 10, 11, 12, 13, 14, 15, *inter alia*]. These approaches vary vastly by procedure and task but typically involve zero-shot or few-shot prompting and utilizing the generative nature of LLMs to generate synthetic text corpora or annotate existing datasets. A study by Gilardi, Alizadeh, and Kubli [17] found the zero-shot performance of ChatGPT exceeded crowd workers by about 25%, making LLMs a promising alternative to

expensive human annotation for simpler labeling tasks within their capability set. He et al. [71] introduced a general-purpose framework called “Generate, Annotate, and Learn” for task-specific unlabeled text generation and achieved strong results on the GLUE benchmark [72] of NLP tasks.

Synthetic data has also been applied the various stages of training a LLM. For example, LLMs have been used to generate synthetic instruction-tuning datasets from some seed examples or templates for the purposes of instruction-tuning other pre-trained language models [73, 74, 75]. Other works have also applied synthetic generation to reasoning and explanation data as part of instruction-tuning by producing synthetic explanations or reasoning steps [76, 64, 77]. At the pre-training level, a few works demonstrate synthetic data as a promising technique for training small, performant models [22, 78] by synthetically generating pre-training sized corpora of various topics (math, science, history, etc.) entirely with LLMs.

2.4.1 Encouraging Diversity

One main challenge with these generation strategies is that LLMs often suffer from mode collapse and low diversity when sampling [24]. There are a few general approaches to address this. A common approach is to condition generations on a set of sampled attributes [9] in the prompt that are changed for every inference of the model or by conditioning on a description of a persona or perspective [79] to elicit diversity of viewpoint similar to surveying a diverse audience. Other approaches have explored utilizing and manipulating the probability distribution of output tokens to enhance diversity [80].

2.4.2 Refining Datasets and Self-Improving

There are a number of approaches that combine synthetic data with a feedback signal (either human or artificial) to filter the synthetic data to a higher-quality subset before training. Without direct human supervision, reward models, models trained to rate the quality of model outputs [6], are a popular choice to provide this kind of signal. Other approaches involve using LLMs to self-critique and judge the quality of their own outputs [81, 40, 82, 83, 84].

2.5 TASKS DISCUSSED IN THIS DISSERTATION

Chapters 3, 5, and 6 discuss various tasks I have applied synthetic data to in my work. Below, I introduce background on each task and relevant prior work.

2.5.1 Machine Translation

In Chapter 3, we begin with applying synthetic data to machine translation. Machine translation is the task of translating from one language to another with a machine learning model [85]. When we are translating from or to a language that may not have a large amount of data available for it, we typically refer to this task as low-resource machine translation [86]. As discussed previously, backtranslation is a technique in machine translation that helps improve performance by training on pairs created by reversing model-generated translations [20]. Early works in training LLMs explored using machine translation as a benchmark task [4, 3, 87]. Han et al. [19] uses LLMs and prompting to perform unsupervised machine translation by perform-

ing rounds of bootstrapping translation examples, a synthetic data procedure which I adapt and expand upon in Chapter 3.

2.5.2 Text Style Embeddings

In Section 5.1, I train style embeddings, text embeddings that embed two texts closely together if they share the same style regardless of content. Style representations are typically learned from unlabeled texts [88, 89, 90, *inter alia*]. Due to the lack of parallel datasets covering diverse style features, Hay et al. [91] and Wegmann, Schraagen, Nguyen, et al. [92] have both used authorship as a proxy for style as we discussed previously. Rivera-Soto et al. [93] trained embeddings to uniquely represent different authors. Although these embeddings capture features representative of authors’ style, they also capture content features due to the lack of control for content-related aspects. Patel et al. [94] trained LISA, a style vector created by annotating texts with their stylistic features using LLMs. LISA, however, trades off performance to create a vector with interpretable dimensions.

2.5.3 Agent Models

In Section 5.2, I train agent models, LLMs specifically trained to complete tasks by performing a sequence of actions towards a goal, and evaluate on the WebArena benchmark [95] of assistant-style tasks to complete on websites. With limited to no supervised action sequence data to train such a model, we implement a self-improvement procedure to improve model performance in an unsupervised fashion by training on self-generated synthetic data. Previously, Bousmalis et al. [96] demonstrated self-improving embodied agents for complex robotics tasks.

Gulcehre et al. [97] introduced a method for self-improving agents on a simpler multi-step question answering task. Song et al. [98] proposed a procedure of filtering trajectories and fine-tuning, but primarily focuses on supervised filtering, does not explore generating novel tasks and synthetic data, and evaluates on less realistic and complex benchmarks. Pan et al. [99] explored using vision models for critique to improve on WebArena.

2.5.4 Vision-Language Models

In Section 5.3, we train vision-language models, multimodal models that can take both images and text as input, to better understand text-rich images (charts, figures, diagrams, tables, etc.). Generating synthetic images with annotations grounded in known source representations (like code used to generate the figure or the tabular data) has been widely used in domains with limited vision-language data [100, 101, 102, 103]. This approach has been applied to chart and plot vision question-answering, typically using a limited, small set of chart types and by instantiating handcrafted question templates [104, 105, 106, 107]. Following this, Li and Tajbakhsh [108] and Carbune et al. [109] explore using text-only LLMs to generate annotations from tables or text descriptions associated with charts to train VLMs. Other recent approaches explored generating data and code to render synthetic charts [110, 111, 112, 113].

2.5.5 Pre-Training and Instruction-Tuning

In Chapter 6, we pre-train a model on synthetic pre-training data by converting standard pre-training documents into a large number of synthetic instruction-answer pairs. While large language models are primarily trained on data sampled from web pages, books, and other natural

sources, a number previous works have explored incorporating synthetic training data generated by an LLM itself as part of the pre-training process [114, 78, 115, 116, 117]. Other approaches have looked at filtering documents [118, 119, 120, 121] with LLM judgments of quality or re-weighting and re-sampling the mixture of documents [63, 122, 123, 120, 124, 125]. Recent approaches have applied more sophisticated techniques, converting vanilla pre-training documents into instruction-answer pairs and other task formats [126, 121, 127] or rephrasing low-quality documents into a higher-quality, clean document [128, 121, 129]. In our work, we generate this synthetic training data by instantiating instruction templates we create. Other work has explored synthetically generating instruction-tuning data and reasoning data with LLMs and prompting [130, 40, 131, 132, 133, 134, 135, 136, 137, 138, 139]. Köksal et al. [140] generate instruction-answer pairs for long-form generation, pairing pre-training documents with synthetic instructions that would plausibly generate them. Using instruction templates is also common approach to creating instruction-tuning datasets [141, 7, 8, 142, 143, 144]. These approaches typically use instruction templates that are handcrafted or crowdsourced, resulting in a far smaller number of instruction templates, lower diversity, and fewer total example instances compared to our work.

3 | ENHANCING MODEL PERFORMANCE USING SYNTHETIC DATA AND IN-CONTEXT LEARNING

Large language models possess few-shot learning capabilities and zero-shot instruction following capabilities, despite only being pre-trained with a self-supervised causal language modeling objective (which is to predict the next token) [4, 5]. When performing in-context learning with these models, supervised examples of the task are often provided. In machine translation, it can be extremely challenging to get parallel texts between two low-resource languages. Therefore, machine translation is a useful task that can serve as a case study to demonstrate how synthetic data combined with in-context learning can be an effective strategy for tasks where supervised datasets are lacking.

In this chapter, we demonstrate how LLMs can perform zero-shot, unsupervised machine translation using synthetic, bootstrapped parallel examples for in-context learning instead of real examples. As a case study, this work demonstrates how synthetic data can be effectively utilized to enhance a model’s performance on a given task. Interestingly, we find our unsupervised zero-shot approach even outperforms a supervised two-shot setting, suggesting synthetic data may be useful to deploy even when supervised data is available.

3.1 EXPERIMENTAL SETUP

We empirically demonstrate a procedure where we use the mT5_{3.7B} [87] model, a bidirectional LLM, to synthesize its own synthetic parallel examples that can enhance its machine translation performance. The synthetic parallel examples are bootstrapped in a completely unsupervised fashion using a zero-shot translation prompt with no examples. The zero-shot prompt format looks like:

Translate Spanish to English. Spanish: Los árboles son importantes.</s> English: <X>
--

We use the SAP (Sequential Autoregressive Prompting) method [145] to enable prompting and in-context learning since this model is trained on a bidirectional objective and does not naturally generate text. We adapt the bootstrap process of Han et al. [19] to retrieve these synthetic parallel examples. The process, as depicted in Figure 3.1, consists of three steps:

Step 1 (sampling): Generate synthetic parallel examples using a zero-shot translation prompt (with no examples) to translate sentences from a monolingual source language corpus.

Step 2 (filtering): Filter out low-quality synthetic examples to keep only high-quality, self-consistent synthetic examples using an unsupervised scoring technique (discussed in Section 3.1.1).

Step 3 (self-amplification): Translate any source language sentence desired using these synthetic parallel examples in the few-shot prompt.

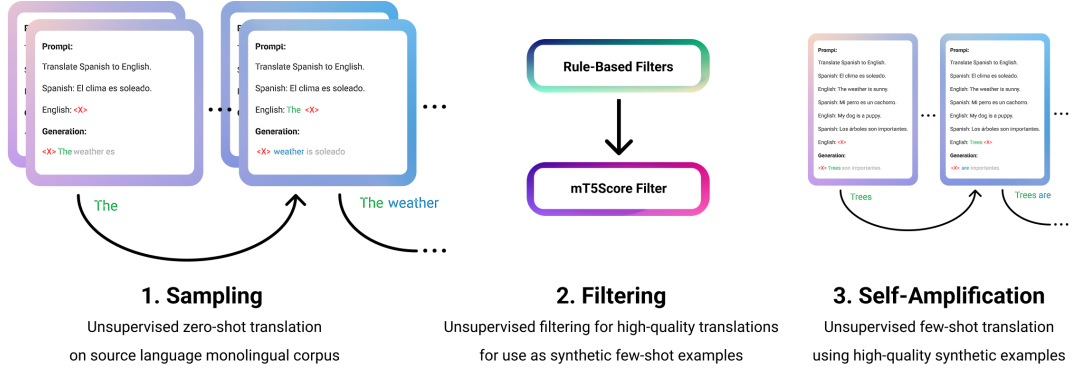


Figure 3.1: A visualization of the bootstrapping process.

We iteratively run multiple rounds of this bootstrap by repeating step 2 and step 3 to form a better few-shot prompt. The few-shot prompt after self-amplification is used to translate more source language sentences. These are then filtered using the scoring technique used in step 2 and so on. In our experiments, we run four bootstrapping rounds and sample 100 source language sentences from a training dataset in each round. Note that none of the target language parallel sentences from the training dataset are used in this zero-shot setting; following Han et al. [19], only the source language sentences are used.

3.1.1 Filtering Down to High-Quality, Self-Consistent Translations

The filtering step of the bootstrap requires an unsupervised scoring method for assessing the quality of translations. We first use `langdetect`,¹ a language identifier, as a simple rule-based filter to ensure the generated text is in the desired target language. We then score the remaining generated translations against their corresponding original sentence in the source language. For this unsupervised multilingual similarity metric, we utilize the BERTScore [146] algorithm with `mT5300M` (mT5-small),² dubbing it “mT5Score”.

¹<https://pypi.org/project/langdetect/>

²The BERTScore Python library [146] directly supports using mT5 instead of BERT.

We take the top two synthetic parallel examples with the highest mT5Score in the filtering step and use those as synthetic few-shot examples in the prompt in the self-amplification step.

In essence, this procedure biases towards selecting synthetic examples from mT5 that the model is self-consistent on, i.e., synthetic translations from the model that the model itself also scores as "high-quality".

3.1.2 Translating with an Ensemble of Prompts

Because the two examples used in the prompt can greatly affect the quality of the generated translations, some prompts containing low-quality synthetic examples may cause poor translations for certain sentences. To combat this and reduce variation in performance, we keep the top N synthetic examples instead of two synthetic examples. We use these to form $\frac{N}{2}$ different few-shot prompts with two synthetic parallel examples each. Each sentence in the test set is then translated with these $\frac{N}{2}$ different prompts to produce $\frac{N}{2}$ translations. The best translation of the $\frac{N}{2}$ translations is chosen in a fully unsupervised manner with mT5Score, as done in the filtering step of the bootstrap.

We find this ensembling technique helps make unsupervised zero-shot performance competitive with few-shot performance. Unless otherwise stated, we use a 4 prompt ensemble: $\frac{N}{2} = 4$.

In sum, we sample and zero-shot translate 100 sentences from a monolingual corpus, keep the top eight synthetic parallel examples scored by mT5Score, and use them to form four few-shot prompts, each of which has two synthetic examples.

3.1.3 English-Centric Bootstrapping

While Han et al. [19] only performed a bootstrap on English-French and French-English pairs, we perform bootstrapping on some language pairs which may contain at least one low-resource language or non-English language.

It has been found that multilingual language models perform best in English, due to imbalance of languages in the pre-training corpus where English has the highest amount of data [147]. Therefore, when running the bootstrap on various language pairs, we modify the bootstrap to favor generating English, or pivot through English when neither the source nor target language is English.

3.2 RESULTS

Following Lin et al. [147], we evaluate our technique on 14 languages from the FLORES-101 dataset [148] that span high-resource and low-resource languages.³ We evaluate SentencePiece BLEU (spBLEU) [148] in every direction, leading to an evaluation over 182 language pairs in total.

Abbreviated results can be found in Table 3.1, and the matrix of full results can be found in Table 3.2. Results are average spBLEU scores over subsets of the 182 language pairs (src $\rightarrow$ tgt) where “LR” is a low-resource language and “HR” is a high-resource language. “All” represents the average spBLEU score over all 182 language pairs. Supervised results correspond to the M2M-124 615M model from Goyal et al. [148]. XGLM_{7.5B} results correspond

³HR: English (en), German (de), French (fr), Catalan (ca), Finish (fi), Russian (ru), Bulgarian (bg), Chinese (zh), Korean (ko), Arabic (ar), Swahili (sw); LR: Hindi (hi), Malayalam (my), Tamil (ta)

		HR → HR	LR → HR	HR → LR	LR → LR	All
Number of Language Pairs		110	33	33	6	182
Supervised		21.5	10.3	8.6	4.3	16.6
GPT-3 _{6.7B}	(32-shot)	8.1	0.4	0.1	0.1	5.0
XGLM _{7.5B}	(32-shot)	15.3	8.7	6.8	3.8	12.2
mT5 _{3.7B} + SAP	(2-shot)	14.5	9.8	8.2	7.1	12.3
mT5 _{3.7B} + SAP	(zero-shot)	15.5	10.7	9.1	8.2	13.2

Table 3.1: Abbreviated machine translation results on FLORES-101 devtest (spBLEU).

to the model from Lin et al. [147]. Bold denotes best of GPT-3, XGLM, and mT5. spBLEU computed using the implementation from Goyal et al. [148].

			en	de	fr	ca	fi	ru	bg	zh	ko	ar	sw	hi	my	ta	avg
en	Supervised		–	32.6	42.0	31.2	24.2	27.1	37.4	19.3	18.5	17.9	26.9	28.1	3.5	3.4	24.0
	GPT-3 _{6.7B}	(32-shot)	–	25.9	36.1	23.8	10.2	11.2	5.9	12.5	1.2	1.1	0.5	0.3	0.1	0.0	9.9
	XGLM _{7.5B}	(32-shot)	–	27.6	36.0	34.0	23.3	24.2	33.1	15.6	12.0	11.5	18.0	19.9	<u>11.0</u>	<u>8.5</u>	21.1
	mT5 _{3.7B} + SAP	(2-shot)	–	23.2	34.2	26.2	15.8	20.1	27.9	9.5	10.4	11.4	17.3	14.0	<u>11.0</u>	<u>11.2</u>	17.9
	mT5 _{3.7B} + SAP	(zero-shot)	–	26.0	33.2	28.4	15.7	21.2	27.1	11.3	10.5	12.7	19.1	16.1	13.2	13.1	19.0
de	Supervised		35.8	–	35.5	25.8	22.6	24.6	31.5	17.2	16.6	14.8	21.0	23.4	2.3	2.3	21.0
	GPT-3 _{6.7B}	(32-shot)	40.4	–	26.2	17.2	8.1	9.3	4.8	9.0	1.0	0.9	0.5	0.3	0.1	0.1	9.1
	XGLM _{7.5B}	(32-shot)	38.8	–	27.9	19.1	20.5	19.7	25.8	12.3	3.4	6.6	11.7	14.3	9.9	<u>4.8</u>	16.5
	mT5 _{3.7B} + SAP	(2-shot)	33.0	–	24.4	17.8	14.1	15.7	20.2	8.2	9.1	7.7	11.0	10.0	<u>9.8</u>	<u>9.6</u>	14.7
	mT5 _{3.7B} + SAP	(zero-shot)	<u>35.9</u>	–	25.9	22.5	14.3	17.4	21.0	8.2	8.4	8.7	13.4	10.4	<u>9.0</u>	10.8	15.8
fr	Supervised		37.2	28.5	–	28.7	21.9	24.5	32.2	17.6	16.7	15.4	17.2	22.9	2.1	0.8	20.4
	GPT-3 _{6.7B}	(32-shot)	42.8	20.9	–	23.7	8.0	9.7	4.6	9.1	1.0	1.0	0.4	0.3	0.1	0.0	9.4
	XGLM _{7.5B}	(32-shot)	40.4	20.4	–	32.1	19.4	19.8	26.3	10.6	2.4	5.9	14.5	13.7	<u>9.7</u>	<u>6.6</u>	17.1
	mT5 _{3.7B} + SAP	(2-shot)	38.0	19.2	–	26.7	13.7	18.3	23.5	8.6	9.2	9.9	15.0	12.1	10.8	<u>9.7</u>	16.5
	mT5 _{3.7B} + SAP	(zero-shot)	<u>38.1</u>	21.1	–	<u>30.1</u>	12.9	18.1	22.3	8.7	9.2	11.1	15.7	11.0	<u>9.6</u>	11.1	16.8
ca	Supervised		33.4	24.8	35.1	–	19.0	21.1	28.6	15.1	13.9	13.4	18.7	20.5	2.1	2.6	19.1
	GPT-3 _{6.7B}	(32-shot)	40.2	18.6	31.4	–	7.0	9.3	4.3	8.0	0.9	0.9	0.3	0.4	0.1	0.1	9.3
	XGLM _{7.5B}	(32-shot)	41.1	18.9	33.8	–	11.3	3.3	23.9	10.8	1.3	0.8	13.8	6.1	7.9	<u>3.1</u>	13.6
	mT5 _{3.7B} + SAP	(2-shot)	<u>33.4</u>	14.9	29.5	–	10.7	14.0	15.6	6.5	7.0	5.6	12.4	7.3	8.7	<u>6.7</u>	13.3
	mT5 _{3.7B} + SAP	(zero-shot)	<u>37.1</u>	19.3	32.4	–	12.4	16.7	19.1	7.9	7.4	8.5	14.5	9.4	<u>8.3</u>	<u>9.8</u>	15.6
fi	Supervised		27.2	23.0	29.3	21.6	–	20.6	26.4	16.0	14.8	12.4	14.2	19.8	1.7	0.9	17.5
	GPT-3 _{6.7B}	(32-shot)	25.3	13.5	17.1	10.0	–	6.4	2.8	5.7	0.7	0.7	0.3	0.3	0.1	0.0	6.4
	XGLM _{7.5B}	(32-shot)	29.2	17.4	22.2	17.0	–	16.5	17.5	12.4	7.5	7.6	8.0	10.1	<u>6.2</u>	<u>2.0</u>	13.4
	mT5 _{3.7B} + SAP	(2-shot)	24.1	16.1	19.8	14.9	–	14.2	17.0	7.0	5.8	7.1	8.3	5.6	8.5	<u>3.9</u>	11.7
	mT5 _{3.7B} + SAP	(zero-shot)	23.2	16.1	20.5	16.3	–	14.5	16.3	8.0	5.9	6.3	10.0	7.5	<u>5.9</u>	<u>8.2</u>	12.2
ru	Supervised		27.5	23.5	30.1	22.0	19.4	–	31.0	16.5	15.3	13.5	18.1	20.9	2.2	2.3	18.6
	GPT-3 _{6.7B}	(32-shot)	28.1	14.8	20.4	13.1	5.4	–	7.4	1.2	0.2	0.2	0.1	0.2	0.1	0.1	7.0
	XGLM _{7.5B}	(32-shot)	30.4	17.9	24.0	14.6	8.0	–	26.3	11.6	5.5	7.4	7.1	9.1	<u>7.3</u>	<u>3.1</u>	13.2
	mT5 _{3.7B} + SAP	(2-shot)	26.9	16.6	22.4	14.5	11.2	–	25.2	6.1	8.0	6.4	11.3	9.1	9.8	<u>8.4</u>	13.5
	mT5 _{3.7B} + SAP	(zero-shot)	<u>27.9</u>	17.1	22.5	19.4	13.1	–	25.4	8.3	8.7	9.1	12.0	9.0	<u>9.0</u>	10.3	14.8
bg	Supervised		33.0	26.1	33.7	24.9	20.8	26.5	–	17.5	16.4	14.5	20.9	23.1	2.3	2.4	20.2
	GPT-3 _{6.7B}	(32-shot)	21.6	11.4	16.0	9.7	4.3	6.5	–	1.2	0.2	0.2	0.1	0.2	0.1	0.1	5.5
	XGLM _{7.5B}	(32-shot)	35.5	19.2	26.3	12.9	14.2	22.9	–	11.9	6.8	9.2	9.4	7.5	<u>3.2</u>	<u>1.0</u>	13.9
	mT5 _{3.7B} + SAP	(2-shot)	31.0	17.0	23.8	18.3	10.9	22.9	–	7.2	8.3	8.1	11.7	7.4	9.5	<u>6.6</u>	14.1
	mT5 _{3.7B} + SAP	(zero-shot)	32.5	17.3	24.5	21.7	10.6	23.2	–	8.7	7.5	9.0	13.0	8.6	<u>7.9</u>	10.1	15.0
zh	Supervised		20.9	17.6	24.3	17.4	16.0	17.2	22.1	–	15.9	11.6	15.5	18.5	1.9	2.5	15.5
	GPT-3 _{6.7B}	(32-shot)	21.1	9.5	14.3	8.2	4.3	3.6	1.3	–	1.1	0.4	0.2	0.2	0.1	0.0	4.9
	XGLM _{7.5B}	(32-shot)	20.7	8.3	8.5	10.5	4.4	4.8	14.8	–	9.3	4.2	5.6	12.0	<u>8.6</u>	<u>6.2</u>	9.1
	mT5 _{3.7B} + SAP	(2-shot)	19.0	10.9	14.9	11.9	8.0	10.6	11.9	–	8.9	6.0	9.1	8.0	10.0	<u>7.6</u>	10.5
	mT5 _{3.7B} + SAP	(zero-shot)	18.5	10.9	14.8	12.8	8.8	10.7	11.8	–	9.2	6.5	9.0	8.9	<u>8.2</u>	8.9	10.7
ko	Supervised		20.9	16.7	22.1	16.5	14.9	15.5	21.1	15.7	–	10.6	15.1	18.7	1.9	4.0	14.9
	GPT-3 _{6.7B}	(32-shot)	8.3	4.6	6.4	4.4	2.1	1.7	0.8	2.5	–	0.2	0.1	0.1	0.1	0.1	2.4
	XGLM _{7.5B}	(32-shot)	19.9	10.3	13.7	5.3	1.4	1.2	10.9	11.9	–	2.7	3.2	1.0	<u>2.2</u>	1.4	6.5
	mT5 _{3.7B} + SAP	(2-shot)	18.3	10.1	13.7	11.3	7.9	10.1	12.6	7.8	–	6.3	7.2	6.6	<u>2.6</u>	<u>4.7</u>	9.2
	mT5 _{3.7B} + SAP	(zero-shot)	18.1	10.1	13.8	12.8	7.8	9.9	11.4	7.6	–	5.5	8.0	6.7	8.1	8.2	9.8
ar	Supervised		25.5	18.7	25.7	18.9	15.6	17.8	23.8	13.1	13.3	–	15.4	19.4	1.8	0.9	16.1
	GPT-3 _{6.7B}	(32-shot)	10.5	5.3	9.6	6.0	2.2	2.2	0.9	0.9	0.1	–	0.1	0.1	0.2	0.0	2.9
	XGLM _{7.5B}	(32-shot)	27.7	12.2	17.9	8.8	8.5	9.1	18.4	8.9	0.8	–	7.7	7.8	<u>3.4</u>	<u>3.7</u>	10.4
	mT5 _{3.7B} + SAP	(2-shot)	23.7	10.8	17.5	11.0	8.0	12.2	13.8	5.9	7.1	–	10.3	8.0	8.0	<u>8.0</u>	11.1
	mT5 _{3.7B} + SAP	(zero-shot)	<u>26.9</u>	11.5	19.8	15.9	7.8	14.5	13.6	6.3	7.6	–	11.0	8.0	8.8	<u>9.3</u>	12.4
sw	Supervised		30.4	19.4	26.7	20.1	15.6	17.6	23.8	13.2	12.2	12.0	–	19.2	2.1	4.0	16.6
	GPT-3 _{6.7B}	(32-shot)	5.0	2.9	3.9	2.8	1.7	1.8	1.3	1.3	0.5	0.5	–	0.4	0.1	0.1	1.7
	XGLM _{7.5B}	(32-shot)	31.6	13.4	21.8	15.4	10.2	13.1	15.2	9.5	6.0	8.9	–	7.6	<u>3.4</u>	<u>1.0</u>	12.1
	mT5 _{3.7B} + SAP	(2-shot)	27.0	12.6	19.0	15.1	9.2	12.2	15.8	5.9	6.0	8.3	–	6.5	5.4	<u>6.0</u>	11.5
	mT5 _{3.7B} + SAP	(zero-shot)	30.0	13.5	20.0	18.0	9.5	14.5	15.8	6.9	5.7	7.7	–	6.5	<u>2.7</u>	7.0	12.1
hi	Supervised		27.9	19.4	25.9	18.9	15.7	16.9	23.9	13.5	13.9	12.2	16.8	–	2.5	3.8	16.2
	GPT-3 _{6.7B}	(32-shot)	1.2	0.9	1.4	0.8	0.4	0.4	0.3	0.2	0.1	0.1	0.1	–	0.1	0.2	0.5
	XGLM _{7.5B}	(32-shot)	25.2	12.3	15.4	8.8	9.8	11.5	11.3	10.8	8.5	6.1	4.7	–	1.5	1.9	9.8
	mT5 _{3.7B} + SAP	(2-shot)	25.7	12.4	17.0	13.0	8.0	12.2	15.4	7.2	4.4	7.4	8.9	–	<u>9.6</u>	<u>9.0</u>	11.6
	mT5 _{3.7B} + SAP	(zero-shot)	27.1	12.6	17.3	14.3	9.0	12.4	14.5	8.0	6.7	8.1	8.9	–	10.2	12.8	12.5
my	Supervised		10.0	6.9	10.4	8.5	6.0	6.7	9.5	5.7	6.1	4.6	7.2	9.1	–	2.5	7.2
	GPT-3 _{6.7B}	(32-shot)	0.5	0.3	0.4	0.4	0.2	0.1	0.2	0.0	0.0	0.0	0.1	0.2	–	0.1	0.2
	XGLM _{7.5B}	(32-shot)	14.1	7.6	10.1	3.8	5.7	7.1	8.9	7.1	6.9	3.6	3.5	8.9	–	<u>2.6</u>	6.9
	mT5 _{3.7B} + SAP	(2-shot)	16.8	8.5	12.9	11.0	6.7	6.1	9.2	5.2	2.9	5.0	8.0	7.0	–	<u>5.7</u>	<u>8.1</u>
	mT5 _{3.7B} + SAP	(zero-shot)	<u>16.4</u>	9.0	11.9	11.6	6.9	<									

3.3 DISCUSSION

In this unsupervised setting, we find our zero-shot results exceed our 2-shot results; furthermore, they significantly exceed the performance of the XGLM and GPT-3 results reported in Lin et al. [147] on an average spBLEU score over all 182 pairs (+1.0 spBLEU). We note strong performance on language pairs that contain one or more low-resource languages.

THE EFFECT OF SELF-CONSISTENCY FILTERING We ablate the use of mT5Score as a filter to select the high-quality synthetic examples during bootstrapping in Table 3.3. We perform the ablation over two language pairs, English-Russian and Russian-English. When using random selection, the synthetic parallel examples chosen may be extremely low-quality or non-sensical leading to a 0.0 spBLEU score after self-amplification as shown for the English-Russian language pair.

	English-Russian	Russian-English
Random Selection	0.0	25.5
mT5Score Filtering and Selection	20.0	26.3

Table 3.3: Results from ablating the use of mT5Score as a filter to select the high-quality, self-consistent synthetic examples during bootstrapping.

THE EFFECT OF PROMPT-ENSEMBLING Intuitively, we can explain the zero-shot performance surpassing the supervised, few-shot performance through our use of prompt ensembling in the zero-shot setting. As prompt ensembling utilizes four prompts with two synthetic parallel examples each, it essentially uses eight synthetic examples, instead of just two real examples in the few-shot setting. Our synthetic examples are nearly as high-quality as real examples (similar to the findings of Han et al. [19]) as demonstrated in an experiment we show in Table 3.4. In this experiment, we compare utilizing a single few-shot prompt with two

synthetic parallel examples to perform the final translation with utilizing an ensemble of 4, 8, and 16 distinct few-shot prompts each with two synthetic parallel examples that generate 4, 8, and 16 translations respectively from which the best translation (by mT5Score) is selected as the final translation over two language pairs, English-Russian and Russian-English. Prompt ensembling not only reduces performance variation if low-quality synthetic examples are selected during the bootstrap, but it also boosts performance beyond the supervised, few-shot setting as demonstrated by Table 3.4 (Russian-English 26.9 $\rightarrow$ 27.9 spBLEU).

	English-Russian	Russian-English
<i>Supervised Machine Translation</i>		
Few-Shot Prompt (2-shot)	20.1	26.9
<i>Unsupervised Machine Translation</i>		
Single Prompt	20.0	26.3
4 Prompt Ensemble	20.9	27.9
8 Prompt Ensemble	20.7	28.6
16 Prompt Ensemble	20.9	28.6

Table 3.4: Results with prompt ensembles of size 4, 8, and 16.

THE EFFECT OF ENGLISH-CENTRIC BOOTSTRAPPING We also ablate our use of English-centric bootstrapping in Table 3.5. We perform the ablation over two language pairs, English-Russian and Russian-Chinese.

	English-Russian	Russian-Chinese
Standard bootstrap	20.9	5.8
English-centric bootstrap	21.2	8.3

Table 3.5: Results from ablating the use of English-centric bootstrapping.

3.4 CONCLUSION

In summary, the synthetic data procedure I outlined improves the zero-shot machine translation performance of an LLM using synthetic examples generated by the model itself. The procedure applies a number of inductive biases on the synthetic data generation process and example selection & filtering process to use our prior knowledge about the task to allow the model to better learn and improve.

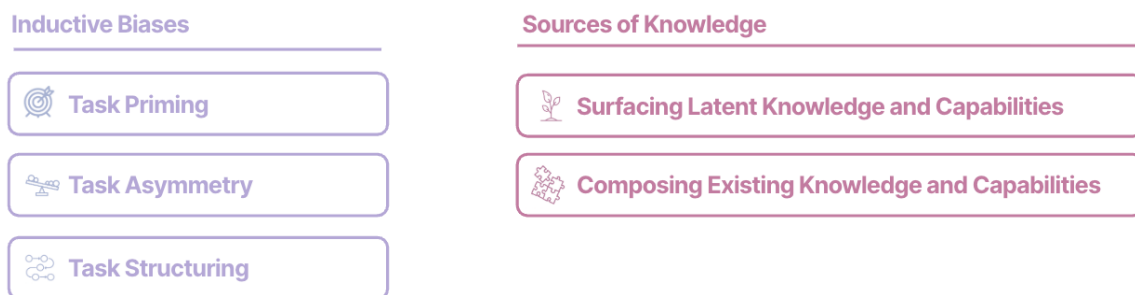


Figure 3.2: Inductive biases and sources of knowledge highlighted by the work on improving the zero-shot machine translation performance of a model with synthetic data.

The primary inductive bias applied here is priming a general model to more narrowly perform the specific task we want of machine translation with in-context structured examples. And the source of knowledge here is surfacing latent knowledge the model may already have acquired during pre-training from uncured, scattered examples of the languages on the internet. This is knowledge may not be as readily accessible without such priming.

When applying self-consistency filtering, the inductive bias applied is using prior knowledge of the asymmetry in the task. For example, we may suspect it is easier for the model to verify whether translations are good or bad rather than generate translations. And the source of knowledge is composing together these two existing capabilities of the model to create a stronger result.

When performing English-centric bootstrapping, along with taking advantage of “task asymmetry” by using the hypothesis that working with English is likely easier than working with other languages for the model due to over-representation of English in the training data, we also impose a “structured process” (task structuring) using this prior knowledge of how to best perform the task. The source of knowledge here is composing existing knowledge and capabilities together.

This work used a fairly small synthetic dataset comprised of just a handful of synthetic examples. In the next chapter I will discuss DataDreamer, a framework developed for synthetic data generation and training workflows to allow researchers to generate and train on larger synthetic datasets.

3.5 SUMMARY OF CONTRIBUTIONS

This work is based on our paper “Bidirectional Language Models Are Also Few-shot Learners” published at the International Conference on Learning Representations 2023 [145]. I led the overall research direction, conducted all experiments, and was the primary author responsible for writing the paper. My co-authors helped guide and advise the research direction and contributed minor edits and rewrites that refined the presentation of our findings.

4 | DATADREAMER: A FRAMEWORK FOR SYNTHETIC DATA GENERATION AND TRAINING WORKFLOWS

While simple to deploy, in-context learning has fundamental limitations due to the restricted context length of language models and the ephemeral nature of learning in that setting, as model weights are not updated. In contrast, the paradigm of using pre-trained models to build generalized knowledge and fine-tuning them on task-specific data has consistently yielded the strongest performance on NLP tasks [2, 149]. In the last chapter, we demonstrated how synthetic examples generated by a model can enhance its performance by providing few-shot examples for in-context learning. However, fully leveraging the potential of synthetic data requires experimenting with larger datasets and developing tools that allow practitioners to construct such datasets effectively. Encoding task-specific knowledge often necessitates complex, multi-stage workflows for synthetic dataset construction [150], where an LLM is iteratively used to generate, refine, filter, and select synthetic training examples. These workflows can be challenging to implement, engineer, maintain, and reproduce, making large-scale utilization of synthetic data generation difficult for practitioners.

This chapter introduces **DataDreamer**, a framework designed to streamline synthetic data generation and training, making both the creation of synthetic data and the steering of model learning more accessible and approachable for researchers and practitioners. DataDreamer supports workflows for fine-tuning and aligning models across a range of tasks. By enabling large-scale synthetic data generation and simplifying its incorporation into training workflows,

DataDreamer extends the utility of synthetic data, showcasing its role in systematically shaping model performance.

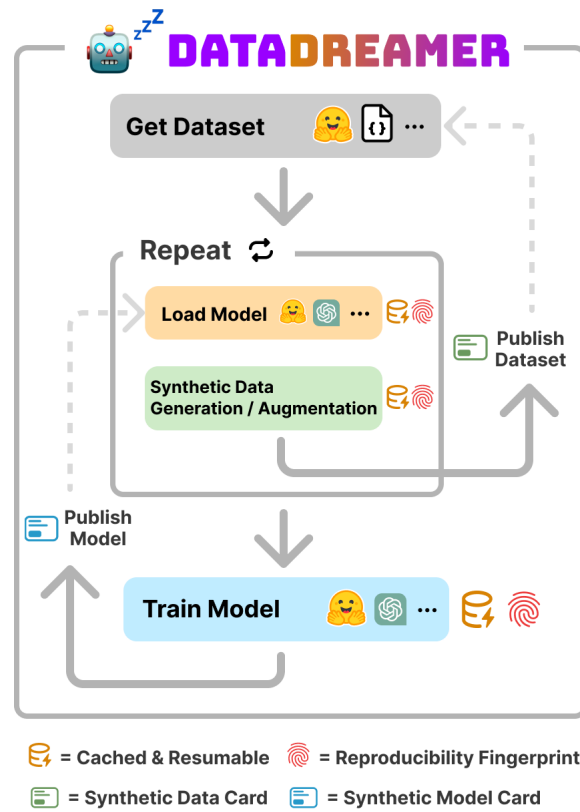


Figure 4.1: DataDreamer helps researchers implement many types of LLM workflows easier and makes reproducibility automatic and simple. These workflows often involve synthetic data generation with a LLM-in-the-loop and/or fine-tuning, aligning, and distilling models.

4.1 MOTIVATION

While large language models (LLMs) have established a new era in NLP research through the prompt-and-predict paradigm that has proven effective for a wide variety of tasks and synthetic data generation, the use of these models has come with significant drawbacks [151]. Many

popular models like GPT-4 [60] are closed source and behind a remote API, while running models locally can be technically complex and expensive due to their scale. Moreover, the now well-established prompting paradigm can be brittle with results widely varying between different models, configurations, and environments [152, 153]. These challenges have made it difficult for researchers to share, reproduce, extend, and compare work, hindering the rate of research progress.

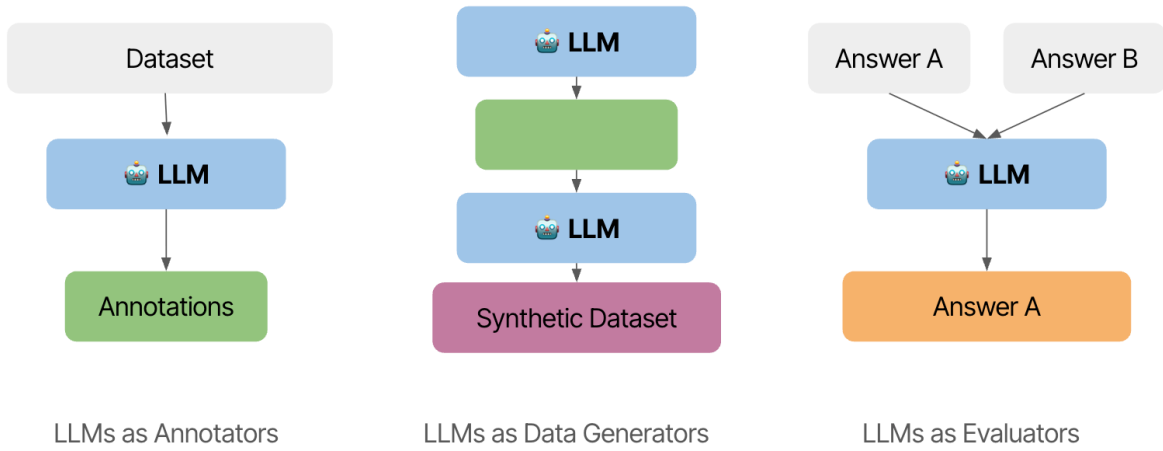


Figure 4.2: Various ways LLMs are used in synthetic data workflows.

4.2 CURRENT CHALLENGES WITH LLM WORKFLOWS

To further motivate DataDreamer, we first discuss the LLM workflows that it supports and we discuss challenges that arise from these usage patterns. DataDreamer offers an easier solution to efficiently implement these workflows and make them reproducible, important for transparency in science and maintainability in industry. These LLM workflows are often used in combination with each other [84], and orchestration of multi-stage workflows is frequently implemented through multiple shell or Python scripts. Reproducing these multi-stage workflows

is challenging as shell scripts may rely upon a particular author’s job scheduler or environment and require execution in a specific order. In Section 4.3 and 4.4, we discuss how DataDreamer’s task orchestration, caching system, and simple multi-GPU training make it easier to implement these multi-stage workflows in a single Python program, minimizing these issues.

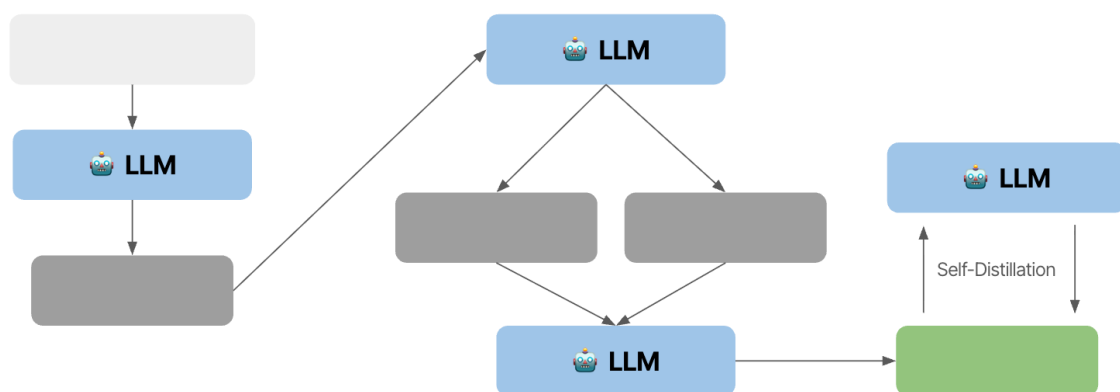


Figure 4.3: Synthetic data workflows are becoming increasingly complex, where LLMs are often chained to process data over multiple stages or are used to judge and filter generated outputs. The resulting data from these stages are also used sometimes to perform fine-tuning, distillation, or self-distillation.

SYNTHETIC DATA GENERATION Recent work has explored using LLMs to create synthetic data for tasks or to augment existing datasets to boost task performance [9, 10, 11, 12, 13, 14, 15, *inter alia*]. Synthetic data generation involves using a LLM once or multiple times in a multi-stage workflow to process data, sometimes referred to as “chaining” [150]. When prompting LLMs to generate or augment datasets, a reproducibility challenge that arises is “prompt sensitivity” where even small variations in a prompt can lead to significantly different results [152]. Moreover, it is imperative to tag synthetically generated datasets because of model degradation concerns [154].

LLMS FOR TASK EVALUATION Another increasingly common workflow is using LLMs as judges or as automatic metrics for evaluating a model’s performance on a task [155, 156, 157, 158, *inter alia*]. Many of the reproducibility challenges applicable to synthetic data also arise here.

FINE-TUNING AND ALIGNMENT Another common workflow is the creation of task-specific expert models using knowledge from larger models to create smaller, more efficient models via fine-tuning and distillation [159, 160, 161]. Instruction-tuning is fine-tuning that allows base pre-trained models to better follow natural language human instruction and improve their generalized task performance [6, 8, 7, 162]. Closely related, alignment techniques steer model responses towards those more preferable to humans [163, 164, 70]. Implementing resumability and efficient training techniques are practical challenges often faced. Reproducibility challenges include sharing exact data and hyperparameters.

SELF-IMPROVING LLMS Self-improving LLMs through self-feedback training loops is an increasingly active area of research interest [81, 40, 82, 83, 84, 22]. These workflows can be uniquely complex to both implement and reproduce due to requiring multiple rounds that chain together synthetic data generation, automatic evaluation, and model re-training. DataDreamer supports all of these workflows and makes it simple to chain data between them.

4.3 DATADREAMER

Before delving into the structure and implementation of DataDreamer, we first provide a simple demonstration of DataDreamer’s capabilities and API through an example synthetic data gener-

ation and distillation workflow in Example 1. The LLM used in this example is GPT-4 [60]. As an initial step, the example uses the LLM to generate 1,000 NLP research paper abstracts. The LLM is then used to summarize those abstracts in a tweet-like style. These two steps result in a fully synthetic dataset of abstracts and tweets summarizing them. Using a trainer, this synthetic dataset is then distilled to a small, local model that is capable of summarizing paper abstracts in a tweet-like style. As a final step, the example demonstrates how both the synthetic dataset and the trained model can be published and shared. For illustrative purposes, we demonstrate a sample generation of the trained model’s output on the DataDreamer paper’s abstract:

“Introducing DataDreamer, an open source Python library for advanced #NLP workflows. It offers easy code to create powerful LLM workflows, addressing challenges in scale, closed source nature, and tooling. A step towards open science and reproducibility! #AI #MachineLearning”

```

1 from datadreamer import DataDreamer
2 from datadreamer.llms import OpenAI
3 from datadreamer.steps import DataFromPrompt, ProcessWithPrompt
4 from datadreamer.trainers import TrainHFFineTune
5 from peft import LoraConfig
6
7 with DataDreamer("./output"):
8     # Load GPT-4
9     gpt_4 = OpenAI(model_name="gpt-4")
10
11     # Generate synthetic arXiv-style research paper abstracts with GPT-4
12     arxiv_dataset = DataFromPrompt(
13         "Generate Research Paper Abstracts",
14         args={
15             "llm": gpt_4,
16             "n": 1000,
17             "temperature": 1.2,
18             "instruction": (
19                 "Generate an arXiv abstract of an NLP research paper."
20                 " Return just the abstract, no titles."
21             ),
22         },
23         outputs={"generations": "abstracts"},
24     )
25
26     # Use GPT-4 to convert the abstracts to tweets
27     abstracts_and_tweets = ProcessWithPrompt(
28         "Generate Tweets from Abstracts",
29         inputs={"inputs": arxiv_dataset.output["abstracts"]},
30         args={
31             "llm": gpt_4,
32             "instruction": "Given the abstract, write a tweet to summarize the work.",
33             "top_p": 1.0,
34         },
35         outputs={"inputs": "abstracts", "generations": "tweets"},
36     )
37
38     # Create training data splits
39     splits = abstracts_and_tweets.splits(train_size=0.90, validation_size=0.10)
40
41     # Train a model to convert research paper abstracts to tweets with the
42     # synthetic dataset
43     trainer = TrainHFFineTune(
44         "Train an Abstract => Tweet Model",
45         model_name="google/t5-v1.1-base",
46         peft_config=LoraConfig(),
47     )
48     trainer.train(
49         train_input=splits["train"].output["abstracts"],
50         train_output=splits["train"].output["tweets"],
51         validation_input=splits["validation"].output["abstracts"],
52         validation_output=splits["validation"].output["tweets"],
53         epochs=30,
54         batch_size=8,
55     )
56
57     # Publish and share the synthetic dataset
58     abstracts_and_tweets.publish_to_hf_hub("repo_id")
59
60     # Publish and share the trained model
61     trainer.publish_to_hf_hub("repo_id")

```

Example 4.1: In this demonstration snippet, DataDreamer generates a fully synthetic dataset of tweets summarizing research paper abstracts and then trains a smaller T5 distilled model.

4.3.1 Supported Features and Comparison with Alternatives

Below, we detail the features supported by DataDreamer and compare with alternative solutions.

Feature	LangChain ¹	Axlotl ²	HF Transformers + TRL ³	DataDreamer
<i>Implementation</i>				
Accessible via Python API	✓	✗	✓	✓
Built for Researchers	✗	✗	✓	✓
<i>Integrations</i>				
Open Source Models	✓	✓	✓	✓
Commercial & API-based Models	✓	✗	✗	✓
<i>Tasks</i>				
Prompting & Prompt “Chaining”	✓	✗	✗	✓
Synthetic Data Generation & Augmentation	✓	✗	✗	✓
Fine-tuning LLMs	✗	✓	✓	✓
Instruction-tuning LLMs	✗	✓	✓	✓
Aligning LLMs	✗	✓	✓	✓
Training Classifier Models	✗	✗	✓	✓
Training Embedding Models	✗	✗	✗	✓
<i>Conveniences</i>				
Caching	○	✗	✗	✓
Resumability	✗	✓	○	✓
Simplifies Boilerplate Code (tokenization, etc.)	✓	✓	✗	✓
Simplifies Multi-GPU Inference and Training	✗	○	✗	✓
Publishing Datasets & Models	✗	○	✓	✓
<i>Open Science and Reproducibility</i>				
Reproducibility Fingerprints	✗	✗	✗	✓
Saves Intermediate Outputs	✗	✗	✗	✓
Synthetic Data and Model Cards	✗	✗	✗	✓

Table 4.1: We compare feature coverage between other popular libraries and solutions available to researchers today that target similar workflows. DataDreamer integrates these features into a single library with a standardized interface making experimentation and chaining data between tasks simple. (✗= No; ✓= Yes; ○ = Partial Support)

¹<https://github.com/langchain-ai/langchain>

²<https://github.com/OpenAccess-AI-Collective/axolotl>

³Wolf et al. [165] and Werra et al. [166]

4.3.2 Built-Ins Functionality in DataDreamer

We also detail common operations, models, and training techniques built-in to DataDreamer.

Type	Examples
Steps	Load a Dataset DataSource, HFHubDataSource, JSONDataSource, CSVDataSource, ...
	Prompting Prompt, RAGPrompt, ProcessWithPrompt, FewShotPrompt, DataFromPrompt, DataFromAttributedPrompt, FilterWithPrompt, RankWithPrompt, JudgeGenerationPairsWithPrompt, ...
	Other Embed, Retrieve, CosineSimilarity, ...
Models	OpenAI, OpenAIAssistant, HFTransformers, CTransformers, VLLM, Petals, HFAPIEndpoint, Together, MistralAI, Anthropic, Cohere, AI21, Bedrock, Vertex, ...
Trainers	TrainOpenAIFineTune, TrainHFClassifier, TrainHFFineTune, TrainSentenceTransformer, TrainHFDPO, TrainHFPP0, ...

Table 4.2: A few examples of built-in steps, models, and trainers available in DataDreamer.

4.3.3 Installation

DataDreamer can be installed with:

```
pip install datadreamer.dev
```

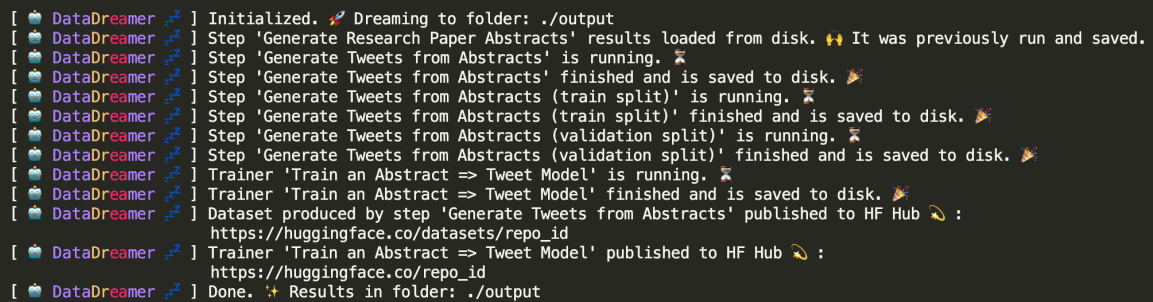
4.3.4 Sessions

All code using the DataDreamer library is placed within a “session” using a Python context manager instantiated using the with keyword:

```
from datadreamer import DataDreamer

with DataDreamer("./output"):
    ...
```

Workflow tasks can be run within the session context manager. These tasks are called “steps” (loading a dataset, prompting a model, etc.) or “trainers”. The session allows DataDreamer to automatically organize the resulting datasets, outputs, caches, training checkpoints, and trained models that result from tasks run within the session into the `./output/` folder. Each step in a workflow assigns a custom descriptive name for its subfolder under `./output/`. DataDreamer sessions automatically provide user-friendly logging around workflow tasks run within the session:



```
[ DataDreamer ] Initialized. Dreaming to folder: ./output
[ DataDreamer ] Step 'Generate Research Paper Abstracts' results loaded from disk. It was previously run and saved.
[ DataDreamer ] Step 'Generate Tweets from Abstracts' is running.
[ DataDreamer ] Step 'Generate Tweets from Abstracts' finished and is saved to disk.
[ DataDreamer ] Step 'Generate Tweets from Abstracts (train split)' is running.
[ DataDreamer ] Step 'Generate Tweets from Abstracts (train split)' finished and is saved to disk.
[ DataDreamer ] Step 'Generate Tweets from Abstracts (validation split)' is running.
[ DataDreamer ] Step 'Generate Tweets from Abstracts (validation split)' finished and is saved to disk.
[ DataDreamer ] Trainer 'Train an Abstract => Tweet Model' is running.
[ DataDreamer ] Trainer 'Train an Abstract => Tweet Model' finished and is saved to disk.
[ DataDreamer ] Dataset produced by step 'Generate Tweets from Abstracts' published to HF Hub :
https://huggingface.co/datasets/repo_id
[ DataDreamer ] Trainer 'Train an Abstract => Tweet Model' published to HF Hub :
https://huggingface.co/repo_id
[ DataDreamer ] Done. Results in folder: ./output
```

Figure 4.4: DataDreamer logs produced by the workflow in Example 1 when resuming from a prior interrupted run.

4.3.5 Steps

Steps are the core operators in a DataDreamer session. A step in DataDreamer transforms from an input dataset to an output dataset [167]. This is useful for tasks like generating synthetic data

from LLMs, or data augmentation for existing datasets. The output of one step can be directly used as the input to another step or as the input to a trainer, allowing users to chain together multiple steps/trainers to create complex workflows. DataDreamer comes with a number of built-in steps for common operations in LLM workflows, some examples of which can be seen in Table 4.2. Useful standard data processing operations such as `.map()`, `.filter()`, and `.shuffle()` can also quickly be applied to the output of a step for custom processing. DataDreamer uses memory-mapping to handle large datasets stored on disk and can be run lazily over iterable, streaming datasets.

4.3.6 Models

Models can be loaded in a DataDreamer session and then be passed as an argument to steps like `FewShotPrompt` and `ProcessWithPrompt`. DataDreamer creates a standardized interface for accessing open source and commercial LLMs. It includes interfaces for embedding models as well as LLMs. Examples of supported models and model providers can be found in Table 4.2.

4.3.7 Trainers

Trainers can train on a dataset produced by a step in a DataDreamer workflow. The dataset may be loaded from an external source or produced as the output of a step in a multi-step workflow. DataDreamer's trainers support a wide variety of techniques and tasks including fine-tuning, instruction-tuning, alignment via RLHF [6] and DPO [70], distillation, training classifiers, and training embedding models. Examples of supported techniques are shown in Table 4.2.

4.3.8 Caching and Sharing Workflows

Caching has practical utility in LLM workflows as these large models can be both computationally and financially expensive to run. Therefore, eliminating re-computation can save both time and resources. Caching in DataDreamer happens at multiple levels. When a step or trainer is completed, its resulting dataset or trained model is saved to disk and loaded from disk if the step or trainer is executed again with the same inputs and arguments, instead of being run again. Additionally, DataDreamer caches at the model-level, caching the results of prompts or texts being run against a model to a SQLite database file. During training, DataDreamer similarly automatically saves checkpoints and resumes from them if interrupted and restarted. Caching uses minimal disk space (storing mainly text) and adds minimal overhead in these workloads dominated by heavy model inference computation, but can be granularly disabled if desired.

DataDreamer's cache system allows a researcher to share both their workflow script and their session output folder with others, giving them access to useful caches and saved outputs. These allow others to easily reproduce and extend the entire workflow while also benefiting from avoiding expensive computations when unnecessary. For example, a researcher could extend another researcher's workflow by adding another step at the end. Only the additional added step would need to be computed, while all of the original steps could have their results loaded from disk.

4.3.9 Resumability

Caching allows resumability during development, so scripts can be interrupted and resumed. This allows graceful handling of crashes, server preemption, and other situations where only a portion of a workflow was previously computed. Furthermore, caching can be useful during

experimentation of a workflow. For example, when modifying a single prompt in the middle of a multi-step synthetic data generation workflow, the change may only affect a certain number of inputs to the next step. If so, only that portion of the work will be re-computed.

4.3.10 Sharing Open Data and Open Models

DataDreamer provides convenient utilities for exporting and publishing datasets and trained models produced by steps or trainers. Resources can be exported to disk or published to the Hugging Face Hub.⁴ When resources are published, DataDreamer can automatically upload a demonstration snippet and set up the live demonstration widget on the Hugging Face Hub, which makes shared resources easily usable. Additionally, these resources are automatically given appropriate metadata such as tags clearly indicating when data is synthetically generated and its source LLM. DataDreamer also produces what we call “synthetic data cards” and “synthetic model cards”. Synthetic data and model cards are automatically produced by recursively tracing through all steps, models, and trainers that DataDreamer used to produce the dataset or model. Each step, model, and trainer has associated metadata including license information and citation information. DataDreamer collects this information and produces a synthetic data card (or model card) that reports the information along with reproducibility information for each step, model, and trainer in the workflow. The information collected in our cards is defined in Table 4.3.

⁴<https://huggingface.co/>

Date & Time
The date and time the step or trainer was run. This is important to document when using API-based LLMs that can be updated over time.
Dataset Name & Card
The name of any datasets used as part of a step or trainer's operation along with their data cards.
Model Name & Card
The name of any models used in a step or trainer's operation along with their model cards.
URL
A URL that can be referenced for more information about the step or trainer.
License
Any known license that may apply as a result of a model or dataset being used in a step or trainer.
Citations
Citations for datasets and models used in a trainer.
Reproducibility Fingerprint
A hash of all inputs, arguments, and configurations that may affect reproducibility for a step or trainer. When steps and trainers are chained in a multi-stage workflow, the reproducibility hash is computed recursively through the chain. These fingerprints can be used to compare if two workflows within DataDreamer are exactly identical.
Other Reproducibility Information
Other miscellaneous reproducibility information such as environment information, system information, and versions of packages and dependencies.

Table 4.3: Information automatically recorded in a synthetic data card or synthetic model card.

These automatically generated synthetic data cards and model cards can aid in preventing contamination of pre-training sources with model-generated synthetic data. As synthetic data generation becomes more prevalent, contamination can be a concern due to the performance degradation that has been observed when synthetic datasets are shared and trained on, possibly without the knowledge of the model developer [154]. DataDreamer's cards can also help other

researchers understand what license restrictions may apply to the synthetically generated data, among other usability concerns. These automatically generated cards are not a replacement for traditional data cards and model cards [168, 169] that recommend a wider set of important attributes such as potential dataset biases. Instead, they provide supplemental information that is crucial to the usability and reproducibility of LLM workflows. We encourage researchers to review and add information that cannot be automatically detected to our generated cards.

4.3.11 Efficiency and Optimizations

LLMs workflows often benefit from or require certain optimizations to be applied in order to load or process the scale of data and models typically used. DataDreamer supports many of the common optimizations that researchers may want to apply.

PARALLELIZATION DataDreamer supports running steps in background processes and running steps concurrently to easily implement parallel task orchestration in a workflow.

QUANTIZATION AND ADAPTERS DataDreamer supports quantization of model weights that can reduce memory usage [170] as well as parameter-efficient fine-tuning techniques like LoRA adapters [68, 171]. It standardizes using these optimizations across different model architectures and minimizes boilerplate, making it as simple as a single argument to configure training with LoRA in Example 1. DataDreamer attempts to create uniform support for features across all of its supported integrations when possible. So while the underlying `sentence-transformers` and `transformers` libraries do not support training embedding models with LoRA [172, 165], DataDreamer supports this, which extends the benefits of LoRA to these models.

MULTI-GPU USAGE DataDreamer makes it simple to load models on multiple GPUs and train models on multiple GPUs with PyTorch FSDP [173, 174]. For example, training a model on multiple GPUs is as simple as passing a list of `torch.devices` to the `device` parameter of a trainer (`device=["cuda:0", "cuda:1"]`). DataDreamer automatically configures FSDP and launches distributed processes within the session so that a command line launcher like `torchrun` never has to be used, simplifying multi-GPU training. The use of `torchrun` can often force complex, multi-stage workflows being split into multiple scripts launched via shell scripts since training portions need to be isolated from data generation or data processing portions. This added complexity in running the workflow end-to-end can make reproducibility challenging. With DataDreamer, workflows do not need to be re-orchestrated around portions needing to be launched via `torchrun`. Since DataDreamer handles this distributed orchestration automatically, users can build multi-stage workflows involving data generation, data processing, and training on multiple GPUs all in a single Python program, obviating the use of orchestration through multiple shell scripts.

4.3.12 Configuration and Extensibility

DataDreamer seeks to minimize configuration and boilerplate code that for most research workflows do not need to be customized, for example automatically handling tokenization and applying the correct padding, among other tasks. DataDreamer applies sensible defaults and standard research practices to minimize configuration. Some researchers, however, may need to customize these choices and the option to override and extend is provided and well-documented.

4.4 REPRODUCIBLE LLM RESEARCH AND WORKFLOWS

We outline a few best practices, specific to the emerging use of LLMs in research workflows that DataDreamer adopts. We believe instituting these practices can alleviate a number of reproducibility concerns. Of course, when closed-source models are involved, these concerns can never be fully eliminated. We discuss how DataDreamer makes it easier to implement these practices or automatically implements these practices in this section.

ADAPTABLE TO MODEL SUBSTITUTION While experimental workflows can often be sensitive to model choice and the transferability of prompts can be unreliable [151], for reproducibility purposes and for ease of experimentation, workflow implementation code should attempt to minimize dependence on a specific model and should allow other researchers to easily substitute one LLM for another. This can also be useful if a model is not accessible to another researcher or if a model has become obsolete. DataDreamer’s API and model abstractions make model substitution simple.

SHARING PROMPTS Exact prompts used should be shared since even minor variations can significantly impact performance [152]. DataDreamer makes it easy to share an entire workflow and session output folder. DataDreamer can also help ensure a re-implementation is exactly identical between two experimental setups by comparing the reproducibility fingerprints of individual steps or the entire workflow in aggregate.

SHARING INTERMEDIATE OUTPUTS In multi-stage workflows, intermediate outputs should be shared for inspection and analysis by other researchers as well as for extendability purposes. DataDreamer makes this simple by automatically saving the results of each step

in a multi-stage workflow in an easily inspectable Hugging Face datasets format [167]. When API-based LLMs are used, there is greater risk to reproducibility. DataDreamer allows workflows to be exactly reproduced from caches in the session output folder, even if the remote API is no longer available.

SYNTHETIC DATA CARDS AND MODEL CARDS Synthetic data and model cards can help other researchers understand the source of synthetic data, license restrictions that may apply, citations that may apply, among other attributes. Importantly, these cards and other metadata-like tags can help prevent contamination of pre-training data [154]. Finally, these cards carry reproducibility information, useful for validating two experimental setups as identical.

SHARING OPTIMIZATION CONFIGURATIONS Optimizations like quantization can have an effect on generations [153]. DataDreamer's reproducibility fingerprints account for these configurations and with its easily shareable workflows, DataDreamer makes it easy to reproduce an exact workflow, along with configured optimizations.

ENVIRONMENT-AGNOSTIC CODE For reproducibility, code should attempt to minimize dependence on local environments, job schedulers, shell scripts, etc. DataDreamer helps make this easier by providing tools for workflow orchestration (steps, parallelization, managed distributed processes for multi-GPU training) that can be all be done within Python. DataDreamer also minimizes dependencies on local file paths, by organizing results and outputs into the session output folder automatically.

4.5 EXAMPLES

In this section, I provide a few key examples of the library performing common synthetic data workflows and procedures as well as some complex workflows that are made simple to implement.

4.5.1 Augmenting an Existing Dataset

In this demonstration snippet, we augment an existing dataset, HotpotQA [175], a multi-hop QA dataset. DataDreameer makes it easy to perform synthetic dataset augmentation with a LLM.

In this example, we add intermediate questions required to solve the multi-hop question.

```

1 from datadreamer import DataDreameer
2 from datadreamer.llms import OpenAI
3 from datadreamer.steps import ProcessWithPrompt, HFHubDataSource
4
5 with DataDreameer("./output"):
6     # Load GPT-4
7     gpt_4 = OpenAI(model_name="gpt-4")
8
9     # Get HotPot QA questions
10    hotpot_qa_dataset = HFHubDataSource(
11        "Get Hotpot QA Questions",
12        "hotpot_qa",
13        config_name="distractor",
14        split="train",
15    ).select_columns(["question"])
16
17    # Keep only 1000 questions as a quick demo
18    hotpot_qa_dataset = hotpot_qa_dataset.take(1000)
19
20    # Ask GPT-4 to decompose the question
21    questions_and_decompositions = ProcessWithPrompt(
22        "Generate Decompositions",
23        inputs={"inputs": hotpot_qa_dataset.output["question"]},
24        args={
25            "llm": gpt_4,
26            "instruction": (
27                "Given the question which requires multiple steps to solve,"
28                " give a numbered list of intermediate questions required to"
29                " solve the question. Return only the list, nothing else."
30            ),

```

```

31     },
32     outputs={"inputs": "questions", "generations": "decompositions"},
33 ).select_columns(["questions", "decompositions"])

```

4.5.2 Instruction-Tuning a LLM

In this demonstration snippet, we instruction-tune a model [6, 176, 131]. DataDreameer reduces boilerplate around tokenization, caching, training resumability, multi-GPU training, parameter-efficient fine-tuning, and more.

```

1 from datadreamer import DataDreameer
2 from datadreamer.steps import HFHubDataSource
3 from datadreamer.trainers import TrainHFFineTune
4 from peft import LoraConfig
5
6 with DataDreameer("./output"):
7     # Get the Alpaca instruction-tuning dataset (cleaned version)
8     instruction_dataset = HFHubDataSource(
9         "Get Instruction-Tuning Dataset", "yahma/alpaca-cleaned", split="train"
10    )
11
12    # Keep only 1000 examples as a quick demo
13    instruction_dataset = instruction_dataset.take(1000)
14
15    # Some examples taken in an "input", we'll format those into the instruction
16    instruction_dataset.map(
17        lambda row: {
18            "instruction": (
19                row["instruction"]
20                if len(row["input"]) == 0
21                else f"Input: {row['input']}\n\n{row['instruction']}"
22            ),
23            "output": row["output"],
24        },
25        lazy=False,
26    )
27
28    # Create training data splits
29    splits = instruction_dataset.splits(train_size=0.90, validation_size=0.10)
30
31    # Define what the prompt template should be when instruction-tuning
32    chat_prompt_template = "### Instruction:\n{{prompt}}\n\n### Response:\n"
33
34    # Instruction-tune the base TinyLlama model to make it follow instructions
35    trainer = TrainHFFineTune(
36        "Instruction-Tune TinyLlama",
37        model_name="TinyLlama/TinyLlama-1.1B-intermediate-step-1431k-3T",

```

```

38     chat_prompt_template=chat_prompt_template,
39     peft_config=LoraConfig(),
40     device=["cuda:0", "cuda:1"],
41     dtype="bfloat16",
42 )
43 trainer.train(
44     train_input=splits["train"].output["instruction"],
45     train_output=splits["train"].output["output"],
46     validation_input=splits["validation"].output["instruction"],
47     validation_output=splits["validation"].output["output"],
48     epochs=3,
49     batch_size=1,
50     gradient_accumulation_steps=32,
51 )

```

4.5.3 Self-Rewarding LLMs

This demonstration snippet implements a simplified version of the self-rewarding LLMs [84] procedure. This workflow involves using an LLM to judge its own generations in order to self-align and self-improve itself over a number of rounds. DataDreamer allows this complex multi-stage workflow to be implemented intuitively, without needing to split generation and training logic into separate files and without needing to involve a launcher like `torchrun` to perform multi-GPU training. DataDreamer also makes this complex multi-round, multi-stage workflow automatically cachable and resumable.

```

1 from datadreamer import DataDreamer
2 from datadreamer.steps import (
3     HFHubDataSource,
4     Prompt,
5     JudgeGenerationPairsWithPrompt,
6 )
7 from datadreamer.trainers import TrainHFDPO
8 from datadreamer.llms import HFTransformers
9 from peft import LoraConfig
10
11 with DataDreamer("./output"):
12     # Get a dataset of prompts
13     prompts_dataset = HFHubDataSource(
14         "Get Prompts Dataset", "Intel/orca_dpo_pairs", split="train"
15     ).select_columns(["question"])
16
17     # Keep only 3000 examples as a quick demo

```

```

18 prompts_dataset = prompts_dataset.take(3000)
19
20 # Define how many rounds of self-reward training
21 rounds = 3
22
23 # For each round of self-reward training
24 adapter_to_apply = None
25 for r in range(rounds):
26     # Use a partial set of the prompts for each round
27     prompts_for_round = prompts_dataset.shard(
28         num_shards=rounds, index=r, name=f"Round #{r+1}: Get Prompts"
29     )
30
31     # Load the LLM
32     llm = HFTransformers(
33         "TinyLlama/TinyLlama-1.1B-Chat-v1.0",
34         adapter_name=adapter_to_apply,
35         device_map="auto",
36         dtype="bfloat16",
37     )
38
39     # Sample 2 candidate responses from the LLM
40     candidate_responses = []
41     for candidate_idx in range(2):
42         candidate_responses.append(
43             Prompt(
44                 f"Round #{r+1}: Sample Candidate Response #{candidate_idx}",
45                 inputs={"prompts": prompts_for_round.output["question"]},
46                 args={
47                     "llm": llm,
48                     "batch_size": 2,
49                     "top_p": 1.0,
50                     "seed": candidate_idx,
51                 },
52             )
53         )
54
55     # Have the LLM judge its own responses
56     judgements = JudgeGenerationPairsWithPrompt(
57         f"Round #{r+1}: Judge Candidate Responses",
58         args={
59             "llm": llm,
60             "batch_size": 1,
61             "max_new_tokens": 5,
62         },
63         inputs={
64             "prompts": prompts_for_round.output["question"],
65             "a": candidate_responses[0].output["generations"],
66             "b": candidate_responses[1].output["generations"],
67         },
68     )
69
70     # Unload the LLM
71     llm.unload_model()
72
73     # Process the judgements into a preference dataset

```

```

74 dpo_dataset = judgements.map(
75     lambda row: {
76         "question": row["prompts"],
77         "chosen": (
78             row["a"]
79             if row["judgements"] == "Response A"
80             else row["b"]
81         ),
82         "rejected": (
83             row["b"]
84             if row["judgements"] == "Response A"
85             else row["a"]
86         ),
87     },
88     lazy=False,
89     name=f"Round #{r+1}: Create Self-Reward Preference Dataset",
90 )
91
92 # Create training data splits
93 splits = dpo_dataset.splits(train_size=0.90, validation_size=0.10)
94
95 # Align the TinyLlama chat model with its own preferences
96 trainer = TrainHFDPO(
97     f"Round #{r+1}: Self-Reward Align TinyLlama-Chat",
98     model_name="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
99     peft_config=LoraConfig(),
100     device=["cuda:0", "cuda:1"],
101     dtype="bfloat16",
102 )
103 trainer.train(
104     train_prompts=splits["train"].output["question"],
105     train_chosen=splits["train"].output["chosen"],
106     train_rejected=splits["train"].output["rejected"],
107     validation_prompts=splits["validation"].output["question"],
108     validation_chosen=splits["validation"].output["chosen"],
109     validation_rejected=splits["validation"].output["rejected"],
110     epochs=3,
111     batch_size=1,
112     gradient_accumulation_steps=32,
113 )
114
115 # Unload the trained model from memory
116 trainer.unload_model()
117
118 # Use the newly trained adapter for the next round of self-reward
119 adapter_to_apply = trainer.model_path

```

4.6 IMPACT AND CASE STUDIES

DataDremer has demonstrated significant impact in the domain of synthetic data generation and large language model (LLM) workflows, achieving over 1,100 GitHub stars. The framework supports all major LLM providers, models, frameworks, generation techniques, and training methodologies, enabling adoption by researchers, industry practitioners, and startups. By lowering the barrier to entry for state-of-the-art workflows, it has also facilitated their use in educational settings. Recently, DataDremer was employed for large-scale data generation to develop PixMo-Docs, a VQA dataset integral to the instruction-tuning process of Molmo, a frontier open-source multimodal model [177] (a synthetic dataset generated from a pipeline developed in DataDremer by my co-author Yue Yang and me). In Chapters 5 and 6, I provide additional examples of how DataDremer has been used in research to generate and train on synthetic datasets to improve model performance and enable new capabilities.

4.7 CONCLUSION

In summary, DataDremer helps practitioners implement complex, multi-step synthetic data generation and training workflows easily and makes it simple to practically incorporate inductive biases into the synthetic data generation process.

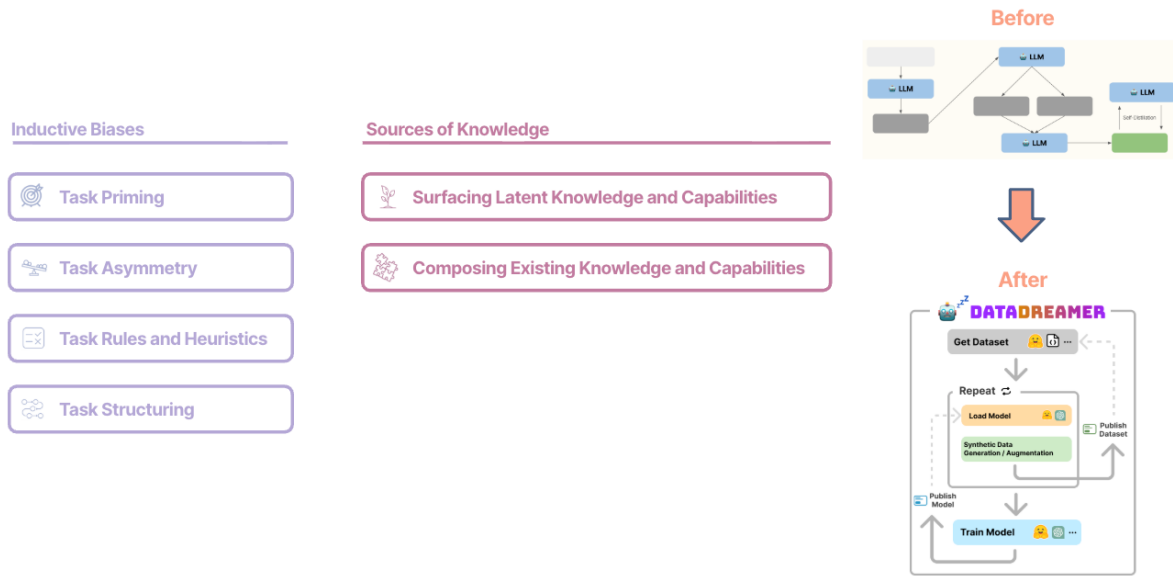


Figure 4.5: Inductive biases and sources of knowledge can be easily applied through complex synthetic data workflows that DataDreamer enables.

The next chapter will discuss three distinct examples of using DataDreamer to generate larger synthetic datasets to use with fine-tuning to enable new capabilities in models.

4.8 SUMMARY OF CONTRIBUTIONS

This work is based on our paper “DataDreamer: A Tool for Synthetic Data Generation and Reproducible LLM Workflows” published at the Association for Computational Linguistics 2024 [16]. I led the overall development of the library and was the primary author responsible for writing the paper. My co-authors helped guide and advise the development and contributed minor edits and rewrites that refined the presentation of our findings.

5 | ENABLING NEW MODEL CAPABILITIES BY FINE-TUNING ON SYNTHETIC DATA

In Chapter 3, I detailed how synthetic examples generated by a model can be used to enhance its own performance in the in-context learning setting. However, that setting is limited and can only utilize a handful of examples that can fit within a prompt. In the last chapter, I introduced a simple framework called DataDreamer to make it easier to generate large synthetic datasets constructed out of complex, multi-stage workflows. The framework also makes it easier to fine-tune LLMs on these larger synthetic datasets, enabling models to learn from larger amounts of synthetic data and potentially unlocking new capabilities. In this chapter, I will demonstrate deploying DataDreamer to generate carefully crafted synthetic datasets that can enable new capabilities in models after fine-tuning. I discuss three examples in my research where I have successfully utilized synthetic data to enable new capabilities in models.

5.1 EXAMPLE 1: TRAINING STYLE EMBEDDING MODELS

The most common objective when training text embeddings is to place texts with similar semantics close together in the embedding space [178]. Style representations, by contrast, aim to embed texts with similar writing styles near each other and texts with different styles far apart, regardless of their semantic content [92]. Embeddings are usually trained via contrastive

learning, with triplets consisting of an anchor text, a positive text (which should be embedded closely to the anchor), and a negative text (which should be embedded far from the anchor) [179, 180, 181]. Therefore, parallel datasets where style varies while content is held constant are needed to train style embeddings, but such datasets are rare.

Existing approaches often use social media datasets with the assumption that all writing by the same author shares a similar style and that texts by different authors exhibit dissimilar styles [92]. However, these methods are limited by the imperfect nature of data acquired under such assumptions. For example, the same author may write about the same topic even in different subreddits. As a result, these imperfect contrastive triplets do not explicitly control for content, leading to style embeddings with weak content-independence. To overcome this, a more controlled approach to style contrastive learning is necessary.

We introduce **STYLEDISTANCE**, a novel method for training stronger, content-independent style embeddings which leverages synthetic parallel text examples generated by a large language model (LLM) [60]. By creating near-exact paraphrases with controlled stylistic variations, we produce positive and negative examples across 40 distinct style features. This synthetic dataset, which we call **SYNTHSTEL**, enables more precise contrastive learning (visualized in Figure 5.1) and is more robust to the content leakage inherent in existing datasets. We evaluate our final trained models, measuring the content-independence, quality, and utility of **STYLEDISTANCE** embeddings.

5.1.1 Experimental Setup

We use a LLM to generate a synthetic dataset with controlled content and style. The dataset is composed of pairs of sentences which are paraphrases of each other (i.e. their content is similar) but differ in style. Each pair includes a positive example that showcases a specific stylistic

Style Feature: Usage of Active Voice

Anchor:

● *"I **adored** the Dolce & Gabbana mauve leather creation with a signature lock."*

Positive & Negative Examples:

● *"I **observed** the impact rising temperatures had on tropical diseases."*

● *"The Dolce & Gabbana mauve leather creation with a signature lock was **adored by me**."*

POSITIVE

NEGATIVE

Same Style
Different Content

Same Content
Different Style

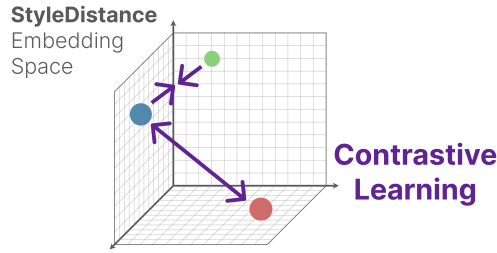


Figure 5.1: STYLEDISTANCE embeddings are trained using contrastive learning from synthetic parallel (positive and negative) examples representing 40 style features. The illustrated example is for the “Usage of Active Voice” feature.

feature (e.g., usage of active voice or emojis) and a corresponding negative example that lacks this feature. In this section, we outline the selected style features and provide implementation details for our synthetic data generation procedure.

Style Feature Selection

There is no predefined set of style features, and what features are considered to describe style vs. content can vary across different studies [182]. For this work, we select 40 style features

across 7 broad categories (visualized in Figure 5.2) which have been addressed in different works on text style [183, 184, 185, 182, 94].

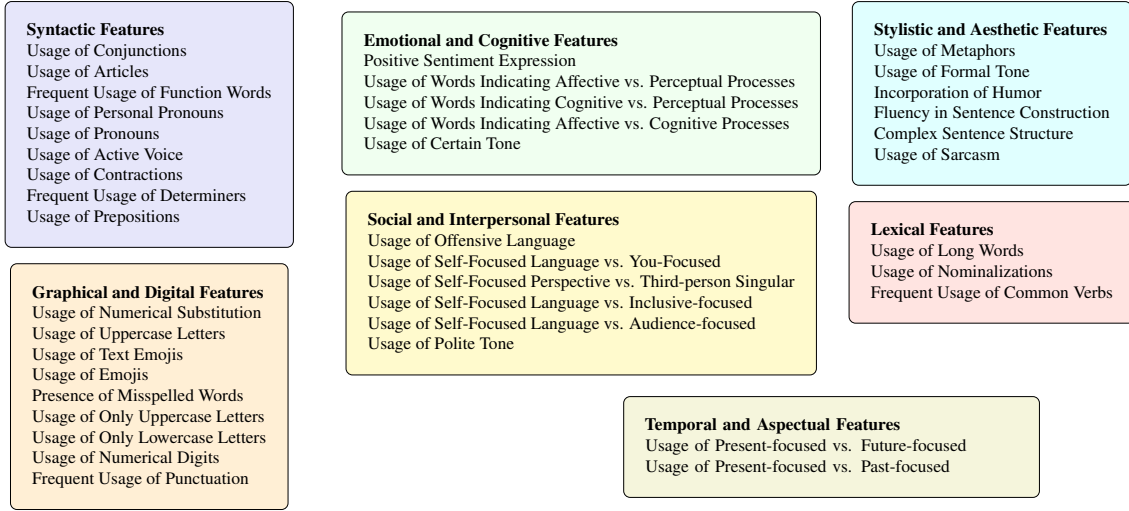


Figure 5.2: We generate synthetic parallel examples to train STYLEDISTANCE for a wide range of style features in seven linguistic and stylistic categories.

Generation

For each of the selected features, we generate 100 pairs of positive and negative examples by prompting GPT-4 [60] with the DataDreamer library [16]. Each generated pair contains a sentence where the feature is present (positive example), and a paraphrase where the feature is absent or less present (negative example). We use the attributed prompt (AttrPrompt) method [9] to create diverse and realistic synthetic examples. To generate many examples per style feature, we use a random sentence from C4 [186], and extract the fine-grained topic of the sentence using GPT-4 [60] to use in the attributed prompt to use as the topic for the generated examples. Along with a new unused topic each time, we randomly sample a new permutation of the attributes in the prompt, and we perform sampling with temperature = 1.0 and top_p = 1.0. We demonstrate an example below for the “Usage of Active Voice” style feature.

Generate a pair of active and passive sentences with the following attributes:

1. Topic: {topic}
2. Length: {sentence_length}
3. Point of view: {point_of_view}
4. Tense: {tense}
5. Type of Sentence: {sentence_type}

Ensure that the generated sentences meet the following conditions:

1. There is no extra information in one sentence that is not in the other.
 2. The difference between the two sentences is subtle.
 3. The two sentences have the same length.
- {special_conditions_for_style_feature}

Use Format:

Active: [sentence]

Passive: [sentence]

Your response should only consist of the two sentences, without quotation marks.

We call our final synthetically generated dataset SYNTHSTEL, and create train and test splits using a 90%/10% split.

We next describe how we trained STYLEDISTANCE embeddings using our synthetic dataset.

Sampling Contrastive Triplets

After generating 100 pairs of positive and negative examples for each style feature, we construct feature-specific triplets as follows: We select an “anchor” (a) and a “positive” example (p) from different pairs available for a feature, ensuring that the two examples are **identical in style** but not in content. For the “Usage of Active Voice” example in Figure 5.1, a and p are two active

sentences (*I adored ...*, *I observed...*) on different topics. As a negative example (n), we use the paraphrase of either a or p which does not contain the feature; therefore, n is always different in style. In the example in Figure 5.1, n is the paraphrase of the anchor in passive voice (*...adored by me*).

In terms of **content**, n is a paraphrase of the anchor a in half of the triplets, and has different content in the other half. This ensures that the trained model will not only learn to discriminate parallel (paraphrased) texts, but will be able to generalize to texts with different content during inference. This sampling process results in ~320K unique triplets. The implementation of this simple algorithm is shared in the supplementary materials.

Contrastive Learning Objective

In our final dataset of triplets $\mathcal{D}$, each triplet $(a, p, n) \in \mathcal{D}$ contains an anchor text (a), a positive text (p), and a negative text (n). We train our embedding model $f_\theta(\cdot)$ with a triplet loss (with margin α) [181]:

$$L_t(\theta) = \sum_{(a,p,n) \in \mathcal{D}} \left[\|f_\theta(a) - f_\theta(p)\|_2^2 - \|f_\theta(a) - f_\theta(n)\|_2^2 + \alpha \right]_+$$

We use roberta-base as our base model for fine-tuning—the same base model used for the style embeddings in Wegmann, Schraagen, Nguyen, et al. [92]—and perform training with DataDreamer and LoRA [16, 68]. We use a margin of 0.1, a learning rate of 1e-4, and a batch size of 512. We further split our training set into a train and validation split (90%/10%) and train using an early stopping patience of 1 epoch.

Training

We train two versions of our model. `STYLEDISTANCESYNTH` is fine-tuned only on the synthetic triplets described in Section 5.1.1. We also train a version using the synthetic triplets for data augmentation (`STYLEDISTANCE`). In this case, our training set is comprised of 50% natural data—i.e. the triplets used to train the Wegmann, Schraagen, Nguyen, et al. [92] model¹—and 50% synthetic data. For the augmented model, we hypothesize that mixing in these perfectly parallel synthetic examples will help regularize the model, and discourage it from representing content-related features in favor of style-related ones offering the potential advantages of both approaches: (1) enhanced content-independence, and (2) the ability to capture niche style features in the natural data.

5.1.2 Results

We perform an evaluation of `STYLEDISTANCE` embeddings with the STEL and STEL-or-Content tasks [185, 92] that allow for a direct evaluation of the quality of style representations. We use other leading style representations like LISA [94] and the embeddings from Wegmann, Schraagen, Nguyen, et al. [92] as baselines, and compare them with our style embeddings. We also compare to the LUAR model [93], an authorship representation model which does not explicitly train content-independent representations but captures both content and style features in an attempt to represent different authors. We briefly describe these tasks below and illustrate examples of these tasks in Figure 5.3:

- **STEL:** Given two anchor sentences (A1, A2) and two test sentences (S1, S2), STEL measures the ability of an embedding model to pair each test sentence with the anchor

¹We use the train-conversation split.

sentence that shares the same style based on the cosine similarity of the embeddings of the sentences.

- STEL-or-Content:** The STEL-or-Content task is similar to STEL but more adversarially challenging, hence better for testing content-independence. In this task, there are again two test sentences (S1, S2) but only a single anchor sentence (A). The test sentence that best matches the *style* of the anchor must be selected; but the incorrect test sentence—which is written in a different style—is a paraphrase of the anchor with similar *content*. Therefore, in order to succeed on the STEL-or-Content task, a model needs to represent style features stronger than content features.

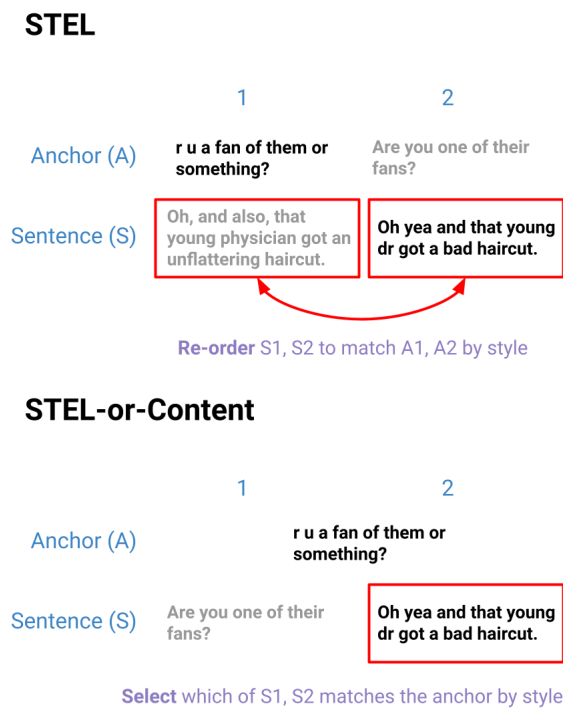


Figure 5.3: A visualization of the STEL and STEL-or-Content tasks.

In their paper, Wegmann and Nguyen [185] provide a STEL and STEL-or-Content evaluation benchmark over five features with curated natural data. We test our models on this benchmark and present the results in Table 5.1.

Model	Formal		Complex		Numb3r		C'tion		Emoji		Avg	
	STEL	S-o-C	STEL	S-o-C	STEL	S-o-C	STEL	S-o-C	STEL	S-o-C	STEL	S-o-C
<i>Content-Aware Representations</i>												
roberta-base	0.83	0.09	0.73	0.01	0.94	0.13	1.00	0.00	0.98	0.03	0.90	0.05
LUAR	0.80	0.14	0.67	0.00	0.74	0.03	0.77	0.00	0.99	0.02	0.86	0.03
<i>Content-Independent Style Representations</i>												
LISA	0.73	0.05	0.65	0.00	0.85	0.03	0.92	0.00	0.77	0.02	0.71	0.03
Wegmann, Schraagen, Nguyen, et al. [92]	0.83	0.70	0.58	0.27	0.56	0.03	0.96	0.02	0.95	0.07	0.77	0.22
STYLEDISTANCE	0.89	0.72	0.64	0.25	0.84	0.12	1.00	0.20	0.99	0.15	0.87	0.29
STYLEDISTANCE _{SYNTH}	0.85	0.73	0.60	0.27	0.71	0.28	0.99	0.22	0.63	0.07	0.76	0.31

Table 5.1: Accuracy on the STEL/STEL-or-Content (S-o-C) tasks. STYLEDISTANCE leads on both tasks among representations trained for content-independence, and STYLEDISTANCE_{SYNTH} generalizes remarkably well to real text data despite being trained only on synthetic data.

5.1.3 Discussion

Our results are consistent with results reported by Wegmann and Nguyen [185], who showed that even untrained models like roberta-base can capture style information well, resulting in stronger STEL performance than any of their fine-tuned models. However, the more challenging STEL-or-Content task, which better tests content-independence, shows that only models specifically trained for content-independence are able to capture style features better than content features. Our results indicate that the embeddings generated with our STYLEDISTANCE approach lead over other style representations on both the STEL and STEL-or-Content tasks. Interestingly, we find that STYLEDISTANCE_{SYNTH} captures style remarkably well, and manages to generalize to the natural text examples in the evaluation benchmark despite only being trained on our synthetic contrastive triplets. We conclude using synthetic parallel examples during training makes the model more content-independent and helps better capture style.

Generalization To Unseen Features

In an ablation study, we test the ability of our training approach to produce embeddings that can generalize to unseen style features which are not present in the synthetic dataset. We conduct this evaluation by ablating features from the data used to train $\text{STYLEDISTANCE}_{\text{SYNTH}}$ under three conditions and show the results in Table 5.2.

Features Tested	Features Used	Formal		Complex		Numb3r		C'tion		Emoji		Avg		Retained Perf.
		STEL	S-o-C	STEL	S-o-C	STEL	S-o-C	STEL	S-o-C	STEL	S-o-C	STEL	S-o-C	
In-Domain	40 out of 40	0.85	0.73	0.60	0.27	0.71	0.28	0.99	0.22	0.63	0.07	0.76	0.31	100%
Out-of-Domain	34 out of 40	0.85	0.67	0.62	0.28	0.56	0.15	0.80	0.00	0.60	0.01	0.68	0.22	65%
Out-of-Distribution	25 out of 40	0.64	0.49	0.65	0.26	0.57	0.11	0.63	0.02	0.68	0.02	0.63	0.18	50%

Table 5.2: We evaluate how well $\text{STYLEDISTANCE}_{\text{SYNTH}}$ embeddings generalize to unseen style features by ablating features from the synthetic training dataset under three conditions: In-Domain, Out-of-Domain, Out-of-Distribution. We evaluate their performance on the STEL and STEL-or-Content (S-o-C) tasks.

In the **In-Domain** condition, all 40 style features are included. In the **Out-of-Domain** condition, we exclude synthetic examples corresponding to the five style features in the STEL/STEL-or-Content benchmark.² In the **Out-of-Distribution** condition, we further exclude examples for any features similar or indirectly related to the five evaluated features. We list the style features ablated (removed from the training data) for the Out-of-Domain and Out-of-Distribution conditions.

Out-of-Domain:

1. Usage of Formal Tone
2. Usage of Contractions
3. Usage of Numerical Substitution

²In our 40 features, there are two separate features for emoji and text emoticons (:-D) so we exclude 6 total features for this condition instead of 5 features, resulting in 34 features.

4. Complex Sentence Structure

5. Usage of Text Emojis

6. Usage of Emojis

Out-of-Distribution:

1. Usage of Formal Tone

2. Usage of Polite Tone

3. Fluency in Sentence Construction

4. Usage of Only Uppercase Letters

5. Usage of Only Lowercase Letters

6. Incorporation of Humor

7. Usage of Sarcasm

8. Usage of Contractions

9. Usage of Numerical Substitution

10. Usage of Numerical Digits

11. Complex Sentence Structure

12. Usage of Long Words

13. Usage of Text Emojis

14. Usage of Emojis

15. Presence of Misspelled Words

We compare the Out-of-Domain and Out-of-Distribution performance of `STYLEDISTANCESYNTH` on the two tasks to its In-Domain performance, obtained when it was trained on data for all 40 style features. Even in the challenging Out-of-Distribution condition, `STYLEDISTANCESYNTH` retains 50% of its performance on the challenging STEL-or-Content task (see the “Retained Perf.” column). This study indicates our training approach generalizes reasonably well to out-of-domain style features which can be composed from style features selected for generation and, to some extent, even to out-of-distribution style features fully outside the selected set.

Applications to Text Style Transfer

Text style transfer is a task where synthetic data is often necessary as supervised parallel datasets of the same text written in multiple styles are scarce. Krishna, Wieting, and Iyyer [45] introduced a technique where a mono-style text corpus can be paraphrased by a model into neutral-style paraphrases to create a synthetic parallel dataset that can be subsequently used for training a text style transfer model. In a research collaboration, we improved this approach to remove the paraphrasing step of the procedure by training a style transfer model to be directly conditioned on a text style embedding of the target style to steer generation in a system we called TinyStyler [187]. While the original TinyStyler system rewrites text by conditioning on Wegmann, Schraagen, Nguyen, et al. [92] embeddings, we demonstrate that `STYLEDISTANCE` provides an alternative, and reproduce the TinyStyler procedure with the exact dataset and hyperparameters in the original paper. We make only one modification: replacing Wegmann embeddings with `STYLEDISTANCE` in the generation and filtering steps. We

include the formality transfer evaluation results in Table 5.3. In these automatic evaluations, the STYLEDISTANCE conditioned model performs comparably. We additionally include examples comparing model output in Table 5.4.

Method	Acc ($\rightarrow F, \rightarrow I$)	Sim ($\rightarrow F, \rightarrow I$)	Fluency ($\rightarrow F, \rightarrow I$)	Joint ($\rightarrow F, \rightarrow I$)	GPT-2
TSTYLER _{WEGMANN}	0.94 (0.90, 0.98)	0.82 (0.81, 0.82)	0.77 (0.83, 0.72)	0.78 (0.77, 0.80)	112.5
TSTYLER _{STYLEDISTANCE}	0.96 (0.94, 0.98)	0.80 (0.80, 0.81)	0.77 (0.82, 0.73)	0.80 (0.79, 0.80)	112.4

Table 5.3: We reproduce the automatic formality transfer evaluation procedure from TinyStyler [187] on the GYAFC dataset [188]. $\rightarrow F$ corresponds to formal transfer, and $\rightarrow I$ corresponds to informal transfer.

Source Text (Informal)	WEGMANN ($\rightarrow$ Formal)	STYLEDISTANCE ($\rightarrow$ Formal)
<i>"its keeping me up at nite, i have to know what it is"</i>	<i>"Have to know what this is, keeping me up at night."</i>	<i>"What is it? It is keeping me up at night."</i>
<i>"i like journey's open arms... but i like mariah carey's version of that song better."</i>	<i>"I prefer mariah carey's version of Journey, Open Arms."</i>	<i>"I think mariah carey's version of that song is better."</i>
<i>"And you can't rely on rumors."</i>	<i>"You can never trust rumors about it."</i>	<i>"I would say that you can't rely on rumors."</i>
Source Text (Formal)	WEGMANN ($\rightarrow$ Informal)	STYLEDISTANCE ($\rightarrow$ Informal)
<i>"I favor the man as he is humorous and grounded."</i>	<i>"i love tahm as he is humorous and grounded..."</i>	<i>"i'm gonna go with the man...hes humorous and grounded."</i>

<i>"I am sure you will both enjoy it."</i>	<i>"ok i am sure you both will enjoy it!"</i>	<i>"oh yea...you'll both enjoy it"</i>
<i>"I very much enjoy this song."</i>	<i>"i love this song so much, i'm a fan of it."</i>	<i>"i like this song so much man..."</i>

Table 5.4: Outputs from TinyStyler, conditioned on Wegmann embeddings and STYLEDISTANCE embeddings.

5.1.4 Summary of Contributions

This work is based on our paper “StyleDistance: Stronger Content-Independent Style Embeddings with Synthetic Parallel Examples” published at the Nations of the Americas Chapter of the Association for Computational Linguistics 2025 [189]. I led the overall research direction with Marianna Apidianaki, conducted all of the training experiments, and contributed to the evaluation experiments. I was the primary author responsible for writing the paper with my co-authors contributing two subsections and helping revise and edit the paper.

5.2 EXAMPLE 2: MAKING AGENT MODELS MORE CAPABLE

LLMs have demonstrated impressive capabilities in a variety of NLP tasks such as summarization and question answering [4, 3, 5] through zero-shot and few-shot prompting techniques [6, 8]. However, prompting techniques alone are insufficient to enable LLMs to act as agents and navigate environments in order to solve complex, multi-step, long-horizon tasks [41]. Fine-

tuning LLMs to perform such tasks is also infeasible due to the scarcity of training data suitable for these tasks. Acquiring data for sequential decision-making and complex interactions is not only time-consuming, but also costly.

A number of proposed self-improvement techniques have demonstrated that LLMs can use zero-shot and few-shot prompting to achieve performance above the baseline without any additional supervised training data [34, 83]. In place of supervised data as a learning signal, many of these techniques use a self-critique technique [190, 84], or obtain a critique through interactions with tools or environments [191]. While self-improvement techniques have shown promise on standard NLP benchmark tasks like machine translation or question answering [192, 34, 83], their efficacy has not yet been thoroughly investigated for long-horizon tasks that require multi-step interactions with a complex and realistic environment.

WebArena [95] is a recently proposed benchmark wherein an LLM agent is required to solve tasks using a web browser. One example WebArena task is to use the OpenStreetMap website to answer the question “*What is the minimum travel time by car from CMU to University of Pittsburgh?*”. Such a task requires an agent to complete a sequence of steps on the website, including entering a start location, entering a destination location, submitting a form, and then, reasoning over the result. The sequence of steps selected by an agent is called a *trajectory*. Unlike existing benchmarks, WebArena tasks are realistic and diverse, require dynamic interaction, and require navigating a complex environment. The baselines presented by [95] demonstrate that while LLMs are capable of interacting with this environment, even the strongest baseline, GPT-4 [60], is only able to solve ~14% of the tasks. This demonstrates that WebArena is a challenging benchmark even for the strongest frontier models [193].

We introduce new techniques that allow LLM agents to better perform complex, multi-step tasks via self-improvement. We detail different strategies for self-improvement that all involve fine-tuning the LLM agent on its own generations (synthetic data) and inducing a signal for

learning by employing unsupervised techniques like self-critique to selectively filter training examples.

5.2.1 Experimental Setup

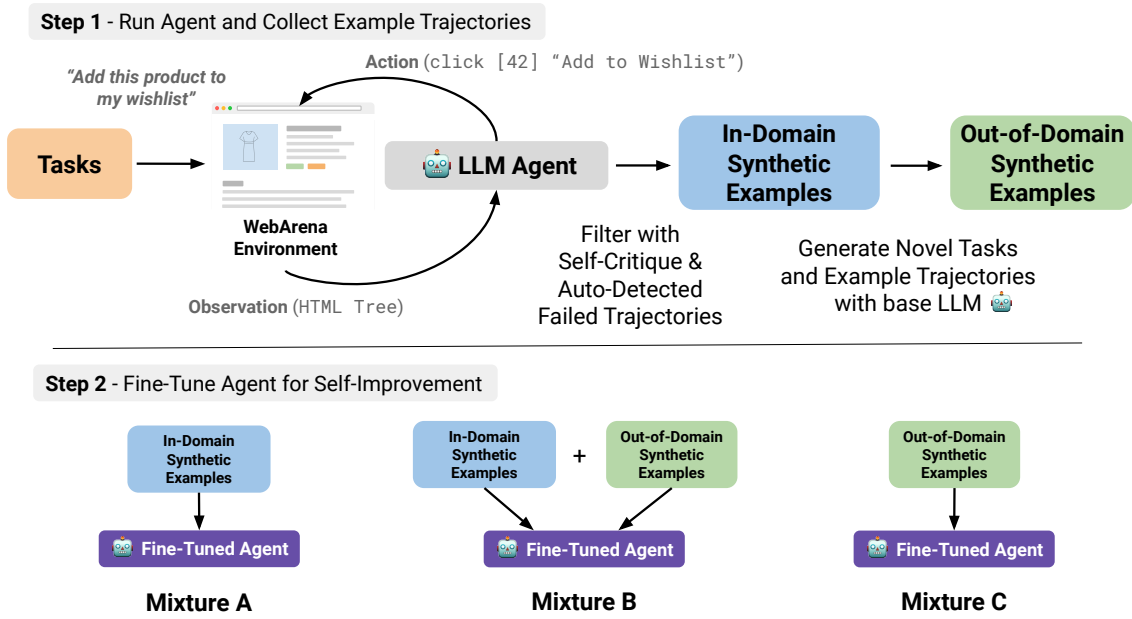


Figure 5.4: We generate synthetic data to fine-tune LLM agents to accomplish WebArena tasks such as “Add this product to my wishlist”. **Step 1:** We first collect an initial set of trajectories, filter out low-quality trajectories in an unsupervised fashion, and keep the remainder as synthetic in-domain examples. We prompt our base LLM to generate novel out-of-domain tasks along with hypothetical solution trajectories by providing a few in-domain examples. **Step 2:** We then fine-tune our base LLM agent on each of the three distinct synthetic training data mixtures and evaluate performance.

Self-improvement techniques for large language models typically involve using the model’s own generations to create synthetic few-shot examples [192] or synthetic fine-tuning data [34]. These techniques amplify knowledge, correct behaviors, and introduce regularization [30], often leading to an overall boost in performance. The self-generated examples are often filtered, post-edited, or ranked with a set of unsupervised techniques such as self-critique to introduce

a signal for learning and improvement [190, 194, 83, 84]. For multi-step agent tasks, the environment itself can additionally provide the LLM agent a way to detect failure in a fully unsupervised manner, which provides another useful signal for learning [191, 195, 98].

Using the WebArena benchmark [95], we define and experiment with both in-domain synthetic training examples and out-of-domain synthetic training examples for web agent tasks, and fine-tune on three different synthetic data mixtures: **Mixture A** (in-domain synthetic examples only), **Mixture B** (both in-domain and out-of-domain synthetic examples), and **Mixture C** (out-of-domain synthetic examples only). Figure 5.4 illustrates our process.

We summarize both kinds of synthetic data briefly before diving into the details of the generation procedure for each:

- **In-Domain Synthetic Data:** For all tasks in WebArena, we collect an initial set of trajectories using the base model. We filter out any trajectories where the model self-detected failure (self-critique) or failure was detectable in the environment and keep the remainder. We denote the remaining set of trajectories as *plausible trajectories*, where the model may or may not have completed the task successfully. Since lower-quality trajectories where the model outright failed to complete the task have been filtered out through self-detection, we hypothesize this remaining higher-quality set of plausible trajectories can serve as reasonably high-quality *in-domain synthetic examples* for fine-tuning. Similar to the self-improvement prior work we discuss earlier, the collection of this data is completely unsupervised and no ground-truth labels are utilized for filtering and selection.
- **Out-of-Domain Synthetic Data:** We also evaluate whether the base model can generate completely novel tasks, objectives, web pages, and solution trajectories that can serve as useful training examples. We use the plausible trajectories as few-shot examples in a prompt for the base model to generate completely new tasks along with potential solu-

tion trajectories. To ensure the model generates examples with sufficient diversity and to improve generalization, we prompt the model to generate *out-of-domain synthetic examples* that are dissimilar from existing tasks and objectives as well as generate tasks for different websites than the set of 6 websites covered by the WebArena benchmark.

In-Domain Synthetic Data Collection

The WebArena environment can be formulated as a partially observable Markov decision process: $\mathcal{E} = \langle \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T} \rangle$, where $\mathcal{S}$ represents the state space, $\mathcal{A}$ represents the action space, $\mathcal{O}$ represents the observation space, and $\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ is the deterministic transition function [95]. An agent model $\mathcal{M}$ produces a next action $a_t \in \mathcal{A}$ provided an objective represented by some natural language intent $\mathbf{i}$, the current observation $o_t \in \mathcal{O}$, and the previous action taken $a_{t-1} \in \mathcal{A}$: $(\mathbf{i}, o_t, a_{t-1})$. This continues for T time steps until the agent produces a stop action or the environment produces an error or stop condition. The model $\mathcal{M}$ we select for our experiments is the Qwen-1.5-72B-Chat model [196], which at the time of this work is a highly ranked³ and competitive open source LLM [193] that is accessible for fine-tuning.

Given this definition, we propose a procedure for sampling a set of in-domain synthetic training examples $\mathcal{D}_{\text{IN-DOMAIN}}$ where each training example is structured as $(\mathbf{i}, o_t, a_{t-1}) \rightarrow a_t$. These examples are sampled from a filtered set of trajectories collected by an initial run of the base agent model $\mathcal{M}$ over all tasks in WebArena in Algorithm 1.

We filter out low-quality trajectories where the model produced a generation stating the task to be “impossible” or that it “cannot” make progress (a form of self-critique). Additionally, we filter out any trajectories where the model produced `stop[N/A]`, `stop[No ...]`, or `stop[]`, indicating when the model may have refused to provide an answer. Finally, we also filter out any trajectories where the WebArena environment encountered an error or the model failed

³<https://chat.lmsys.org/?leaderboard>

Algorithm 1 Collect In-Domain Synthetic Training Examples $\mathcal{D}_{\text{IN-DOMAIN}}$ **Input:** WebArena environment $\mathcal{E}$ and base agent model $\mathcal{M}$ **Output:** A set of in-domain synthetic training examples $\mathcal{D}_{\text{IN-DOMAIN}}$

```

1: Initialize  $\mathcal{P} \leftarrow \emptyset$  ▷ Set of plausible trajectories
2: for  $\mathbf{i}$  in WebArena benchmark do
3:   Initialize trajectory  $\mathcal{X} \leftarrow \emptyset$ 
4:   Initialize observation  $o_0 \leftarrow \text{INITIALOBSERVATION}(\mathcal{E}, \mathbf{i})$ 
5:   Initialize action  $a_{-1} \leftarrow \text{null}$ 
6:   for  $t = 0$  to  $T$  do
7:      $a_t \leftarrow \text{RUNAGENT}(\mathcal{M}, \mathbf{i}, o_t, a_{t-1})$ 
8:     Append  $(\mathbf{i}, o_t, a_{t-1}, a_t)$  to  $\mathcal{X}$ 
9:     if  $a_t = \text{stop}$  or  $\text{ENVIRONMENTERROR}(\mathcal{E}, a_t, o_{t+1})$  then
10:      break
11:    end if
12:     $o_{t+1} \leftarrow \mathcal{T}(o_t, a_t)$  ▷ Observe updated state
13:  end for
14:  if not  $\text{SELFCRITIQUE}(\mathcal{X})$  and not  $\text{ISREFUSAL}(\mathcal{X})$  and not  $\text{HASERROR}(\mathcal{X})$  then
15:    Append  $\mathcal{X}$  to  $\mathcal{P}$  ▷ Filter out low-quality trajectories  
to only keep plausible trajectories
16:  end if
17: end for
18: Initialize  $\mathcal{D}_i, \mathcal{D}_f, \mathcal{D}_{\text{int}} \leftarrow \emptyset$  ▷ Set of initial steps, final steps, intermediate steps
19: for  $\mathcal{X}$  in  $\mathcal{P}$  do
20:   Append  $\mathcal{X}_0$  to  $\mathcal{D}_i$ 
21:   Append  $\mathcal{X}_T$  to  $\mathcal{D}_f$ 
22:   for  $t = 1$  to  $T - 1$  do
23:     Append  $\mathcal{X}_t$  to  $\mathcal{D}_{\text{int}}$ 
24:   end for
25: end for
26:  $\mathcal{D}_{\text{IN-DOMAIN}} \leftarrow \text{RANDSAMPLE}(\mathcal{D}_i, |\mathcal{D}_i|) \cup \text{RANDSAMPLE}(\mathcal{D}_f, |\mathcal{D}_f|) \cup \text{RANDSAMPLE}(\mathcal{D}_{\text{int}}, 2 * |\mathcal{D}_i|)$ 
27: return  $\mathcal{D}_{\text{IN-DOMAIN}}$ 

```

to produce a valid, parsable generation. The final dataset of synthetic examples is balanced by randomly sampling an equal number of initial steps ($t = 0$), final steps ($t = T$), and intermediate steps ($t = 1 \dots (T - 1)$) from the plausible trajectories in $\mathcal{P}$. In Table 5.5, we display how effective this unsupervised filtering process is by measuring the accuracy, precision, and recall of the 58 remaining trajectories kept in $\mathcal{P}$ from the 812 total trajectories to assess the proportion of correct/incorrect examples in $\mathcal{D}_{\text{IN-DOMAIN}}$.

Set of Trajectories	#	Accuracy	F1	Precision	Recall
All Trajectories	812	0.071	0.133	0.071	1.000
Plausible Trajectories $\mathcal{P}$	58	0.919	0.431	0.431	0.431

Table 5.5: Metrics on the proportion of trajectories that successfully completed the task in the set of plausible trajectories kept in $\mathcal{P}$ after filtering out low-quality trajectories. Approximately 43% of trajectories in $\mathcal{P}$ successfully completed the task, up from $\sim 7\%$ with no filtering, indicating useful learning signal is introduced by filtering using self-critiques and information from the environment.

Out-of-Domain Synthetic Data Generation

Using examples from $\mathcal{D}_{\text{IN-DOMAIN}}$ as seed examples, we prompt our base LLM $\mathcal{M}$ to synthetically generate completely novel tasks, objectives, web pages, and solution trajectories to produce $\mathcal{D}_{\text{OUT-OF-DOMAIN}}$ in Algorithm 2.

Algorithm 2 Generate Out-of-Domain Synthetic Training Examples $\mathcal{D}_{\text{OUT-OF-DOMAIN}}$

Input: Base LLM model $\mathcal{M}$ and $\mathcal{D}_{\text{IN-DOMAIN}}$

Output: A set of out-of-domain synthetic training examples $\mathcal{D}_{\text{OUT-OF-DOMAIN}}$

```

1: Initialize  $\mathcal{D}_{\text{OUT-OF-DOMAIN}} \leftarrow \emptyset$  ▷ Set of out-of-domain synthetic training examples
2: Initialize  $\mathcal{I} \leftarrow \{\mathbf{i} \mid \mathbf{i} \in \text{WebArena benchmark}\}$  ▷ Set of 812 objectives in WebArena
3: Initialize  $\mathcal{I}^* \leftarrow \emptyset$  ▷ Set of previously generated objectives
4: for  $j = 1$  to  $|\mathcal{D}_{\text{IN-DOMAIN}}|$  do
5:   while true do
6:      $\mathbf{i}^* \leftarrow \text{GENERATEOBJECTIVE}(\mathcal{M}, \text{RANDSAMPLE}(\mathcal{I}, 2) \cup \text{RANDSAMPLE}(\mathcal{I}^*, 2))$ 
7:     if  $\max(\text{sim}(\mathbf{i}^*, \mathcal{I}^*)) < 0.70$  then ▷ Ensure generated objectives are diverse
8:       Append  $\mathbf{i}^*$  to  $\mathcal{I}^*$ 
9:       break
10:    end if
11:  end while
12:   $\mathbf{p}^* \leftarrow \text{GENERATEPLAN}(\mathcal{M}, \mathbf{i}^*)$  ▷ Generate an outline of a hypothetical solution trajectory
13:   $k \leftarrow \text{RANDCHOICE}(\{1, \dots, |\mathbf{p}^*|\})$  ▷ Randomly select one of the steps in the plan, weighted
to equally balance initial, final, and intermediate steps
14:   $a_{t-1}^*, a_t^* \leftarrow \text{GENERATEACTIONS}(\mathcal{M}, \text{RANDSAMPLE}(\mathcal{D}_{\text{IN-DOMAIN}}, 2), \mathbf{i}^*, \mathbf{p}^*, k)$ 
15:   $o_t^* \leftarrow \text{GENERATEOBSERVATION}(\mathcal{M}, \text{RANDSAMPLE}(\mathcal{D}_{\text{IN-DOMAIN}}, 2), \mathbf{i}^*, \mathbf{p}^*, k)$ 
16:  Append  $(\mathbf{i}^*, o_t^*, a_{t-1}^*, a_t^*)$  to  $\mathcal{D}_{\text{OUT-OF-DOMAIN}}$ 
17: end for
18: return  $\mathcal{D}_{\text{OUT-OF-DOMAIN}}$ 

```

When generating new objectives, we use 4 few-shot examples (two objectives sampled from tasks in WebArena and two sampled from previously generated objectives). We use 2 few-shot examples when generating previous actions, next actions, and observations (web pages in the form of accessibility trees). We use a temperature of 1.0 and set top-p to 1.0 during generation. When generating novel objectives, we specifically prompt the model to generate objectives that are dissimilar to the example objectives to encourage out-of-domain generations. We also ensure each novel objective has < 0.70 cosine similarity with any objective previously generated using the `all-distilroberta-v1` sentence similarity model [178, 197, 198] to promote diversity. Table 5.6 gives examples of out-of-domain objectives that our method generated.

Objectives in WebArena Benchmark	Generated Out-of-Domain Objectives
• Tell me the total cost of my latest pending order? (Shopping) • Compare the time for walking and driving route from AMC Waterfront to Univ of Pittsburgh (Maps) • Check out the most recent open issues (GitLab) • Which customer has placed 2 orders in the entire history? (Shopping Admin) • ...	• Locate and purchase a subscription to The Economist digital edition (https://store.economist.com/...) • Find the nutrition facts for a Grilled Chicken Caesar Salad from Chili's (http://www.chilis.com/...) • Find the active coupons for a one-year subscription to Adobe Creative Cloud (https://www.couponcabin.com/...) • Subscribe to the premium plan for Grammarly to unlock advanced writing features. (https://www.grammarly.com/...) • ...

Table 5.6: A sample of the novel objectives generated compared with the objectives found in WebArena.

We show a full sample of a generated out-of-domain synthetic example in Figure 5.5.

Objective	Subscribe to the premium plan for Grammarly to unlock advanced writing features.
URL	https://www.grammarly.com/plans 1. Click on the "Go Premium" button located at the top right corner of the page. 2. Scroll down to review the available plans (if needed). 3. Click on the "Choose Plan" button for the preferred premium plan. 4. Provide payment information, such as credit card details, in the respective input fields. 5. Enter your billing information, including name, address, and zip code (if required). 6. Type your email address again to confirm it. 7. Create a password for your Grammarly account and type it into the password field. 8. Check the box to agree to the terms and conditions. 9. Click on the "Start Your Free Trial" (or similar) button if you qualify for a trial. 10. Review the confirmation message and wait for the subscription activation. 11. Stop when the subscription is successfully activated and the premium features are unlocked.
Plan	
Selected Step in Plan	3
Previous Action	click [1098] where [1098] is Generic 'Premium'
Next Action	Let's think step-by-step. To proceed with subscribing to the premium plan, the user must now choose their plan. They have already clicked on the 'Premium' plan option. The next logical action is to click on the 'Choose Plan' button to finalize the selection and proceed to the payment page. In summary, the next action I will perform is ``click [1105]`` to select the premium plan and move to the next step in the subscription process.
Observation	[1] RootWebArea 'Grammarly plans - Premium, Business & Student Writing Tools' focused: True [125] Generic 'Grammarly plans - Premium, Business & Student Writing Tools' [127] link 'Privacy Policy' [130] link 'Terms of Use' [133] link 'Cookies' [136] link 'TrustArc Privacy Notice' [587] link 'Language' [1054] link 'United States English' [592] link 'Logout' [595] link 'Invite' [598] link 'Get the Extension' [128] link 'Login' [601] link 'Sign Up' [580] Generic 'Plans' [583] div 'Free' [586] link 'Get Started' [590] Generic 'Premium' [1098] Generic 'Premium' haspopup: submenu expanded: False [640] link 'View Features' [643] StaticText 'Start Now!' [1105] button 'Choose Plan' [1089] Generic 'Business' [649] link 'View Pricing' [652] StaticText 'Get Quote' [1092] Generic 'Student' [646] link 'View Pricing' [649] StaticText 'Start Now!' [566] Generic 'Write with confidence, wherever you work' [570] StaticText 'Powerful writing tools for work, school, and everything in between.' [574] button 'Learn More & try it Free' [1052] table ' [569] row ' [574] rowheader 'Free' [575] gridcell ' [568] row ' [573] rowheader 'Premium' [574] gridcell ' [567] row ' [572] rowheader 'Business' [573] gridcell ' [566] row ' [571] rowheader 'Student' [572] gridcell '

Figure 5.5: A selected full sample of a generated novel, out-of-domain synthetic training example in $\mathcal{D}_{\text{OUT-OF-DOMAIN}}$.

Training

We perform a number of experiments fine-tuning agent models on the synthetic training data mixtures and assess the extent to which the agent model has self-improved over base agent model $\mathcal{M}$ with our evaluation metrics. Table 5.7 displays the results of these experiments.

Baseline Agent Performance

As baselines, we evaluate our base agent model $\mathcal{M}$ as well as implement a trivial agent that always outputs stop [N/A]. A number of tasks in WebArena can be solved by this trivially implementable agent or a weak model that always refuses to continue and exits immediately, therefore, our trivial agent baseline helps discriminate which tasks being completed successfully should contribute to an agent being meaningfully capable when computing the capability score.

Self-Improvement Fine-Tuned Agent Performance

We fine-tune our base agent model $\mathcal{M}$ on the 3 synthetic dataset mixtures previously discussed:

1) $\mathcal{D}_A = \mathcal{D}_{\text{IN-DOMAIN}}$ 2) $\mathcal{D}_B = \mathcal{D}_{\text{IN-DOMAIN}} \cup \mathcal{D}_{\text{OUT-OF-DOMAIN}}$ and 3) $\mathcal{D}_C = \mathcal{D}_{\text{OUT-OF-DOMAIN}}$ with a straightforward auto-regressive loss using QLoRA [199, 68]:

$$L_{\text{FT}}(\theta) = -\mathbb{E}_{[(\mathbf{i}, o_t, a_{t-1}), a_t] \sim \mathcal{D}} [\log P_{\theta}(a_t \mid (\mathbf{i}, o_t, a_{t-1}))]$$

to produce $\mathcal{M}_A$, $\mathcal{M}_B$, and $\mathcal{M}_C$. We perform a 90/10% train-validation split of our datasets and train with an early stopping patience of 5 epochs, using a batch size of 16 examples and a learning rate of 1e-5.

Iterative Self-Improvement Fine-Tuned Agent Performance

We also experiment with iterative self-improvement [83] to assess whether further improvement can be gained from a subsequent round of our self-improvement procedure. We perform this experiment on Mixture A. It is conceivable that after fine-tuning on $\mathcal{D}_A^1$, filtering from a set of trajectories with higher performance might yield a stronger set of plausible trajectories⁴ to produce $\mathcal{D}_A^2$. Mixtures B and C are less likely to demonstrate improvement over a subsequent round since the fine-tuned models are not specifically trained to generate better synthetic out-of-domain examples.

5.2.2 Results

We perform evaluation using the standard metrics proposed by the WebArena benchmark like functional correctness [95] as well as evaluate with new auxiliary metrics we propose that give more nuanced insight into an agent’s performance.

Agent Model	Functional Correctness	Capability	VERTEX _{DTW}
<i>Baseline Agents</i>			
Trivial Agent	4.68	0.00	-
Base Agent Model ($\mathcal{M}$)	7.14	15.44	0.35
<i>Self-Improved Agents</i>			
Agent Model Fine-Tuned on Mixture A ($\mathcal{M}_A$)	8.87	19.12	0.38
Agent Model Fine-Tuned on Mixture B ($\mathcal{M}_B$)	9.36	19.12	0.35
Agent Model Fine-Tuned on Mixture C ($\mathcal{M}_C$)	6.16	16.91	0.28
<i>Iterative Self-Improved Agents</i>			
Agent Model 2x Fine-Tuned on Mixture A ($\mathcal{M}_A^2$)	8.37	16.91	0.37

Table 5.7: Evaluation metrics on WebArena for baseline agents and self-improved agent models.

⁴To maximize data for iterative self-improvement, during filtering, we also fallback to checking the base model trajectory for a task if the self-improved model’s trajectory for a task is filtered out.

Functional Correctness Score

Functional correctness is the standard metric proposed by the WebArena benchmark that is a simple binary task completion score (0 or 1) averaged over all 812 tasks in the benchmark.

Capability Score

While WebArena contains 812 unique task instances, these 812 tasks are instantiated using natural language intent templates like “What is the minimum travel time by car from {{location1}} to {{location2}}?”. Therefore, many tasks actually test the same *capability*. Aggregate-level metrics like the functional correctness score may be misleading since improvements may only be due to the model becoming more robust at solving capabilities it already could solve versus demonstrating the ability to solve new capabilities that were previously unsolvable. There are 241 unique templates in WebArena that are used to instantiate 812 tasks. Moreover, some of these templates are simple paraphrases of each other. For example, “What is the estimated driving time between {{city1}} and {{city2}}?” is a paraphrase of the prior template. Using a sentence similarity model,⁵ we iteratively group these templates into a set of unique capabilities. Each template is grouped with any existing capability if it has a similarity of > 0.60 with any template in the group, otherwise the template is added to a new capability group. This results in 136 unique capabilities. A model receives a score of 1 for each capability group with at least one successful task completed, otherwise it receives a score of 0.⁶ The capability score is then the averaged over all 136 capabilities.

We note, however, that a number of tasks in the WebArena benchmark are trivial tasks and can be solved by a trivial baseline agent or weak model that performs no actions and only immediately exits by always generating stop [N/A]. In the capability score computation, we

⁵We use the all-distilroberta-v1 sentence similarity model [198].

⁶Since we do not count trivial tasks as a successful completion, a single successful completion of a capability provides sufficient evidence of acquisition. We discuss robustness and consistency separately in Section 5.2.3.

do not count such trivial tasks as evidence a model can perform the capability as these are degenerate cases of the capability.

VERTEX_{DTW} Score

Both functional correctness and the capability score only evaluate task completion, however, they do not assess the quality of entire trajectories, therefore, a measure that is sensitive to incremental improvements and degradations in trajectories, independent of task completion, is desirable. We extend the recently proposed VERTEX score [200], which measures the similarity of two relational trajectories by using embeddings to compare node distributions within a computational graph. The VERTEX score integrates the semantic meaning across the distributional path by computing at each node the cross-similarity between the generated embeddings and embeddings sampled from a reference distribution. An ideal reference distribution would be ground-truth reference trajectories produced by humans for all of the WebArena tasks. In absence of this, we use a larger, stronger model, GPT-4 [60], to collect three reference trajectories for each task.

One obstacle to the straightforward application of the VERTEX score is the assumption that both trajectories are of the same length. Agents operating in complex environments, however, are not constrained to a fixed-length for the trajectories they produce. Therefore, we propose modification in the computation of the VERTEX score that enables comparison of sequences with different lengths. Our extension consists of an additional alignment step prior to calculating the VERTEX score for the aligned trajectories. First, we embed all steps of a trajectory $\mathcal{X}$ as $e_t = f(o_t, a_t) \in R^d$, where f is an embedding model⁷ with embedding dimension d . The embedding model f is independent of both the model that generated the reference trajectories as well as the model that generated the test trajectories. Then, we use *Dynamic Time*

⁷We use the `all-mpnet-base-v2` embedding model [201].

Warping (DTW) [202] to align two embedded trajectories $\tilde{\mathcal{X}}_m = (e_0, \dots, e_i, \dots, e_m) \in R^{m \times d}$ and $\tilde{\mathcal{X}}_n = (e_0, \dots, e_j, \dots, e_n) \in R^{n \times d}$ with length m and n , respectively. Consequently, we refer to our proposed measure as $\text{VERTEX}_{\text{DTW}}$. DTW returns an alignment path ν of length T , where each $e_i \in \tilde{\mathcal{X}}_m$ is aligned with a corresponding $e_j \in \tilde{\mathcal{X}}_n$, preserving the order in their respective trajectory. This order preservation occurs because once a node is matched, it is excluded from potential new matches, maintaining the integrity of the temporal alignment. As a scoring function for DTW, we choose cosine distance. In addition to the alignment step, we introduce a linear distance decay factor that decreases the contribution of aligned embeddings if they are far apart in the original trajectories. Once two trajectories are aligned, we compute the VERTEX score by Eq. (4) in [200] with the addition of the distance decay. Therefore, the $\text{VERTEX}_{\text{DTW}}$ score is computed as:

$$s(\tilde{\mathcal{X}}_{\text{ref}}, \tilde{\mathcal{X}}_{\text{test}}, \nu) := \frac{1}{T} \int_{t_0}^{t_T} \left[\min\left(\max\left(0, \frac{1}{1 + |i_{\nu_t} - j_{\nu_t}|}\right) \widetilde{\text{MMD}^2}(e_{\text{ref}}^{\nu_t}, e_{\text{test}}^{\nu_t}) - z_{\text{rand}}, 1\right) \right] dt, \quad (1)$$

where i_{ν_t} and j_{ν_t} are the position indices in the alignment path ν at time t , $\tilde{\mathcal{X}}_{\text{ref}}$ and $\tilde{\mathcal{X}}_{\text{test}}$ are aligned trajectories of embeddings from the reference set and the model under test, respectively, and z_{rand} is a baseline correction from a random baseline.⁸ Furthermore, if we have multiple reference sequences for a given task, we compute the $\text{VERTEX}_{\text{DTW}}$ score for every reference sequence and choose the maximum score, under the assumption that they describe different paths for solving the task.

⁸We use the trivial agent implementation described in Section 5.2.1 for baseline correction in our computation.

5.2.3 Discussion

We summarize key results from our experiments as well as discuss insights towards the efficacy of our self-improvement procedures for complex, multi-step tasks like web agent tasks.

Can models self-improve at web agent tasks?

We find fine-tuning on both Mixtures A and B improve overall benchmark performance with the best performing mixture, Mixture B, completing 18 more tasks correctly, a 31% relative improvement ($7.14 \rightarrow 9.36$). Training on all Mixtures A, B, and C demonstrate self-improvement on at least one metric, with $\mathcal{M}_C$ showing a gain on capability score.

Do self-improved agents acquire new capabilities?

We find agent models can acquire new capabilities through self-improvement, however, they also may lose the ability to perform some capabilities. In net, all of our self-improved agents acquire more capabilities than they lose. We find fine-tuning on both Mixtures A and B improve the capability score equally and lead to the largest net acquisition of capabilities demonstrating 5 more capabilities than the base agent model, a 24% relative improvement ($15.44 \rightarrow 19.12$). We find all agent models demonstrate at least one new capability that no other agent model demonstrates, for example, only $\mathcal{M}_C$ successfully completes the “Fork {{repo}}” capability on the GitLab website. Interestingly, we find that the majority of capabilities acquired by $\mathcal{M}_A$ and $\mathcal{M}_C$ are mutually exclusive, suggesting in-domain synthetic examples and out-of-domain synthetic examples improve acquisition of different capabilities. We list all capabilities $\mathcal{M}_A$, $\mathcal{M}_B$, and $\mathcal{M}_C$ acquire and lose compared to $\mathcal{M}$ in Table 5.8.

Agent Model	Net Change	Capabilities Acquired	Capabilities Lost
$\mathcal{M}_A$	+5	1. Tell me the count of comments that have received more downvotes than upvotes for the user who made the latest post on the <code>{{forum}}</code> forum. 2. Find a subreddit focused on topics related to <code>{{topic}}</code>, and post my question, "<code>{{question}}</code>" there 3. Measure distance between <code>{{location/address.1}}</code> and <code>{{location/address.2}}</code> by walking 4. Tell me the coordinates of <code>{{location}}</code> in DD format 5. Show me the "<code>{{product}}</code>" listings by <code>{{sorting_order}}</code>. 6. Open my latest updated issue that has keyword "<code>{{keyword}}</code>" in its title to check if it is closed 7. Reply to <code>{{position_description}}</code> with my comment "<code>{{content_description}}</code>" 8. Tell me the distance to drive from Carnegie Mellon University to the top computer science school in massachusetts 9. How many commits did <code>{{user}}</code> make on <code>{{date}}</code> in total?	1. Tell me the total cost of my latest <code>{{status}}</code> order? 2. Checkout merge requests assigned to me 3. Today is 6/12/2023. Tell me how many fulfilled orders I have <code>{{period}}</code>, and the total amount of money I spent. 4. Subscribe to the newsletter of OneStopMarket

Continued on next page...

$\mathcal{M}_B$

+5

1. Tell me the count of comments that have received more downvotes than upvotes for the user who made the latest post on the `{{forum}}` forum.
 2. What is the minimum travel time by car from `{{location1}}` to `{{location2}}`?
 3. Find a subreddit focused on topics related to `{{topic}}`, and post my question, "`{{question}}`" there
 4. See all public projects
 5. Set my gitlab status as `{{status}}`.
 6. Show me the route and driving time from `{{city1}}` to `{{city2}}`
 7. Ask for advice about `{{issue}}` in a subreddit for relations
 8. Show me the "`{{product}}`" listings by `{{sorting_order}}`.
 9. Reply to `{{position_description}}` with my comment "`{{content_description}}`"
 10. Show me the way from `{{location}}` to the home stadium of `{{sport_team}}` `{{time}}`
1. Checkout merge requests assigned to me
 2. Today is 6/12/2023. Tell me how many fulfilled orders I have `{{period}}`, and the total amount of money I spent.
 3. Subscribe to the newsletter of OneStopMarket
 4. Show me the command to clone `{{repo}}` with SSH.
 5. Show me the `{{info}}` for order number `{{order_number}}`.

Continued on next page...

$\mathcal{M}_C$	+2	1. Fork <code>{{repo}}</code>. 2. Tell me the count of comments that have received more downvotes than upvotes for the user who made the latest post on the <code>{{forum}}</code> forum. 3. Which US states border <code>{{state}}</code>? 4. What is the minimum travel time by car from <code>{{location1}}</code> to <code>{{location2}}</code>? 5. See all public projects 6. I previously ordered some <code>{{product}}</code> <code>{{time}}</code> and later cancelled. Can you reorder it for me? 7. Today is 3/15/2023, generate a <code>{{report}}</code> <code>{{time-span}}</code> 8. Ask for advice about <code>{{issue}}</code> in a subreddit for relations 9. Show me the "<code>{{product}}</code>" listings by <code>{{sorting-order}}</code>. 10. Pull up the description page of <code>{{location}}</code> on Map 11. Reply to <code>{{position_description}}</code> with my comment "<code>{{content_description}}</code>" 12. Edit my post on <code>{{post}}</code> by adding a line to the body that says "<code>{{content}}</code>" 13. Tell me who has made the most contributions, in terms of number of commits, to the <code>{{repo}}</code> project 14. List the top <code>{{n}}</code> search terms in my store	1. How many commits did <code>{{user}}</code> make to <code>{{repo}}</code> on <code>{{date}}</code>? 2. Checkout merge requests assigned to me 3. What is the estimated driving time between <code>{{city1}}</code> and <code>{{city2}}</code>? 4. Open the thread of a trending post on the forum "<code>{{subreddit}}</code>" and subscribe. 5. Today is 6/12/2023. Tell me how many fulfilled orders I have <code>{{period}}</code>, and the total amount of money I spent. 6. Find the <code>{{space}}</code> around <code>{{location}}</code> 7. What are the main criticisms of this product? Please extract the relevant sentences. 8. Subscribe to the newsletter of OneStopMarket 9. Show me the command to clone <code>{{repo}}</code> with SSH. 10. I want to browse the products in the <code>{{category}}</code> category 11. Show me the <code>{{info}}</code> for order number <code>{{order_number}}</code>. 12. Tell me the total cost of my latest <code>{{status}}</code> order?
-----------------	----	--	---

Table 5.8: Capabilities acquired and lost compared to the base agent model $\mathcal{M}$, along with the net change in the total number of capabilities demonstrated, for each self-improved fine-tuned agent model.

Are self-improved agents more robust?

For $\mathcal{M}_B$, we find a larger improvement in functional correctness (31%) than in capability score (24%), which supports that the agent model is improving at more consistently succeeding at tasks belonging to the same capability, an indicator of one type of robustness. $\mathcal{M}_C$ is less robust by the same measure. Moreover, the capability analysis in Table 5.8 also shows both $\mathcal{M}_A$ and $\mathcal{M}_B$ after self-improvement still demonstrate the majority of capabilities demonstrated by the base agent model $\mathcal{M}$, whereas $\mathcal{M}_C$ only demonstrates a minority. This would indicate $\mathcal{M}_A$ and $\mathcal{M}_B$ more reliably maintain the capabilities of the base agent model after self-improvement, a measure of robustness that would be useful in deployed settings where users of agent models may desire stability in performance.

Is there an effect on the quality of generated trajectories?

Fine-tuning on Mixtures A and B show no degradation in the quality of generated trajectories and show small improvement towards the reference on VERTEX_{DTW}. Fine-tuning on Mixture C degrades the the quality of generated trajectories from the reference. Training on the out-of-domain synthetic examples allows $\mathcal{M}_C$ to demonstrate some unique capabilities no other agent model demonstrates, however, inspecting trajectories from $\mathcal{M}_C$, we find this comes with trade-offs. For example, compared with $\mathcal{M}_A$, we find $\mathcal{M}_C$ produces longer trajectories ($\sim 1.6x$) and produces more invalid actions ($\sim 3.9x$). In comparison with $\mathcal{M}$, $\mathcal{M}_A$ and $\mathcal{M}_B$ do not greatly increase trajectory length ($\sim 1.1x$ and $\sim 1.3x$) or the rate of invalid actions ($\sim 1x$ and $\sim 1.3x$), further explaining the quality difference VERTEX_{DTW} highlights. Due to lack of human reference, the reliability of this evaluation is limited. In Table 5.9, we compute variants of VERTEX_{DTW}, weighting by capability and filtering out trivial tasks. We find these variants make little difference in the relative ranking of agent models.

Agent Model	VERTEX _{DTW}	VERTEX _{DTW-by-cap}	VERTEX _{DTW-notrivial}
<i>Baseline Agents</i>			
Base Agent Model ($\mathcal{M}$)	0.35	0.40	0.38
<i>Self-Improved Agents</i>			
Agent Model Fine-Tuned on Mixture A ($\mathcal{M}_A$)	0.38	0.42	0.42
Agent Model Fine-Tuned on Mixture B ($\mathcal{M}_B$)	0.35	0.40	0.40
Agent Model Fine-Tuned on Mixture C ($\mathcal{M}_C$)	0.28	0.33	0.34
<i>Iterative Self-Improved Agents</i>			
Agent Model 2x Fine-Tuned on Mixture A ($\mathcal{M}_A^2$)	0.37	0.41	0.43

Table 5.9: Variants of the VERTEX_{DTW} score metric: 1) computed over all trajectories 2) weighting the trajectories by capability 3) weighting the trajectories by capability and filtering out trajectories for trivial tasks.

Can models iteratively self-improve at web agent tasks?

Our results are consistent with prior works such as [83] and [203] and we find diminishing returns to successive rounds of self-improvement and training on synthetic data. While the agent model after a second round of self-improvement outperforms the base agent model, it does not perform any better than agent models with a single round of self-improvement. We analyze the set of plausible trajectories in the second round in Table 5.10 and find that while more synthetic training examples can be collected, they are of lower quality and contain a higher proportion of failed trajectories.

Set of Trajectories	#	Accuracy	F1	Precision	Recall
All Trajectories	812	0.071	0.133	0.071	1.000
Plausible Trajectories $\mathcal{P}^1$	58	0.919	0.431	0.431	0.431
Plausible Trajectories $\mathcal{P}^2$	131	0.825	0.317	0.252	0.429

Table 5.10: Metrics on the proportion of trajectories that successfully completed the task in the set of plausible trajectories kept in $\mathcal{P}$ after filtering out low-quality trajectories for each iterative round of self-improvement. On the second round of self-improvement, we keep 131 plausible trajectories making our potential synthetic training dataset larger, however, the accuracy and P/R/F1 metrics indicate it would be a lower quality dataset to fine-tune on.

5.2.4 Summary of Contributions

This work is based on our paper “Large Language Models Can Self-Improve At Web Agent Tasks” [204]. I executed the overall research direction with collaborative input from my co-authors. I conducted the synthetic data generation, training experiments, and agent evaluation experiments. My co-authors contributed to some of the trajectory quality evaluation experiments. I was the primary author responsible for writing the paper, and my co-authors helped edit and rewrite the paper.

5.3 EXAMPLE 3: ENABLING VISION LANGUAGE MODELS TO UNDERSTAND TEXT-RICH FIGURES

Vision-language models (VLMs) that are instruction-tuned have demonstrated strong performance across a variety of multimodal tasks [205, 60, 206]. However, these tasks generally prioritize understanding natural images rather than the specific reasoning needed for interpreting text-rich images like charts, documents, diagrams, signs, labels, and screenshots. Being able to reason over these types of images is critical for a range of applications, including analyzing scientific texts and figures [207], enhancing accessibility for people with visual impairments [208], and enabling real-world agentic workflows [209]. These structured visual formats require a combination of text comprehension and spatial reasoning, areas where current models often fall short, in part because there is a lack of high-quality, realistic, and varied vision-language datasets [106].

To tackle these challenges, we developed the **Code Guided Synthetic** data generation (**CoSyn**) system, a flexible framework designed to generate diverse synthetic multimodal data for vision-language instruction tuning. As depicted in Figure 5.7, CoSyn can create multimodal data for a variety of domains from a short natural language query, such as *book covers*. It leverages text-only LLMs, which are highly effective at generating code, to create both the data and the code needed to render a broad range of text-rich images using 11 supported rendering tools (e.g., Python, HTML, LaTeX). This framework also generates corresponding textual instructions, based on the code representation of the images, for building vision-language instruction-tuning datasets.

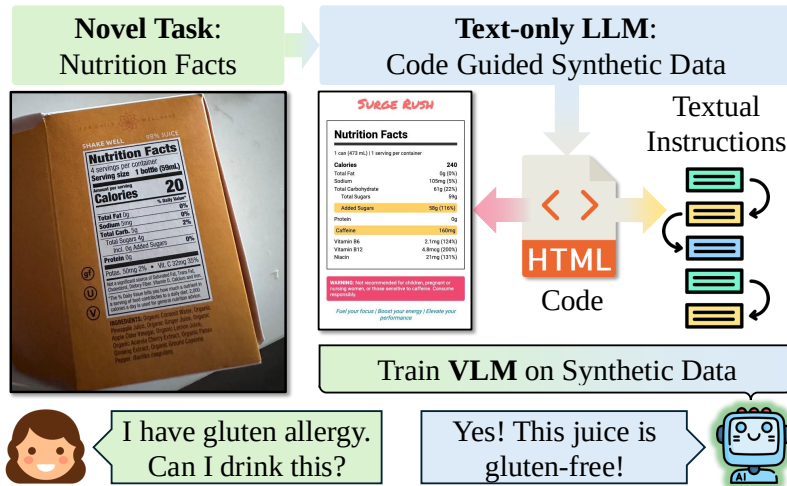


Figure 5.6: Given a novel task (e.g., answering questions about nutrition facts), our code-guided generation system can produce targeted synthetic data to enhance the performance of VLMs on that specific task.

Given a text query q about an image type, e.g., *flow charts*, our goal is to create a synthetic multimodal dataset $\mathcal{D}_q = \{(I, T)\}$, where I is the image, and T is the textual instruction-tuning data (e.g., question-answer pairs). $\mathcal{D}_q$ is used to train a VLM to improve its ability to understand images related to q . The core idea of our approach is to use code C as the intermediate

representation to bridge the image and text. The overall generation process can be decomposed as follows:

$$P(I, T|q) = P_{\text{LM}}(C|q) \cdot P(I|C) \cdot P_{\text{LM}}(T|C)$$

where $P_{\text{LM}}(C|q)$ represents prompting a language model to generate code C , which is executed to render the image, $P(I|C)$. $P_{\text{LM}}(T|C)$ uses code C (without the image) as context for an LLM to generate the textual instruction-tuning data.

5.3.1 CoSyn System For Synthetic Data Generation

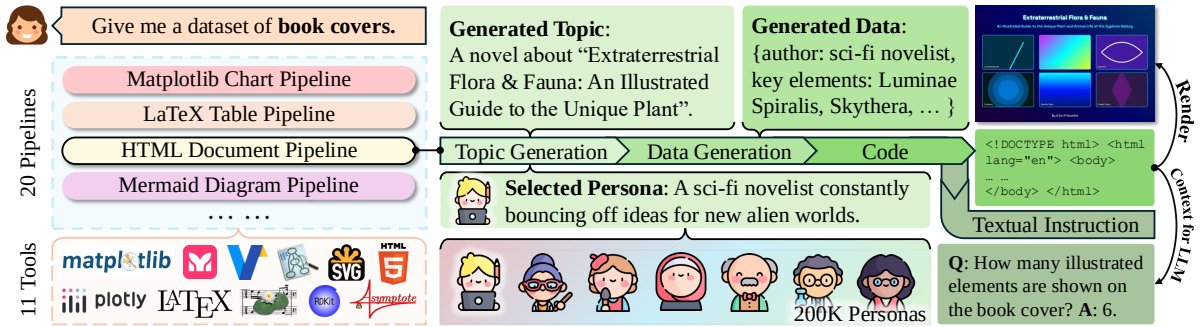


Figure 5.7: The overview of our **Code Guided Synthetic** data generation system (**CoSyn**), which has 20 generation pipelines based on 11 render tools. Given a user query, e.g., “book cover,” CoSyn selects the appropriate pipelines and starts by generating diverse topics conditioned on personas, then synthesizes detailed data for code generation. The code renders the image and is also fed as context for an LLM to construct instruction-tuning data.

Figure 5.7 illustrates the workflow of our **Code-Guided Synthetic** data generation system (**CoSyn**). The system takes a language input, such as “generate a dataset of book covers”, and outputs a multimodal dataset. Based on the input query, CoSyn selects one of 20 generation pipelines built on 11 rendering tools. The process starts with topic generation, conditioned on a sampled persona that guides the style and content. Next, the system generates data content

and converts it into code, which is then executed to render synthetic images. Finally, using the code as context, we prompt the LLM to generate corresponding textual instructions.

The following provides detailed explanations of the rendering tools supported by CoSyn, the tailored generation pipelines based on these tools, our persona-driven approach to diversifying content and styles, and the large-scale dataset of 400K synthetic images generated by CoSyn.

Rendering Tools

We integrate various rendering tools to generate diverse types of images, forming the foundation of CoSyn’s ability for text-rich image generation. For example, [Matplotlib](#), [Plotly](#), and [Vega-Lite](#) are used to create different types of charts. LaTeX and HTML are used for documents and tables, while [Mermaid](#) and [Graphviz](#) generate diagrams. We utilize SVG and [Asymptote](#) to create vector graphics and math-related content. For specialized tasks, we rely on [Lilypond](#) to generate music sheets and [RDKit](#) for chemical structures. We implement customized functions for each tool to execute LLM-generated code and obtain corresponding rendered images. These tools collectively enable CoSyn to produce a wide range of high-quality, text-rich synthetic images.

Pipelines

We design 20 pipelines based on 11 rendering tools.⁹ Each pipeline follows the same procedure: (1) *Topic generation* to define the theme of this synthetic example, (2) *Data generation* to populate the detailed contents, (3) *Code generation* to create executable code that renders the image, and (4) *Instruction generation* conditioned on code to produce instructions, including questions, answers and explanations for chain-of-thought reasoning. Each stage is controlled

⁹Some tools are used in multiple pipelines, e.g., HTML is used for generating documents, tables, and charts.

by a prompt customized for the image category and rendering tool. Examples of the prompts for HTML Document Pipeline can be found below:

Topic Generation: You are an expert in document generation and have a broad knowledge of different topics.

My persona is: "**PERSONA**" I want you to generate **NUM_TOPICS** topics for **FIGURE_TYPE** that I will be interested in or I may see during my daily life given my persona.

Here are the requirements:

1. Each topic is a high-level summary of the contents in **FIGURE_TYPE** with some design details, e.g., "the utility bill for the month of January 2022 with a detailed breakdown of charges".
2. The topics should be diverse to help me generate varied documents. Each topic should be unique and not overlap with others.
3. The topics are conditioned on the document type. Please ensure the topics you provided can be best visualized in "**FIGURE_TYPE**".
4. All topics must be in English, even if the persona is non-English.
5. List **NUM_TOPICS** topics for "**PERSONA**" and separate them with a | character, e.g., topic1 | topic2 | | topicN.

Do not include any additional text at the beginning or end of your response.

Data Generation: You are an expert in content creation and have broad knowledge about various topics.

My persona is: "**PERSONA**" I need some materials about "**TOPIC**", which can be used to generate a **FIGURE_TYPE**.

Here are the requirements:

1. The materials should be related to the topic and customized according to my persona. Its structure must be suitable for the **FIGURE_TYPE**.
2. The materials should be realistic, and the contents should be named using real-world entities. Do not use placeholder names like xxA, xxB, etc. Do not use template data like [Name], [Date], etc.
3. The materials should be diverse and contain information from different aspects of the topic to ensure the document is informative.
4. Do not provide too many materials. Just provide key pieces of information that are essential for a ****one-page document.****
5. All materials must be in English, even if the persona is non-English.

Please provide the materials in JSON format without additional text at the beginning or end.

Code Generation: You are an expert web designer and are good at writing HTML to create documents.

My persona is: "**PERSONA**" I have some materials about **TOPIC** which can be used to generate a **FIGURE_TYPE**.

Here are the materials (JSON format):

<data> **DATA** </data>

Please use HTML and CSS to generate a **FIGURE_TYPE** using the data provided.

Here are the requirements:

1. **Style Requirements**: Feel free to use any CSS framework, libraries, JavaScript plugins, or other tools to create the document.

(1) Try to be creative and make the web page style, fonts, colors, borders and visual layout unique with CSS. Taking persona, topic, and document type into consideration when designing the document.

(2) Select the appropriate design scale (e.g., margins, page size, layout, etc) to ensure the information in the document is clear and easy to understand, with no text overlapping, etc.

(3) **Do not make the page too long or too sparse.** All contents should be in **one page**. This is very important.

2. **Code Requirements**:

(1) You need to hardcode the provided data into the HTML script to generate the document. Be careful with the syntax and formatting of the HTML.

(2) Put everything in one HTML file. Do not use external CSS or JavaScript files.

3. **Output Requirements**: Put "`<html`" at the beginning and "`</html>`" at the end of the script to separate the code from the text.

Please don't answer with any additional text in the script, your whole response should be the HTML code which can be directly executed.

Instruction Generation: You are an expert in data analysis and good at asking questions about documents. My persona is: "persona" I want you to generate some question-answer pairs of a **FIGURE_TYPE** about **TOPIC**, which I would ask. Instead of showing the document, I provide the data and the code that generates the document.

<data> **DATA** </data> <code> **CODE** </code>

Please come up with a list of *reasonable questions* that people will ask when they see the rendered document. Here are the requirements:

1. **Question Types**: All questions should be short-answer questions that are answerable based on the visual information in the document. All questions can be answered with a single word, phrase, or number. (as short as possible)

(1) **Information Retrieval questions** ask for specific information in the document, such as numbers, names, dates, titles, etc. The questions should cover different aspects (areas) of the document. This is the most common type of question.

(2) **Reasoning questions** require reasoning over multiple information in the document. These questions should be more challenging and require a deeper understanding of the document.

(3) **Document Type-specific questions** are questions that are specific and unique to this document type **FIGURE_TYPE**. These questions should be tailored to the content and structure of the document.

2. **Response Format**: Use | to separate the question, explanation, and concise answer for each example.

(1) Follow this format: question | explanation | concise answer, e.g., what is the total revenue? | The total revenue is the sum of all revenue sources in the document, which is \$2000 + \$3000 + \$5000 = \$10000. | \$10000

(2) Separate the question-answer pairs by double newlines. question1 | explanation1 | answer1
question2 | explanation2 | answer2...

(3) Do not provide too many questions, 5-10 questions are enough. Focus on the diversity and quality of the questions. Try to cover different aspects of the document.

(4) The concise answer should be as short as possible and directly answer the question. The answer should be faithful and exactly the same as what you would expect to see in the document, don't rephrase it. All words in the answer should be processed in natural language, no coding terms/characters.

Please follow the format strictly and do not include any additional text at the beginning or end of your response.

Using Personas to Enhance Diversity

LLMs often struggle to generate diverse synthetic data using sampling parameters alone [210], with biases leading to repetitive outputs across different runs. Recent work [79] shows that incorporating personas in prompts can improve diversity by enabling models to generate from varied perspectives. CoSyn adopts personas to enhance diversity during the Topic Generation stage. Each persona is a short sentence describing a personality or identity. For example, as shown in the middle of Figure 5.7, we sample a persona “a sci-fi novelist who likes alien

worlds”, which results in a topic of “*a novel about Extraterrestrial Flora & Fauna*” for generating the book cover image. We use the 200K personas released by Ge et al. [79].

Implementation details

CoSyn is built on the DataDreamer library [211], which supports robust multi-stage synthetic data generation pipelines that are easy to maintain, reproduce, and extend. DataDreamer documents the prompts and parameters used at each generation stage and implements several efficient techniques, such as parallel generation and response caching, to optimize performance. For the data and code generation stages, we use Claude-3.5-Sonnet, which performs well in coding tasks [212]. For instruction-tuning data generation, we select GPT-4o-mini [60] for its cost efficiency.

CoSyn-400K

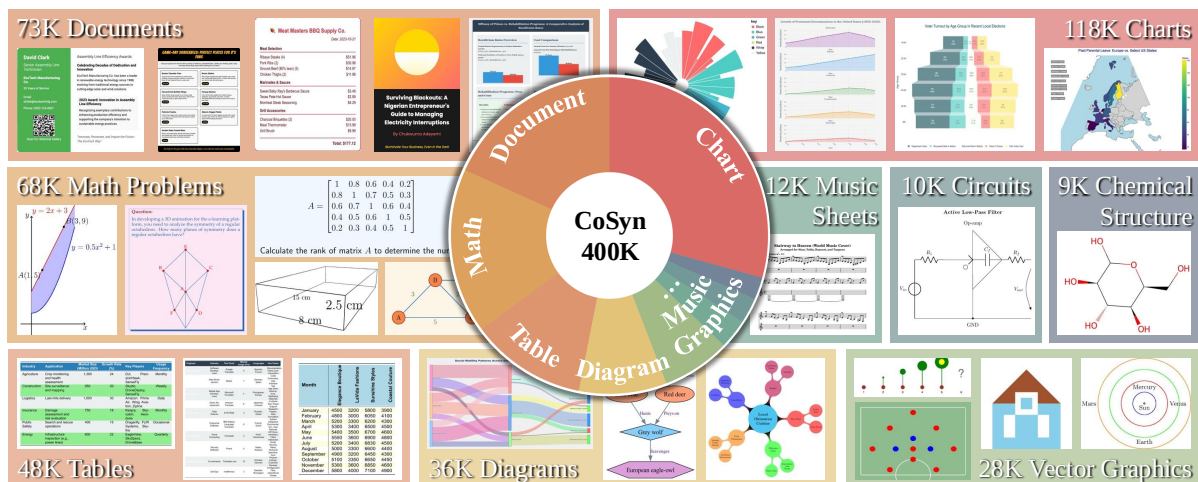


Figure 5.8: Our CoSyn-400K dataset consists of 9 categories of text-rich images with 2.7M instruction-tuning data.

As shown in Figure 5.8, we use CoSyn to generate a large-scale synthetic dataset of 400K images across nine categories: charts, documents, math problems, tables, diagrams, vector

graphics, music sheets, electrical circuits, and chemical structures. Since CoSyn is controlled via language inputs, it can easily generate diverse, fine-grained image types by varying the input queries. For instance, we use over 100 queries to generate document data covering *receipts*, *resumes*, *meal plans*, etc. This ensures that our dataset covers a broad range of domains. Below are some examples of the queries used to generate CoSyn-400K:

- **Charts:** Bar, Line, Pie, Diverge bar, Bubble, Scatter, Histogram, Area, Box plot, Heatmap, Error bar, Radar chart, Rose chart, Stem plot, Stairs plot, Violin chart, 2D contour, Distplots, Log plot, Ternary plots/contour, Candlestick charts, Time series, etc. (**51** queries in total)
- **Documents:** Letter, Form, Report, Receipt, Invoice, Restaurant menu, Newsletter, Schedule, Manual, Brochure, Transaction document, Agenda, Memo, Financial report, Telephone records, Note, Budget, Meeting minutes, Bill, Catalog, Email, Fax, Policy document, Resume, Infographics, Process infographic, Statistical infographic, etc. (**107** queries in total)
- **Math Problems:** Algebra, Counting, Probability, Geometry, Number theory, Precalculus, Prealgebra, Intermediate Algebra, Statistics, Functions, Complex numbers, Logarithms, Inequalities, Linear equations, Exponents, Series, College Algebra, Calculus, Advanced calculus, Linear algebra, Solid geometry, Analytic geometry, Polynomial arithmetic, etc. (**110** queries in total)
- **Tables:** Financial table, Simple table, Pivot table, Comparison table, Timeline table, Decision table, Truth table, Lookup table, Periodic table, Statistical table, Timetable, Hierarchical table, Matrix table, Contingency table, Logarithmic table, Correlation table, etc. (**35** queries in total)
- **Diagrams:** Flow chart, Directed graph, Undirected graph, Decision tree, Mind map, Gantt charts, Finite state machine, Quadrant chart, Chord diagrams, Network diagrams, Sankey diagram, Entity relationship diagram, Sequence diagrams, Bottom-up flow chart, Timeline, State diagram, Concept map, Family tree, Programming flowchart, etc. (**34** queries in total)

- **Vector Graphics:** Visual intelligence test, Spatial intelligence test, Geometry, Solid geometry, Analytic geometry, Polynomial graphs, Trigonometry, Polar coordinates, Coordinate system, Topology, Graph theory, Plane geometry, Functions, Calculus, Vectors, Angles, Perimeter and area problems, etc. (**36** queries in total)
- **Sheet Music:** Classical, Pop, Rock, Jazz, Blues, Hip Hop, Rap, Electronic, Country, Folk, Rhythm and blues, Soul, Reggae, Metal, Punk, Theme, Dance, etc. (**34** queries in total)
- **Electrical Circuits:** Series, Parallel, Hybrid, Household appliances, Industrial appliances, Mobile device, Low-power appliances, High-power appliances, etc. (**30** queries in total)
- **Chemical Structures:** Drug, Organic, Inorganic, Protein, Acids, Bases, Gases, Liquids, Solids, Oxidizers, Flammable liquids, Toxic chemicals, Hazardous chemicals, Aromatic compounds, Aliphatic compounds, Polymers, Metals, Alloys, Electrolytes, etc. (**100** queries in total)

5.3.2 Experimental Setup

Our experiments aim to verify the value of our synthetic data in the supervised fine-tuning stage of training vision-language models. This section introduces the architecture of our model, training strategy, datasets we used, baselines for comparison, and other details on implementation.

Model Architecture

We follow the same image preprocessing and architecture as Molmo [177], which uses the MLP layer to connect the vision encoder and a pretrained LLM. We choose OpenAI’s CLIP (ViT-L/14 336px) [205] as the vision backbone and Mistral-7B [213] as the language model.

Training Process

We adopt the same training strategy as Molmo [177], which consists of two stages: (1) *Pre-training* on dense captions from PixMo-Cap and (2) *Supervised fine-tuning* on three categories of datasets below:

- **Evaluation Datasets** We evaluate our model on seven text-rich benchmarks, including ChartQA [214], DocVQA [215], InfographicVQA [216], TableVQA-Bench [217], AI2 Diagrams [218], TextVQA [219], and ScreenQA [220]. We adopt their official metrics for calculating performance. In total, we have 138K training images from the evaluation datasets.¹⁰
- **Auxiliary Datasets** We select additional academic datasets for fine-tuning: VQAv2 [221], GQA [222], OK-VQA [223], OCR-VQA [224], A-OKVQA [225], ScienceQA [226], TabMWP [227], ST-VQA [228], TallyQA [229], DVQA [105], FigureQA [104], and PlotQA [106]. The auxiliary datasets contain around 1M training images.
- **Synthetic Datasets** As shown in Figure 5.8, our synthetic datasets include 400K text-rich images from 9 categories.

Our best-performing model uses all three categories of datasets above. We also trained a zero-shot model using only auxiliary and synthetic data without any examples from the evaluation datasets, which still exhibits competitive benchmark performance, as shown in the last row of Table 5.11.

Baselines

We compare recent open-source VLMS with a similar scale (7B), including PaliGemma-3B [230], BLIP-3-4B [231], Cambrian-7B [232], LLaVA-1.5-7B [206], LLaVA-Next-8B [233], LLaVA OneVision-7B [234], Pixtral-12B [235], Llama 3.2 V [236]. We also include proprietary models: GPT-4V [60], Gemini-1.5-Flash [237], and Claude-3 Opus [238].

¹⁰TableVQA is an eval-only benchmark (no training split), and we do not use the training split from ScreenQA.

Implementation Details

We train our model on TPU v3-128 with a batch size of 32. Our best-performing model is trained for 60K steps, taking about 30 hours. The checkpoints with the highest validation performance are retained for testing.

5.3.3 Results

Model	ChartQA	DocVQA	InfoVQA	TableVQA	AI2D	TextVQA	ScreenQA	Average
GPT-4V	78.1	87.2	75.1	60.5	89.4	78.0	41.6	72.8
Gemini 1.5 Flash	85.4	89.9	75.3	72.6	91.7	78.7	40.1	76.2
Claude-3 Opus	80.8	89.3	55.6	70.0	88.1	67.5	39.8	70.2
PaliGemma-3B [†]	71.4	84.8	47.8	46.4	73.3	76.5	32.2	61.8
BLIP-3-4B [†]	60.0	61.4	31.5	24.3	74.2	71.0	26.2	49.8
Cambrian-7B [†]	73.3	77.8	41.6	40.6	73.0	71.7	44.4	64.2
LLaVA-1.5-7B ^{†*}	17.8	28.1	25.8	33.1	55.5	58.2	17.6	33.7
LLaVA-Next-8B [†]	69.5	78.2	43.8	43.9	71.6	65.3	34.2	58.1
LLaVA-OneVision-7B [†]	80.0	87.5	<u>68.8</u>	64.6	<u>81.4</u>	<u>78.3</u>	46.3	72.4
Pixtral-12B	81.8	90.7	50.8	67.0	79.0	75.7	39.4	69.2
Llama 3.2 11B	<u>83.4</u>	88.4	63.6	51.1	91.9	73.1	87.7	<u>77.0</u>
Ours (7B) [†]	86.3	<u>90.0</u>	70.5	<u>65.8</u>	91.9	82.0	<u>80.1</u>	80.9
Ours (7B-zero-shot) ^{†*}	80.8	82.9	59.8	64.9	83.9	72.7	78.1	74.7

Table 5.11: Results on 7 text-rich benchmarks. The result of the best-performing open-source model is **bold**, and the second-best is underlined. Models with [†] stand for open data and code for multimodal training. Models with * are zero-shot models, which means the models are not trained on instances from any of the evaluation datasets.

Table 5.11 compares our model’s performance with both open and closed models across seven text-rich benchmarks. On average, our 7B model achieves the highest performance, surpassing the second-best model (Llama 3.2 11B) by 3.9%. Notably, our model ranks first in four out of the seven datasets and second in the remaining three. More surprisingly, our zero-shot model (the last row in Table 5.11) outperforms most open and closed models without exposure to any training instances from the evaluation datasets. In contrast, these competing models often rely on benchmark training data and are thus not true zero-shot models. This

result demonstrates that the capabilities learned from our synthetic data can transfer effectively to downstream tasks.

5.3.4 Discussion

In the following experiments, we quantify the contribution of synthetic data to the benchmark performance by ablating the combinations of fine-tuning datasets. Then, we demonstrate that our CoSyn system can efficiently assist VLMs in generalizing to novel tasks. Finally, we show that synthetic data can help mitigate the overfitting of biases.

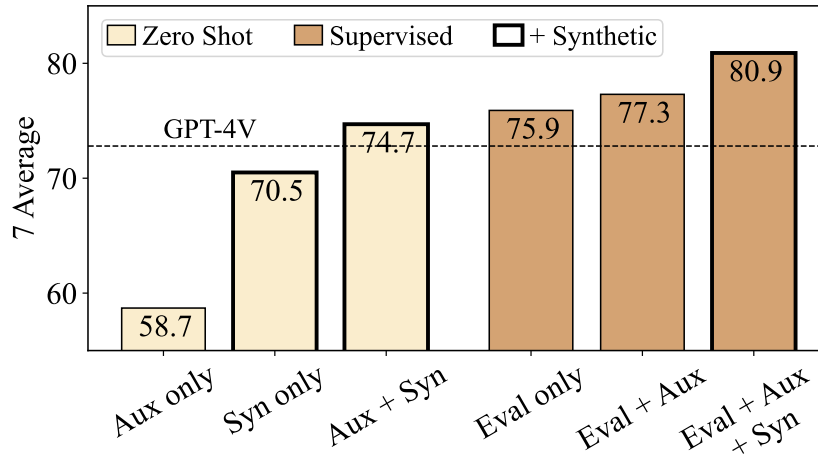


Figure 5.9: An ablation on training data selection. Aux, Syn, and Eval stand for auxiliary, synthetic, and evaluation datasets, respectively. We report the average score on seven benchmarks.

Synthetic Data Enhances the Performance

Figure 5.9 presents an ablation study on the choices of supervised fine-tuning data. In the zero-shot settings, when the model is trained on auxiliary datasets (over 1M training images not directly from the evaluation tasks), it fails to generalize effectively to the evaluation tasks, with

a substantial performance gap of 14.1% below GPT-4V. However, using only 400K synthetic samples achieves a performance comparable to GPT-4V. Our best zero-shot model surpasses GPT-4V when jointly training synthetic and auxiliary data. Under the supervised settings, training with in-domain data alone yields strong performance. However, adding 1M auxiliary samples provides a modest improvement of 1.4%, while incorporating synthetic data results in a more significant 3.6% boost. These results demonstrate the effectiveness of synthetic data in enhancing VLMs' performance on text-rich tasks.

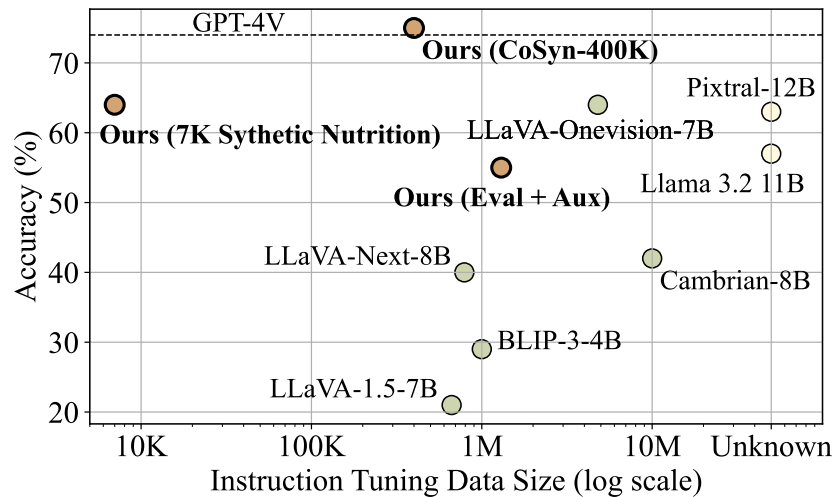


Figure 5.10: Zero-shot performance on NutritionQA. The x-axis denotes the number of training examples used for the instruction-tuning stage. The models on the upper left side demonstrate better data efficiency.

Zero-shot Generalization on a Novel Task

Vision-language models typically rely on in-domain data to perform well on specific tasks. When encountering a novel task, such as answering questions about nutrition labels in Figure 5.6, models without seeing similar examples during training may struggle with this novel task. However, our CoSyn system enables controllable data generation. Given the task name as input, CoSyn can generate task-specific data to fine-tune the model.

To validate this, we annotated a small evaluation dataset called **NutritionQA**, which includes 100 examples of questions about photos of nutrition labels. Some questions require multi-hop reasoning, as Figure 5.11 illustrates.



Figure 5.11: Examples from our newly collected **NutritionQA** dataset.

We evaluated GPT-4V and several open-source VLMS on this dataset and report the performance in Figure 5.10. The x-axis in Figure 5.10 represents the amount of data used during the instruction fine-tuning stage.

Despite being trained on millions of images, we observe that open-source VLMS are not data-efficient and perform poorly on this novel task compared to GPT-4V. Although many open-source VLMS claim to achieve GPT-4V-level performance, they fall short when tested on new tasks in the wild. Without synthetic data, our model (Eval + Aux) achieves results similar to those of open models. However, when trained on 400K synthetic samples, our model matches GPT-4V’s performance.

More impressively, we used CoSyn to generate 7K synthetic nutrition label samples and fine-tuned the model using only this 7K data. The resulting model outperforms most open-source VLMS on the NutritionQA task. These results demonstrate that code-guided synthetic data is an effective and efficient method for adapting VLMS to new domains.

Synthetic Data for Chain-of-Thought Reasoning

Existing text-rich datasets, such as ChartQA [214], are typically annotated with short answers. However, questions like “Compute the mean of the data in the plot” require step-by-step mathematical reasoning to arrive at the correct answer. Models trained only with short-answer supervision may fail to learn proper plot comprehension, but instead overfit to annotation biases in these datasets. On the contrary, our CoSyn-400K includes explanation text alongside the short answer. Each instruction-tuning example consists of a (*question*, *explanation*, *short answer*) triplet, enabling models to learn chain-of-thought (CoT) reasoning. During fine-tuning, we design two prompt templates for our synthetic data:

CoT Prompt: <Question> Provide reasoning steps and then give the short answer.

<Explanation> Answer: <Answer>

Short Answer Prompt: <Question> Answer with as few words as possible. <Answer>

Those prompts allow VLMS to switch between the two answering styles and perform CoT reasoning when necessary. Figure 5.12 shows that incorporating CoT reasoning improves performance on ChartQA, TableVQA, and NutritionQA, as these datasets contain examples requiring multi-hop reasoning. However, we observe that adding CoT reasoning reduces performance on DocVQA and InfoVQA. We find that answer biases in these benchmarks cause this decline. Specifically, the ground-truth answers favor short responses, often penalizing more detailed and verbal responses. For instance, in DocVQA, the ground-truth for an example is “T-Th”, whereas the model responds with “Tuesday to Thursday”. Although the response is correct, the strict string-matching metric assigns it a zero score. This highlights key limitations of current multimodal benchmarks, including answering biases and rigid evaluation metrics that fail to capture the full extent of a model’s capabilities.

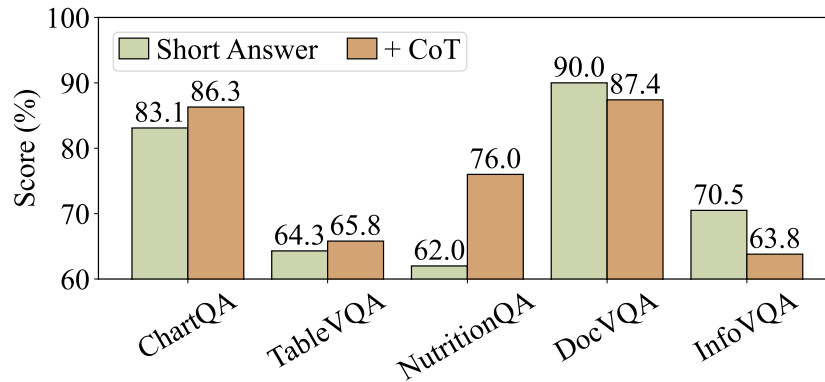


Figure 5.12: An ablation of using Chain-of-Thought reasoning. Short Answer represents prompting model to output the answer as short as possible. + CoT stands for providing Chain-of-Thought reasoning before giving the final answer.

Synthetic Data for Mitigating Biases

Our previous experiments reveal answering biases in multimodal benchmarks, which VLMS trained solely on these datasets often inherit. To further validate this issue, we analyze ChartQA and observe a distribution shift in question types. As shown in the pie charts above Table 5.12, some ChartQA questions are human-annotated, while others are generated by the language model T5 [239], which is heavily influenced by prompt phrasing and limited to a fixed set of question templates. During training, most questions (73.9%) in ChartQA are machine-generated, while the test set contains an even distribution of human-annotated and machine-generated questions. Models trained exclusively on ChartQA tend to overfit to T5-generated questions. Table 5.12 illustrates this issue: PaliGemma [230] and ChartPali [240] achieve high accuracy on machine-generated questions but experience a significant performance drop of over 30% on human-annotated questions.

Similarly, without synthetic data, our model shows a noticeable 21.8% gap between the two question types. However, incorporating synthetic data during training reduces this gap to

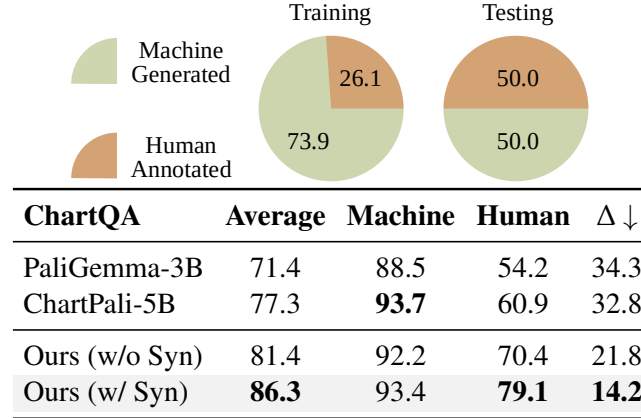


Table 5.12: Results on human and machine-generated questions of ChartQA. The pie charts above display the percentage distribution of two question types in training and testing. Δ ($\downarrow$ lower is better) denotes the performance gap between human and machine questions.

14.2%. This suggests that synthetic data can mitigate overfitting on benchmarks and enhance VLMs’ usability in real-world human-asked questions.

Quantifying the Diversity of our Synthetic Data

To quantify the diversity of images and text in our synthetic dataset $\mathcal{D} = \{(I, T)\}$, we propose the following two metrics to compute the diversity:

$$\mathbf{Diversity}(\mathcal{D})_{\text{Image}} = \frac{1}{|\mathcal{D}|^2 - |\mathcal{D}|} \sum_{I_i \in \mathcal{D}} \sum_{I_j \in \mathcal{D}}^{i \neq j} (1 - \text{sim}(I_i, I_j))$$

$$\mathbf{Diversity}(\mathcal{D})_{\text{Text}} = \frac{1}{|\mathcal{D}|^2 - |\mathcal{D}|} \sum_{T_i \in \mathcal{D}} \sum_{T_j \in \mathcal{D}}^{i \neq j} (1 - \text{sim}(T_i, T_j))$$

where $\text{sim}(\cdot)$ is the cosine similarity function. Both metrics compute the average pairwise cosine distance between the features of every instance in the dataset. For image diversity, we extract features using CLIP, while for text diversity, we use Sentence-BERT [241] to obtain embeddings of question-answer pairs. Table 5.13 shows that our synthetic charts are significantly

more diverse than those in existing datasets, such as FigureQA and ChartQA, in both image and text diversity.

Dataset	Image Diversity	Text Diversity
FigureQA	0.268	0.567
DVQA	0.307	0.752
PlotQA	0.420	0.743
ChartQA	0.340	0.742
Ours (Charts)	0.596	0.823

Table 5.13: Compare image and text diversity across different chart datasets. We randomly sample 10K instances from each dataset to compute the results.

Diversity Correlates with Model Performance

We observe that data diversity significantly affects model performance on downstream tasks. To investigate this, we compare synthetic chart data generated using only a single tool (Matplotlib) with charts generated by all five tools available in our CoSyn system. As shown in Table 5.14, using multiple tools results in higher image diversity and notably improved performance on ChartQA. This experiment underscores the importance of data diversity for enhancing the generalizability of models.

n. of Tools	Diversity	ChartQA		
		Average	Machine	Human
SINGLE	0.572	73.9	66.5	81.5
MULTIPLE	0.607	75.2	68.6	82.0

Table 5.14: Single vs. Multiple Rendering Tools for Data Generation. Each row uses the same number of 45K synthetic images. SINGLE only uses Matplotlib, while MULTIPLE involves four other rendering tools: HTML, LaTeX, Plotly, and VegaLite.

Scaling the Size of Synthetic Data

In addition to diversity, the scale of synthetic data also impacts model performance. As shown in Figure 5.13, increasing the number of synthetic chart images leads to improved performance on ChartQA. This demonstrates that scaling up synthetic data can further enhance VLMs on downstream tasks. Due to resource constraints, our final dataset consists of 400K images, which cost us about \$8,000. Future work could explore scaling up the dataset size to push the boundaries of synthetic data’s potential.

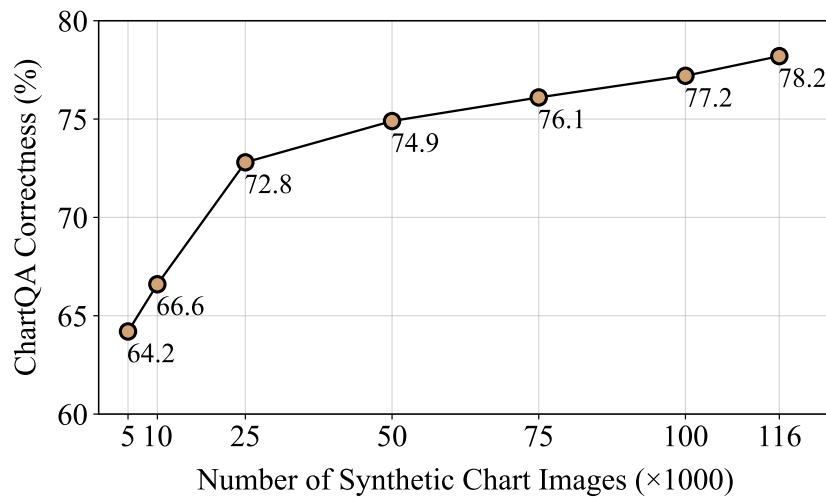


Figure 5.13: Performance while scaling the size of synthetic data. We evaluate the *zero-shot* performance on ChartQA of models fine-tuned on increasing numbers of synthetic images.

Comparing LLMs for Synthetic Data Generation

In the default setting, CoSyn uses Claude-3.5-Sonnet as the underlying LLM for code generation. To highlight the importance of strong coding capabilities, we compare it with data generated by GPT-4o. As shown in Table 5.15, synthetic data generated by Claude-3.5-Sonnet yields significantly better results than GPT-4o. Our qualitative observation reveals that GPT-4o has a higher failure rate in code generation, particularly for less common coding languages or

libraries. This result emphasizes that a strong LLM is essential for the successful synthetic data generation for VLMs.

LLM for Data Generation	ChartQA		
	Average	Machine	Human
GPT-4o	72.4	65.8	78.9
Claude-3.5-Sonnet	77.2	71.0	83.8

Table 5.15: Compare the LLMs used for synthetic data generation. For both LLMs, we create 100K synthetic charts for fine-tuning the VLMs. We report the zero-shot evaluation results on ChartQA.

5.3.5 Applications to AI Agents

Pointing enables VLMs to answer questions by providing specific points on images. This allows models to ground their responses in visual content and interact with environments, which is crucial for developing digital agents. We find that our code-guided system can synthesize pointing data.

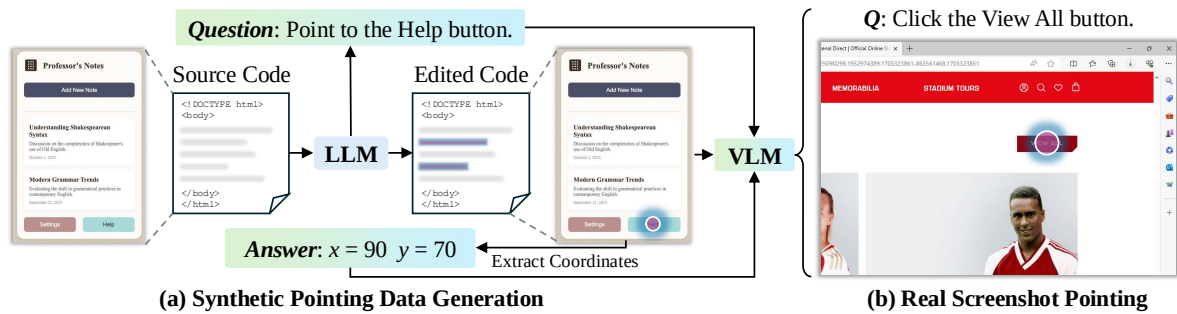


Figure 5.14: An overview of enabling VLMs to point through synthetic data. (a) We synthesize pointing data by prompting an LLM to generate pointing questions and edit the code to draw the answer points explicitly. (b) We demonstrate that the VLM trained on synthetic pointing data can be generalized to real agentic tasks.

Method

Since we have access to the source code for all generated images, we can prompt an LLM to modify the code to draw points on the images explicitly. As illustrated in Figure 5.14, we feed the image’s source code as context to the LLM, which generates a pointing question and edits the code to draw points with a predefined color. By extracting the pixel values of these points, we can obtain their exact (x,y) coordinates.¹¹ We then use this pointing data to train VLMS, enabling them to answer questions by providing point coordinates. In total, we generate pointing data for 65K synthetic images. Figure 5.15 shows some qualitative examples from our synthetic pointing dataset.

Setup

We evaluate pointing ability on ScreenSpot [242], where the task requires models to provide the correct click location based on a given instruction. ScreenSpot contains screenshots from mobile phones, desktops, and web pages. To assess the effectiveness of our synthetic pointing data, we compare it to the model trained on PixMo-point [177], which consists of 155K human-annotated images. Our best-performing model uses both PixMo-point and synthetic pointing data. Additionally, we compare against existing methods like CogAgent [243], SeeClick [242], and UGround [244], which is trained on 1.3M screenshots.

Results

Table 5.16 compares the click accuracy of our models with previous methods. Using 65K synthetic pointing samples, our model achieves performance comparable to that trained on 155K human-annotated samples. When combining synthetic and human data, our model achieves state-of-the-art performance on ScreenSpot, surpassing the recent UGround [244], which was

¹¹The coordinates of points are normalized to (0, 100) to mitigate the influence of image resolution.

Online Portfolio Hosting Fee

Invoice Details: Invoice Number: INV-1024, Date: 2023-10-01, Due Date: 2023-10-15

Hosting Service: Provider: CloudHost, Plan: Business Pro, Monthly Fee: \$18, Annual Fee: \$216. Features: Unlimited bandwidth, Custom domain, 24/7 customer support, Enhanced analytics.

Website Performance Analytics: Monthly Visits: 2,500, Avg. Session Duration: 3 minutes, Bounce Rate: 40%, Page Load Time: 1.5s, SEO Score: 85, Mobile Friendly: Yes.

Total Due: \$18 USD

Conservative Issues Poll Results
Texas State University 2023

Issue	Support	Opposition	Neutral	Sample Size
Support for Second Amendment Rights	78%	12%	10%	200
Alternative Action Policies	34%	57%	10%	200
Tax Cuts	65%	15%	10%	200
Border Security	85%	10%	5%	200
Abortion Rights	18%	45%	30%	200
Climate Change Policies	33%	41%	20%	200
School Choice	74%	11%	15%	200
Government Surveillance	13%	10%	10%	200
Police Funding	80%	13%	8%	200

Data collected from Texas State University students in 2023. Sample size: 200 for each issue.

Golden Age of Textiles Notifications

Manage notifications for events, exhibitions, and research in Bradford's textile history.

Events & Exhibitions

Receive Event Notifications
Stay updated about upcoming textile history events in Bradford. ☒

VIP Invitations
Exclusive invites to private viewings and lectures. ☐

Research Updates

New Research Findings
Notifications for the latest research in Bradford's textile industry. ☐

Published Papers
Read the latest publications. [View](#)

Community & Networking

Join Textile Historians Forum
Connect with experts and enthusiasts. [Join](#)

Vintage Pattern Gallery
Explore classic textile designs. [View](#)

Q: Point out the page load time performance of the website, which indicates the site's speed optimization

A: (64.1, 73.7)

Q: Generate points for all issues where opposition exceeds 60%.

A: (56.2, 61.1) (56.2, 81.4)

Q: Provide a point where users can toggle event notifications.

A: (35.7, 76.9)

9:41 5G 100%

FC Rosengård Koppargårds/Göteborg FC

[View more highlights](#)

Eco-Friendly Printing: Sustainable Practices for the Classroom

Jane Doe, Classroom Print Services Advisor

Reduce Paper Waste
Adopt strategies to reduce paper consumption. Encourage students to use both sides of paper and opt for digital resources.

Use Recycled Materials
Opt for recycled paper and eco-friendly ink for all classroom printing needs. Donate the excess paper to local schools.

Eco-Friendly Printing Tips
Adopting eco-friendly practices in the classroom helps reduce environmental impact. Use digital resources and double-sided printing to conserve paper.

Nature-Inspired Aesthetics
Incorporate nature-themed designs and graphics. Use recycled and eco-friendly materials to promote a green and sustainable classroom environment.

DigitalMarketingPro Software
Monthly Subscription \$8

Customer Details
Name: Jane Doe, Position: Digital Marketer, Company: Creative Brands Inc., Address: 123 Marketing Lane, Johannesburg, SA, Contact: jane.doe@example.com

Billing Information
Bill Date: October 2023, Due Date: November 5, 2023, Payment Method: Credit Card

Subscriptions

Service	Description	Cost
Social Media Management	Scheduling and managing posts across multiple platforms	\$25.00
SEO Optimization Tools	Keyword research, on-page analysis, and performance tracking	\$10.00
Email Marketing Suite	Campaign creation, list management, and analytics	\$15.00
Analytics Dashboard	Real-time performance metrics and insights	\$15.00
Content Creation Tools	Templates for blogs, ads, and social posts	\$15.00
Total Amount		\$100.00

Please ensure payment of \$100.00 is made by November 5, 2023. Thank you for choosing DigitalMarketingPro Software for your digital marketing needs!

Q: Point out the main title of the book.

A: (50.0, 10.9)

Q: Identify which logo represents FC Rosengård in this match.

A: (22.1, 56.2)

Q: Highlight all services that cost more than \$30.

A: (88.7, 54.6) (88.6, 61.1)

Dinner Planner

[Filter Meals](#) [Add to Shopping List](#)

Monday
Spaghetti Bolognese
20 mins
Spaghetti, Ground Beef, Tomato Sauce
[View Recipe](#)

Tuesday
Grilled Chicken Salad
15 mins
Chicken Breast, Lettuce, Caesar Dressing
[View Recipe](#)

Today's Mix

1. Michael Jackson - Billie Jean
2. Prince & The New Power Generation - The Love Machine
3. Prince & The New Power Generation - The Love Machine
4. Prince & The New Power Generation - The Love Machine
5. Prince & The New Power Generation - The Love Machine

[Play](#)

Q: Find Michael Jackson songs that are longer than 4 minutes.

A: (9.8, 12.7) (9.8, 26.4)

Q: Show me the pasta icon for the Italian dish.

A: (15.0, 26.3)

Fantasy Finance Careers Portal

Home Jobs Applications Profile Logout

Filter by Category
Data Analyst, Financial Analyst, Risk Management, Portfolio Management

Senior Financial Data Analyst
Goldman Sachs - New York, NY
[View Details](#)

Risk Management Specialist
JP Morgan - Chicago, IL
[View Details](#)

Your Application Quest
You're on your way to unlocking the 'Career Knight' badge!
Applied for 3 jobs, Complete profile, Receive first interview

Q: Point the button to submit application for the Senior Financial Data Analyst.

A: (32.8, 25.5)

Figure 5.15: Randomly selected examples from our synthetic pointing data.

trained on 1.3M screenshots. These results demonstrate that synthetic pointing data is a data-efficient approach for improving VLM performance on agentic tasks involving click prediction.

Model	Mobile		Desktop		Web		Avg
	Text	Icon	Text	Icon	Text	Icon	
GPT-4o	20.2	24.9	21.1	23.6	12.2	7.8	18.3
CogAgent	67.0	24.0	74.2	20.0	70.4	28.6	47.4
SeeClick	78.0	52.0	72.2	30.0	55.7	32.5	53.4
UGround	82.8	<u>60.3</u>	82.5	<u>63.6</u>	<u>80.4</u>	70.4	<u>73.3</u>
Synthetic	90.8	53.3	78.4	58.6	80.0	47.1	68.0
Human	84.2	59.0	88.1	52.9	76.5	50.5	68.5
Combined	<u>89.0</u>	65.1	<u>87.6</u>	65.7	83.0	<u>58.7</u>	74.9

Table 5.16: Click accuracy on ScreenSpot. We report our models trained on different pointing data. Human stands for using the human-annotated data from PixMo-point [177]. Combined means combining human-annotated data with our synthetic pointing data.

5.3.6 Summary of Contributions

This work is based on our paper “Scaling Text-Rich Image Understanding via Code-Guided Synthetic Multimodal Data Generation” published at the Association for Computational Linguistics 2025 [245]. I helped construct the main synthetic data generation pipeline for CoSyn in my package DataDreamer and contributed significantly to writing the paper as a co-first author. The research direction was largely led by my co-author Yue Yang. Yue has reproduced parts of this paper in his own thesis.

5.4 CONCLUSION

In summary, in this chapter, I have covered three different use cases of fine-tuning on synthetic data that enable new capabilities in models.

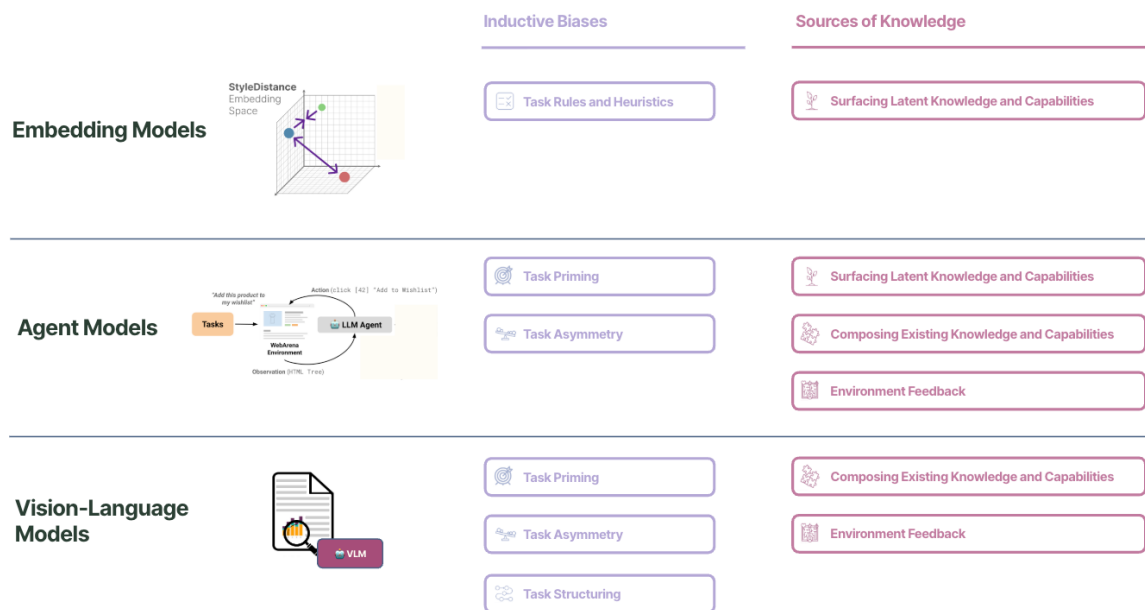


Figure 5.16: Inductive biases and sources of knowledge highlighted by the three examples discussed in this chapter that fine-tune on synthetic data to enhance a model’s performance.

While each synthetic data approach is unique and specially crafted for the given task, what they have in common is, the learning biases imposed can still largely be categorized into the same four broad inductive biases I introduced earlier.

In our work on style embeddings models, we generated a synthetic dataset targeting 40 style features. This essentially encodes a rule or heuristic of at least 40 style features we choose to have style embedding model to represent, although there are an infinite number of style features possible. By training on the fixed-set of 40 style features, the STYLEDISTANCE model was able to generalize reasonably well to unseen style features, which it may already have latent

knowledge about from its pre-training that we are able to surface through the synthetic training procedure.

In our work on agent models, we use the LLM itself to filter trajectories down to the plausibly successful ones, taking advantage of the fact that it is easier for the LLM to verify trajectories as correct or not than successfully perform a task in an environment. There is also additional knowledge being gained by the LLM in this process by getting feedback from the environment on the actions it chose to take through error messages or the pages that result in response the agent's actions. We also use the LLM to generate novel, solution trajectories for entirely different websites the LLM may have knowledge about essentially surfacing latent world knowledge the LLM may have and taking advantage of the fact it is easier to construct a hypothetical environment and solutions for tasks in it than successfully perform agent tasks in an unfamiliar environment.

In our work on vision language models, this procedure takes advantage of the prior knowledge that it is easier to generate data and code to render a figure and generate questions from that code than it is to answer questions given an image of a figure. And it composes multiple capabilities of a model to form the synthetic dataset. It also uses external tools in the environment (code interpreter, rendering libraries) to gain new knowledge about the task. The synthetic dataset also benefits from prior knowledge that structuring the task in this way, by using data and code as an intermediate representation, is useful towards better understanding the relationship between the raw data and data visualization images.

In the final chapter, I will discuss one more work that uses synthetic data to apply a strong inductive bias at the pre-training stage of LLMs to broadly enhance general performance with synthetic data, rather than task-specific performance.

6 | ADVANCING LARGE LANGUAGE MODEL PRE-TRAINING WITH SYNTHETIC DATA

Self-supervised pre-training of large language models on vast amounts of unlabeled text data has reshaped natural language processing, enabling capabilities that span complex reasoning, contextual understanding, and multi-task generalization [2]. The quality and quantity of data that LLMs pre-train on is critical to their performance. Traditionally, this data comprises of massive corpora of pre-training documents sourced from diverse domains (web pages, books, articles, etc.), which serve as a foundation for the model’s general knowledge [3, 4, 5]. Leveraging synthetic data as a technique in LLM pre-training has gained attention as a potential strategy to improve the efficiency of the pre-training process, potentially leading to more capable models while mitigating the high costs associated with traditional pre-training methods [22, 119]. Demonstrating significant improvements to pre-training with synthetic data would signal these models have reached an inflection point, where they can help optimize their own knowledge acquisition.

The research in this chapter expands on my prior work on synthetic data generation and its impact on downstream task-specific performance by moving toward a broader goal: integrating synthetic data into the pre-training phase of LLMs. This approach aims to explore whether targeted, synthetic data can provide beneficial inductive biases during pre-training, facilitating models that are not just proficient in specific applications but exhibit stronger, generalized capabilities across diverse tasks. This chapter focuses on a method that transforms pre-training documents into synthetic instructions and answer pairs, allowing models to be pre-trained on

data that directly matches the downstream intended usage instead of being trained with a proxy objective like next token prediction over pre-training documents.

6.1 MOTIVATION

Today, self-supervised pre-training, the stage where models are trained using a language modeling task like next token prediction over a large amount of text data is where models acquire the vast majority of their knowledge and where the majority of compute, money, and time is spent in building these models [3, 5, 57]. A pre-trained model can then be adapted to have strong instruction-following capabilities by being further trained on a relatively small amount of instruction-answer examples in a supervised manner in a process known as instruction-tuning [6, 8, 7, 142]. Instruction-tuning datasets are either small consisting of a few thousand examples [246, 247], narrow and unrealistic consisting of academic NLP tasks converted into instruction-tuning formats with a relatively small number of task templates [7, 8, 142], or too limited being constructed by using a proprietary frontier model to generate instruction and answer examples which ultimately only helps mimic those models superficially through distillation [131, 135, 130, 55]. The instruction-tuning post-training stage of LLMs is highly limited due to these data resource limitations and it mainly contributes to helping the model learn to follow instructions and learn response styles while the self-supervised pre-training stage is responsible for encoding the vast majority of knowledge in the model weights [248, 249, 250]. Instruction-tuning, however, makes a model vastly more useful for downstream use cases as it involves supervised learning on a dataset format in-distribution with downstream usage by users (a response conditioned on a query or instruction) instead of self-supervised learning on a language modeling objective.

It is hypothesized that models trained with self-supervised language modeling objectives like next token prediction indirectly learn foundational knowledge for performing tasks through examples of tasks that appear in the pre-training documents. Some recent work explicitly attempts to filter pre-training corpora for such high-quality or educational documents using LLM-generated annotations [118, 119, 120]. It is unclear, however, whether the next token prediction objective over pre-training documents is the most optimal or efficient way for models to absorb such knowledge with some prior work showing simple techniques like rephrasing these documents to be written in a more encyclopedic style with LLMs helps models learn more efficiently [128, 129] and further work demonstrates generating synthetic instruction-answer pairs from a document and appending them to the end of documents improves knowledge absorption during pre-training [126, 121]. While next token prediction over vanilla pre-training documents is effective at absorbing knowledge, these methods demonstrate other pre-training data formats can improve both efficiency of knowledge absorption during pre-training and the performance and capabilities of models trained on these alternative formats.

In this chapter, we introduce a procedure called FineInstructions to transform pre-training documents at scale into diverse, realistic, instructions or queries a user might ask about the knowledge in the document along with an answer or response grounded in the document. Converting pre-training documents into these synthetic instruction and answer pairs allows us to effectively perform supervised instruction-tuning at pre-training scale instead of performing self-supervised next token prediction over the vanilla pre-training documents. We hypothesize such a restructuring of the data may aid knowledge absorption and model performance as prior studies have shown multi-task datasets and instruction-answer formats induce generalization and instruction-following capabilities [3, 6, 250]. This kind of restructuring also prevents pre-training compute budget from being wasted on fitting to potentially low-quality noise in documents (footers, headers, less educational content, etc.) and better utilized towards higher-

quality, more educational sections of a document which prior work has found to be helpful [119]. Finally, it brings the pre-training data further in-distribution with the expected downstream usage by users.

The synthetic instructions and answer pairs we generate differ from prior approaches significantly. First, we attempt to generate highly diverse instructions by creating a large dataset of around ~18M of instruction templates to sample from while most existing instruction-tuning datasets only contain hundreds to thousands of task templates [7, 142, 143] and other approaches in converting pre-training documents to synthetic instruction-answer pairs merely prompt an LLM to produce questions about a document which produces questions with limited diversity [79]. We also generate instructions and queries closer to the kind real users may ask by creating our instruction templates from real user queries, while many prior works focus on generating instructions structured like academic NLP tasks such as multiple choice questions or factual question answering [7, 142, 143, 121] which typically have limited generalization to real-world queries [246, 251].

6.2 FINEINSTRUCTIONS

In this section, we detail the implementation of the FineInstructions pipeline illustrated in Figure 6.1. The FineInstructions pipeline involves sourcing natural, user-written queries (questions, instructions, and tasks). These queries are transformed into generic, reusable, instruction templates that could be instantiated for a different subject, topic, or domain. Given a set of pre-training documents, we match documents with compatible instruction templates where the document contains enough information to both instantiate the query realistically as well as provide a grounded answer. Finally, we can judge the quality of these instruction-answer

pairs and filter down to a high-quality subset. This allows us to create a large set of synthetic instruction-answer pairs that can be used to perform supervised pre-training over, conditioning answer generation on the instructions. This procedure closely resembles a weak supervision procedure where large, unlabeled corpora are transformed into large, supervised datasets with programmatic labeling [47]. In this case, we use a large bank of instruction templates and existing models trained on some amount of supervised data to aid in extracting a larger quantity of supervised data. We use DataDreameer [16], a framework for synthetic data generation and training to generate silver-standard data with task-specific prompts and train efficient distilled models that help perform each step of the pipeline at pre-training scale. An example of a task-specific prompt can be found in Figure 6.2. Full details on our pipeline, prompts, training, and hyperparameters can be found in our code. The LLM we use to generate silver-standard data is Llama-3.3 70B Instruct [252]. We discuss the implementation of each stage of this pipeline in greater detail next.

6.2.1 Generating Instruction Templates

We collect realistic instances of users querying LLMs, asking questions to others online in forums, or asking questions to a search engine. We also collect human-written prompt templates and prompt libraries. We source user-written queries from the following datasets: [WildChat](#) (657K queries) [253], [LMSys Chat](#) (559K queries) [254], [LMSys Chatbot Arena Conversations](#) (26.5K queries) [255], [OAsst1](#) (3.92K queries) [256], [HuggingFace NoRobots](#) (9.49K queries) [247], [HelpSteer](#) (10K queries) [257], [Dolly](#) (14.7K queries) [246], [Reddit QA](#) (7.47M queries), [GooAQ](#) (3.01M queries) [258], [ShareLM](#) (264 queries) [259], [ExpertQA](#) (1.73K queries) [260], [ChatDoctor iCliniq](#) (7.32K queries) [261], [ChatDoctor HealthcareMagic](#) (112K queries) [261], [Awesome ChatGPT Prompts](#) (203 queries), [Anthropic Prompt Library](#) (64 queries), [LangChain](#)

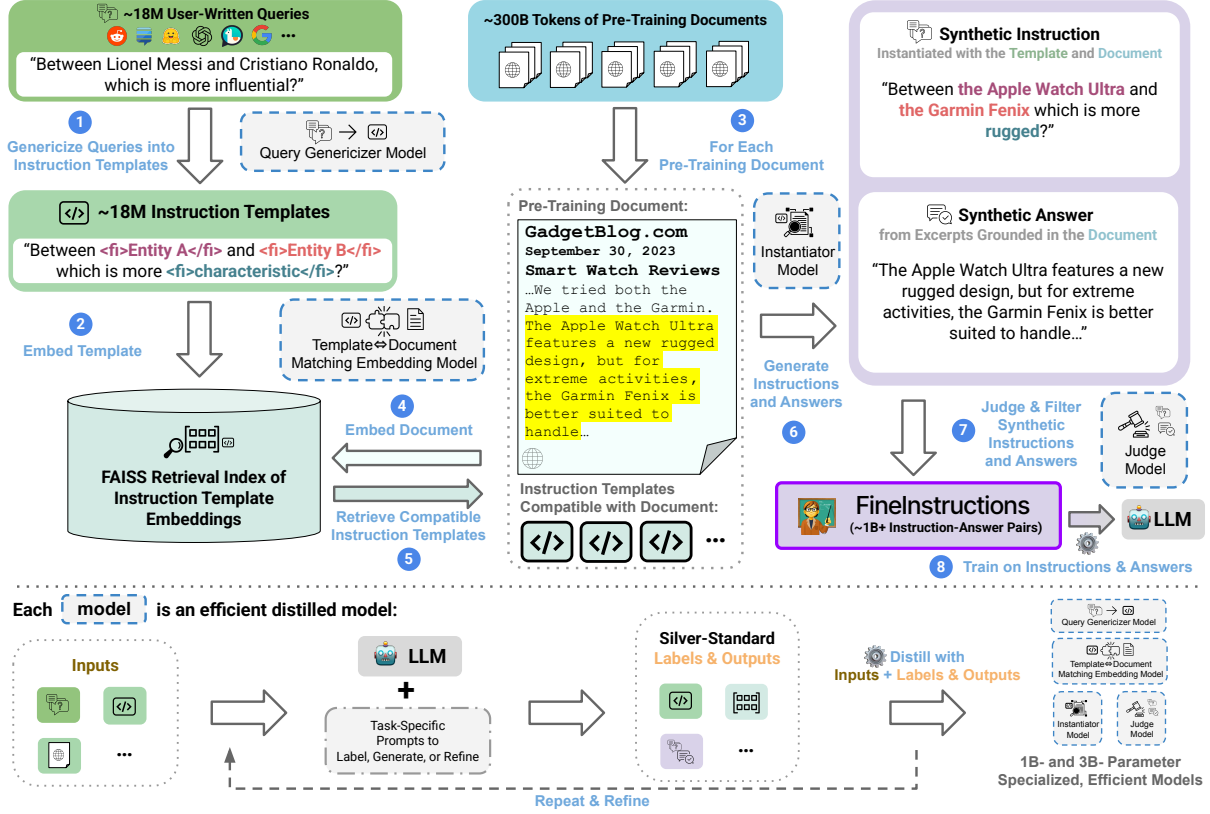


Figure 6.1: The FineInstructions pipeline for efficiently generating diverse, pre-training scale, synthetic instruction-answer pairs.

Prompts (51 queries) [262], NaturalInstructions (757 queries) [142], P3 (544 queries) [7], and FLAN (3.44K queries) [8]. We filter harmful queries using the OpenAI Moderation API [263] and decontaminate the queries from overlap with common benchmarks using the decontamination procedure used by Tulu 3 [251].

Our instruction template format involves using `<fi></fi>` tags that mark spans of the query that could be genericized and apply to other entities or scenarios. Within `<fi></fi>` tags is a short, natural language description of what kind of text might be appropriate at that span. We train an efficient Query Genericizer Model that converts queries to generic instruction templates. Using a series of prompts with an LLM we generate around ~50K query-template examples of from a subset of queries spanning the selected datasets. We also generate “compatible

Below is a generic query template that can be used for instruction-tuning an LLM. It contains a template of a natural language user query, with template variables using `<fi></fi>` tags (“`<fi>description of content that should go there....</fi>`”).

{{example of a template...}}

These templates can then be instantiated from a **compatible** document (such as a web page, article, book, essay, etc.) that has contain enough information to both fill in all of the template variables and also has the answer to the query so that we can create grounded, realistic, diverse instruction-tuning questions and answers using a combination of the template + document.

Given the provided template + a compatible document we found below:

1. ****Task:**** Can you instantiate the template by filling in the `<fi></fi>` template variables to create a realistic instruction that could be realistically asked about the content/topic/subject described in the document?

2. ****Answerability and Template Incompatibility:**** ...

{{other requirements...}}

```
<<<Instruction Template:
{{template}}
>>>
```

```
<<<Compatible Document:
{{document}}
>>>
```

Figure 6.2: An example of a prompt used to create silver-standard outputs to distill the Instantiator Model.

document descriptions” for each of these ~50K templates. The description is a short natural language paragraph describing the kinds of documents that could instantiate such a template as well as provide an answer. We fine-tune a Llama-3.2 1B Instruct model on these examples and transform all of the queries into templates and compatible document descriptions with the distilled model. The result of this pipeline after filtering harmful and contaminated queries is ~18M templates.

6.2.2 Matching Documents to Instruction Templates

We next use a semantic similarity embedding, BGE-M3, to embed the compatible document descriptions for the ~18M templates and build a FAISS retrieval index [264, 265]. Given a representative pre-training corpus of documents, we use an LLM to convert ~200K pre-training documents into a descriptions specifying the type of knowledge contained in each document which we also embed with the same embedding model. With each document’s embedding, we query the FAISS index and retrieve five potentially compatible instruction templates. We use the LLM to judge whether the document and instruction template are compatible to produce hard positive (1) and hard negative (0) examples of compatibility that are used with a cosine similarity loss to fine-tune the BGE-M3 embedding model. We fine-tune this model to directly embed documents and compatible document descriptions of instruction templates close together or far apart based on the compatibility labels [172]. We repeat this process and fine-tune BGE-M3 one more time. This time, we introduce a custom pooling layer that allows us to efficiently produce multiple embeddings at the pooling layer when embedding a document that semantically represent K different sections or chunks of the document so that we can retrieve instruction templates that are relevant to different pieces of information throughout a long document.

GAUSSIAN POOLING FOR DOCUMENT COVERAGE To ensure we can retrieve instruction templates that adequately cover the entire document, we train our embedding model to be used with “Gaussian pooling”. It supplements the standard mean pooling embedding computed globally over the entire document, with multiple “local” embeddings computed with soft, Gaussian-weighted pooling evenly distributed across the sequence. Each Gaussian kernel is centered at a fixed proportion of the input length, allowing the model to capture semanti-

cally distinct regions of long sequences and produce local embeddings that attend to different “chunks” of the document.

Let $H = [h_1, \dots, h_T]$ denote the sequence of token embeddings with attention mask $m_t \in \{0, 1\}$, T the effective sequence length, K the number of Gaussians to produce effectively representing K different chunks of the document, c_k the chunk center positions, $\rho_k = \frac{k}{K+1}$ their normalized positions, σ the Gaussian width parameter, and $\alpha \in [0, 1]$ the blending weight. We choose $K = 5$ chunks (producing 6 retrieval embeddings. including the global embedding), $\alpha = 1.0$, and $\sigma = 0.05$.

Input Token Embeddings and Attention Mask:

$$H = [h_1, \dots, h_T], \quad h_t \in \mathbb{R}^d, \quad m_t \in \{0, 1\}$$

Global Embedding:

$$\bar{h} = \frac{\sum_{t=1}^T m_t h_t}{\sum_{t=1}^T m_t}$$

Gaussian Centers for K Chunks:

$$c_k = \rho_k T, \quad \rho_k = \frac{k}{K+1}, \quad k = 1, \dots, K$$

Gaussian Weights:

$$w_{k,t} = \frac{m_t \exp\left(-\frac{1}{2} \left(\frac{t - c_k}{\sigma T}\right)^2\right)}{\sum_{t'=1}^T m_{t'} \exp\left(-\frac{1}{2} \left(\frac{t' - c_k}{\sigma T}\right)^2\right)}$$

Chunk-Local Embeddings:

$$\tilde{h}_k = \sum_{t=1}^T w_{k,t} h_t$$

Blend Chunk-Local Embeddings with Global Embedding:

$$h_k^* = (1 - \alpha)\bar{h} + \alpha\tilde{h}_k$$

Final Global and K Chunk-Local Embeddings:

$$E = [\bar{h}, h_1^*, \dots, h_K^*] \in \mathbb{R}^{d(K+1)}$$

In this second round of fine-tuning the embedding model, we aim to train the embedding model to maintain local semantic representation in the output token embeddings and reduce representation mixing so that the Gaussian pooling layer can be effective at producing distinct local representations. We create training instances by applying equivalent Gaussian weights at the input attention mask and only attending to tokens with a weight of ≥ 0.5 and producing K distinct local embeddings with these attention masks to retrieve and produce hard negative and hard positive examples once more. This time, we train on hard positives and hard negatives of instruction templates along with labels for which chunk index k of the document is relevant to the template. We compute the loss for the local embedding produced by the Gaussian pooling layer for the labeled chunk. The resulting final embedding model produces output token embeddings with decreased representation mixture. In our final FineInstructions dataset, we find a Pearson correlation of 0.99 between the index of the chunk (k) used to retrieve the instruction template and the location of the retrieved answer excerpt within the source document with most answer excerpts being selected from the range of 19%-71% of the way through the document, increasing linearly with chunk index used.

RETRIEVAL When performing retrieval, we keep matching templates above a manually selected cosine similarity threshold (0.865) and perform a weighted random sample to encourage diversity in the templates retrieved. Since our set of instruction templates contain both shorter, simple templates (with only one or two `<fi></fi>` tags) as well as longer, complex templates (with 10+ `<fi></fi>` tags), we compute sampling weights to match the distribution of the com-

plexity of templates derived from real-world LLM queries in the WildChat, LMSys*, OAsst1, and ShareLM datasets.

6.2.3 Generating Instructions and Answers

Vanilla Pre-Training Document:

Structure and Synchronicity for Better Charting

Two key characteristics will help you to ensure that your notes communicate not only what you did, but also what you were thinking. Fam Pract Manag. 2011 Jul-Aug;18(4):15-17.

How many times have you read a medical note that does not make the selection of the diagnosis or treatment clear? Have you ever read your own notes after receiving notice of a malpractice suit and winced at the inconsistencies? Poorly constructed medical notes are a widespread problem. I've seen it while reviewing the charts of medical students and residents to ensure that they met the standard of care and avoided malpractice risk. While most physicians document history and physical data with the required number of CPT elements, few clearly convey a line of reasoning that reveals their clinical thought process. A note that documents a detailed history and moderately complex decision making does not necessarily illuminate why certain decisions were reached or why a particular treatment was justified.

I wanted to find out where in the educational process students learned key documentation concepts, so I did some research that included informally querying medical students from multiple medical schools who were rotating at our residency. None of them could describe learning a formal structure for completing assessments and plans, which is consistent with a study that found only 4 percent of standardized encounters were accurately charted by medical students.¹ I also discovered that the 2010 United States Medical Licensing Examination clinical skills exam guide states that students are expected to present a list of differential diagnoses in order of likelihood along with desired evaluations, but no requirements existed for discussing the clinical rationale. As a result of these and other findings, I developed a formal framework, described in this article, to teach residents and students an appropriate way to construct their notes. The initial feedback has been highly positive.

Documenting a confirmed diagnosis

To begin, let's review what should be included when documenting a confirmed diagnosis. Generally, six elements are needed...

Instruction Template	Synthetic Instruction	Synthetic Answer
----------------------	-----------------------	------------------

Continued on next page...

I'm looking for a template or guidelines for how to write a <code><fi>type of document, e.g. technical report, research paper, etc.</fi></code> . The <code><fi>type of document</fi></code> is <code><fi>context or field of study</fi></code> -related and intended for a <code><fi>role or position</fi></code> to review the work of the <code><fi>task or investigation</fi></code> of a <code><fi>type of issue or problem</fi></code> related to <code><fi>field or industry</fi></code> .	I'm looking for a template or guidelines for how to write a medical note. The medical note is healthcare-related and intended for a physician to review the work of the clinical investigation of a medical issue related to patient care.	To write a medical note, consider the following guidelines. To begin, let's review what should be included when documenting a confirmed diagnosis. Generally, six elements are needed...
---	---	--

Figure 6.3: Illustrative example from the FineInstructions pipeline.

Finally, with a sample of ~100K pre-training documents and six compatible instruction templates, we use series of prompts with an LLM to instantiate the instruction templates and identify relevant excerpt(s) of the document that would be a relevant answer grounded in the document. Since most documents are not phrased to look like responses, we allow the LLM to rephrase the excerpt slightly or add a few words at the beginning of the excerpt that more directly answers the query or instruction before providing more context and detail with an excerpt. Since answers grounded in the document are more desirable than synthetically generated tokens, we ensure that the ratio of excerpted text in answers is ≥ 0.80 . Finally, these instructions and answers could be long in length and generating such instructions and answers at pre-training scale would be computationally expensive as decode-heavy workloads are far slower as decoding must happen token-by-token and introduces GPU memory bottlenecks. We note that because our answers are mainly direct excerpts from the document and our variables in the instruction template are mainly filled in by direct excerpts from the document as well, we can reduce the computation required at generation by introducing tags in the format of “`<excerpt>`It is known that`<...>`the principle of relativity.`</excerpt>`”. The model only needs to gener-

ate such tags with limited tokens to reference far longer spans from the document and after these tags are generated, we can expand them inexpensively and programatically. We create ~100K silver-standard examples and train a distilled model to instantiate instruction templates and generate answers by fine-tuning Llama-3.2 3B Instruct. With this efficient distilled model, we generate many instruction-answer pairs for a large number of pre-training documents and filter for high-quality instantiations and answers with judge prompts using an LLM until we are able to produce ~100K high-quality instantiated instruction and answer examples to train the final Instantiator Model in a second round. If the LLM determines the retrieved instruction template is not compatible with the document, we keep some of these instances as examples of when to output `null` instead of force a low-quality generation (~5% of the total examples). We ensure these ~100K new examples are also well stratified in length, complexity (number of template variables), and in topic using keyword-based filters to identify instructions-answer pairs related to math and code to ensure balanced representation in distillation examples.

6.2.4 Judging and Filtering Instructions and Answers

Having pre-training data formatted as standalone instruction-answer pairs makes our generated data amenable to being used with off-the-shelf reward and judge models. We implement a judging and filtering stage on our synthetic instructions and answers to yield a higher-quality set. We use the off-the-shelf Flow Judge model, a distilled 3.8B parameter judge model [266] with a 5-point Likert scale rubric ranging from 1 (irrelevant or off-topic) to 5 (fully addresses the query without extraneous, vague, or repetitive content), and retain instructions that score ≥ 4 for pre-training.

6.3 EXPERIMENTAL SETUP

In this section, we describe our experimental setup to validate whether training on synthetic instruction and answer pairs from the FineInstructions pipeline improves knowledge absorption and model performance.

6.3.1 Baselines

We compare standard pre-training against pre-training on synthetic instruction and answer pairs from FineInstructions derived from the same pre-training documents in controlled token-for-token experiments. We also select a number of relevant baselines from prior work that propose similar procedures of converting pre-training documents with synthetic transformations or rephrasing pre-training documents. The authors of these baselines have released the vanilla pre-training corpora used in their experiments as well as their pre-computed synthetic transformations over the pre-training corpora. We describe the various datasets and baselines below.

INSTRUCTION PRE-TRAINING (IPT) The “Instruction Pre-Training” method [126] proposes training an instruction and response synthesizer model to convert documents into instruction-answer pairs. It uses documents and examples of instructions and responses from academic NLP Q&A datasets as training data for this synthesizer model. We use both the IPT vanilla pre-training corpus consisting of ~23B tokens from RefinedWeb [118] and the pre-computed synthetically transformed version of that corpus converted via the IPT instruction synthesizer model.

NEMOTRON-CC The Nemotron-CC method [121] generates synthetic pre-training data by processing high-quality documents filtered from CommonCrawl [267] with an LLM using a mixture of tasks. These tasks include rephrasing documents, generating synthetic Q&A pairs, and extracting, distilling, and listing core knowledge from the document. We use the Nemotron-CC vanilla pre-training corpus (~300B tokens) and the pre-computed Nemotron-CC synthetic pre-training data created using their mixture of tasks. We also select ~300B tokens of pre-training data generated solely by their method of generating diverse Q&A as a standalone baseline that is the most directly comparable to IPT and FineInstructions. Lastly, we select ~300B tokens of their pre-computed synthetically rephrased pre-training data implementing the WRAP (Web Rephrase Augmented Pre-training) technique [128] to compare pre-training on instruction-answer pairs with pre-training on a strong rephrasing baseline.

6.3.2 Pre-Training

Using the vanilla pre-training documents from both of these datasets, we use our pipeline to retrieve six instruction templates (with $K = 5$ Gaussian pooling chunks to cover the document) and generate six instruction-answer pairs per document. For each pre-training document, we only randomly keep instruction-answer pairs with a total token count that does not exceed the token count of the source pre-training document.¹ On average, this results in ~3 instruction-answer pairs per document. We format these instruction and answer pairs with a simple chat template similar to the other synthetic baselines: “Instruction: {{instruction}}\n\nAnswer: {{answer}}” (included in the token count). This allows us to produce a dataset-controlled, token-for-token, equivalent set of FineInstructions pre-training data that is ~23B tokens and

¹If there is remaining token budget because instruction-answer pairs cannot exactly fill the token count of the source pre-training document, we rollover the remaining token budget to instruction-answer pairs for future pre-training documents.

~300B tokens for the IPT and Nemotron-CC datasets respectively. Distinct from FineInstructions, the baseline methods do not produce standalone question-answer pairs and instead produce questions that only make sense when the source document is provided as context (e.g. “What club does Helen like?”). They typically append their question-answer pairs at the end of the document (reading comprehension style Q&A) and we pre-train them in their native format. We perform our pre-training experiments using the Lingua framework [268] designed for controlled pre-training ablations on 8xH100s and pre-train 1.8B parameter models with a Llama-3 tokenizer [252] for each vanilla pre-training dataset, each baseline dataset, and our FineInstructions datasets. We train for a single epoch on datasets derived from Nemotron-CC and four epochs on datasets derived from IPT.

6.3.3 Benchmarks

We select three LLM evaluation benchmarks described below that aim to correlate with human judgments of response quality to evaluate both knowledge absorption and each trained model’s ability to respond to realistic user queries to LLMs that might ask for recommendations, advice, suggestions, etc. When evaluating each method, we format the benchmark questions and instructions into a chat template that matches the method’s training template and use the standard (“Instruction:” and “Answer:”) template for models pre-trained on vanilla pre-training data. We use greedy sampling when generating responses. Manually inspecting the responses for each method, we find all methods are able to respond and format answers to prompts reasonably well for accurate judging with few degenerate responses.

MIXEVAL The MixEval benchmark [269] is a subset of task instances from a variety of academic LLM benchmarks such as TriviaQA [270], MMLU [271], HellaSwag [272], Com-

nonsenseQA [273], among others. It optimizes creating a mixture that correlates with human judgments of response quality and grades responses with an LLM-as-judge given a reference answer. We use GPT-5 mini as a judge [274] and use the “2024-08-11” version of MixEval and on both the “Standard” and “Hard” splits.

MT-BENCH-101 The MT-Bench-101 benchmark is a multi-turn benchmark involving realistic, subjective user queries (opinions, advice, writing, etc.) graded via LLM-as-judge with rubric scaled out of 10 [275]. While it is a multi-turn benchmark, it is common to evaluate on this benchmark in single-turn fashion for models not-tuned for multi-turn chat. We use GPT-5 mini as a judge [274].

ALPACAEVAL The AlpacaEval benchmark evaluates on realistic queries and tasks a user may ask a LLM and evaluates with LLM-as-judge in a head-to-head evaluation between the responses from two models to produce a win rate [276]. It also corrects for length-bias in judging [277]. We use the supported GPT-4-Turbo model as our judge model.

6.4 RESULTS

The results of our pre-training experiments can be found in Table 6.1. Pre-training on FineInstructions outperforms both standard pre-training and the other synthetic baselines on both datasets. For example, we observe a ~69% improvement on MixEval compared to standard pre-training on the IPT dataset and a ~39% improvement on Nemotron-CC. Outputs from training on FineInstructions on average are preferred over outputs from training on other datasets on AlpacaEval. MT-Bench-101 is the only reference-free evaluation (with no gold reference answer

Method	MixEval		MT-Bench-101 <i>Likert Score</i>	AlpacaEval <i>FI Win Rate % (Δ Win Margin %)</i>
	Acc (%)			
	Standard	Hard		
<i>IPT Corpus (23B)</i>				
Standard Pre-Training	17.8	14.0	1.9	73.6% (Δ 47.2%)
IPT	19.8	16.7	2.4	68.2% (Δ 36.4%)
FineInstructions	31.7	19.2	2.8	–
<i>Nemotron-CC Corpus (300B)</i>				
Standard Pre-Training	24.0	17.1	3.5	63.6% (Δ 27.2%)
WRAP	22.8	18.4	3.6	65.1% (Δ 30.2%)
Q&A	27.1	18.9	3.4	76.1% (Δ 52.2%)
Nemotron-CC	24.5	16.7	3.6	65.9% (Δ 31.8%)
FineInstructions	33.0	21.8	3.9	–

Table 6.1: Benchmark performance of FineInstructions and various baselines. We **bold** the strongest method and **bold** the win rate % if FineInstructions wins.

or head-to-head comparison) resulting in evaluation scores that yield lower spread and differentiation between models. Notably, these improvements hold across both knowledge-focused and open-ended evaluation benchmarks, suggesting that FineInstructions yields more consistent generalization across both kinds of tasks. Both IPT and Nemotron-CC demonstrated fairly small improvement (0-2%) on most academic LLM benchmarks despite generating narrow instruction data in a highly similar style targeting the evaluation benchmark format (multiple choice, reading comprehension Q&A, etc.). On more realistic evaluations correlated to human judgments of response quality and on open-ended tasks real users may ask such as those in AlpacaEval, we find these other techniques yield subpar results compared to pre-training on the more diverse mixture of FineInstructions that does not explicitly generate instructions for a narrow set of academic benchmark tasks like multiple choice Q&A to optimize evaluation performance.

6.5 DISCUSSION

In this section, we discuss an analysis on the diversity of instructions generated, an additional experiment on incorporating a judging and filtering stage, and future directions.

6.5.1 Diversity of Instructions

We analyze the diversity of our generated instructions to ensure there is sufficient diversity with no overrepresentation of certain instruction templates. It is desirable that long-tail instruction templates (e.g. “Give me free apps that convert a file from `<fi>format #1</fi>` to `<fi>format #2</fi>`”) only compatible with highly specific documents are represented along with simpler instruction templates compatible with many documents (e.g. “Give me a summary of `<fi>topic</fi>`”). On the Nemotron dataset, 4.3M unique instruction templates were used to instantiate ~1.08B total instructions. We find no single instruction template comprises more than 0.09% of the generated instructions and that the majority of instruction templates are used to instantiate fewer than 1,000 instructions. Overall, we find a power-fit relationship between the number of pre-training documents (x) and the number of unique instruction templates (y) utilized at least once when generating instructions from those documents from our set of ~18M instruction templates as approximately: $y = 16,891 * x^{0.24}$ with an $r^2 = 0.96$. Around ~50% of instructions are instantiated from a template derived from GooAQ, with Reddit QA (~27%), LMSys Chat (~9%), and WildChat (~6%) following behind. All other sources represent $\leq 1\%$ each.

We also attempt to analyze the type of the instructions generated. We use Llama-3.3 70B Instruct with zero-shot prompts to classify instruction templates into various non-mutually

Domain/Category	Percentage (%)
Science	36.61%
Math	0.58%
Code	0.25%
Medicine	10.40%
Personal Life	14.74%
Reasoning Task	10.99%
Tasky vs. Q&A	6.42%

Table 6.2: 1B+ FineInstructions classified into various categories.

exclusive domains (science, math, code, medicine, etc.) and provide the results in Table 6.2. Additionally, we define categories for measuring if an instruction is a task requires reasoning over knowledge (e.g. “Compare and contrast `<fi>entity A</fi>` and `<fi>entity B</fi>`”) as well as if the instruction is “tasky” (e.g. “Write a critique of `<fi>media performance</fi>?`”) vs. a simple knowledge-recall Q&A task. Finally, we use the LLM to generate a short, one sentence description of each instruction template describing the task at hand and visualize these sentences in a sunburst chart for a subset of instructions in Figure 6.4 following Köksal et al. [140]. This kind of analysis with zero-shot classification may yield errors, but provides some high-level insights into the composition of tasks. The full prompts for such annotations can be found in our code.

6.5.2 Effect of Judging and Filtering

We conduct an ablation experiment to measure the effect of the final judging and filtering stage. We pre-train with and without judging and filtering and display these results in Table 6.3 on the IPT and Nemotron-CC datasets and find this added stage overall further improves the performance, especially on the AlpacaEval benchmark.

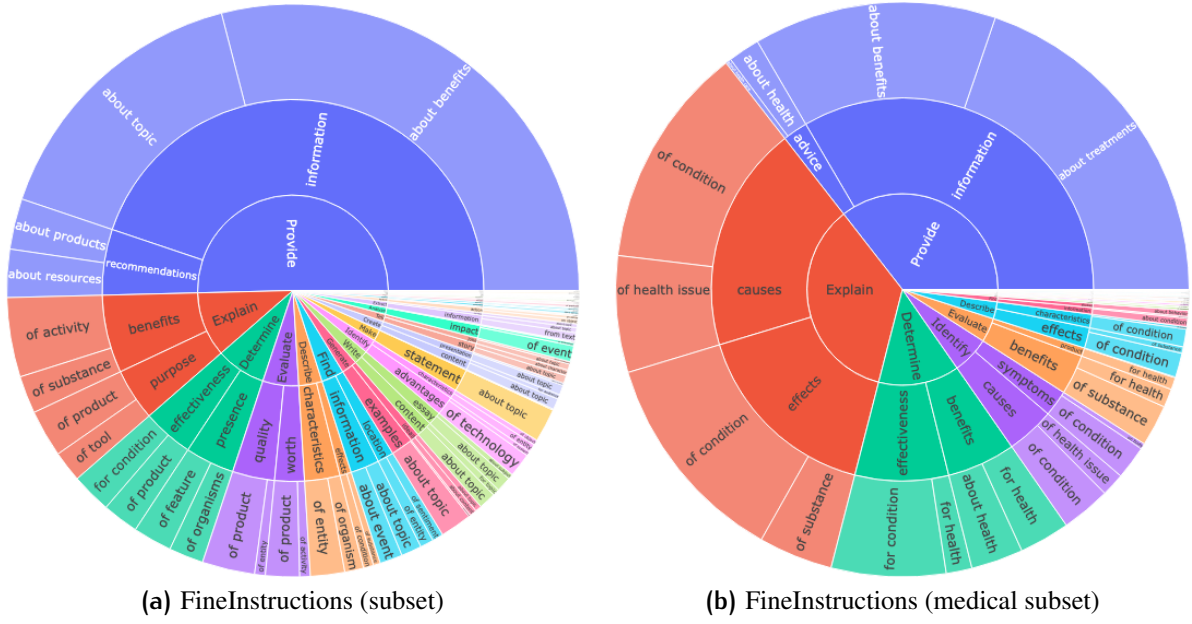


Figure 6.4: A visualization of the task diversity in FineInstructions. We also preview domain-specific diversity, comparing with a subset that was instantiated from medicine-related templates.

Method	MixEval		MT-Bench-101 <i>Likert Score</i>	AlpacaEval	
	<i>Acc (%)</i>			<i>FI Win (%)</i>	<i>FI Judged Win (%)</i>
	Standard	Hard			
<i>IPT Corpus (23B)</i>					
Standard Pre-Training	17.8	14.0	1.9	72.1% (Δ 44.2%)	73.6% (Δ 47.2%)
IPT	19.8	16.7	2.4	61.7% (Δ 23.4%)	68.2% (Δ 36.4%)
FineInstructions	30.1	20.2	2.6	–	46.5% (Δ -7%)
FineInstructions (judged)	31.7	19.2	2.8	–	–
<i>Nemotron-CC Corpus (300B)</i>					
Standard Pre-Training	24.0	17.1	3.5	54.9% (Δ 9.8%)	63.6% (Δ 27.2%)
WRAP	22.8	18.4	3.6	58.4% (Δ 16.8%)	65.1% (Δ 30.2%)
Q&A	27.1	18.9	3.4	68.7% (Δ 37.4%)	76.1% (Δ 52.2%)
Nemotron-CC	24.5	16.7	3.6	55.7% (Δ 11.4%)	65.9% (Δ 31.8%)
FineInstructions	33.3	21.8	3.6	–	61.4% (Δ 22.8%)
FineInstructions (judged)	33.0	21.8	3.9	–	–

Table 6.3: Incorporating a judging and filtering stage on top of FineInstructions leads to further improvements in performance.

6.5.3 Future Directions

The FineInstructions pipeline can be further optimized or scaled to yield better results. For example, the distribution of source queries, calibration of the matching/retrieval embedding, and the sampling weights all have influence on the composition and complexity of instructions generated. An optimal mixture and composition may yield further improvements in model performance. One common failure mode we observe in instructions generated by this pipeline is that complex templates are challenging to match and instantiate. We experimented with 1B and 3B parameter distilled models in this work, however, scaling to larger models could yield stronger performance on complex templates and longer documents. We also find there to be a gap in evaluation benchmarks with few benchmarks targeting the kind of long-tail realistic knowledge tasks users ask LLMs (recommendations, advice, suggestions, etc.) as opposed to factual knowledge. Moreover, pre-training on instruction-answer pairs yields a model that behaves similar to an instruction-tuned model that warps log-probabilities and makes it difficult to use evaluation benchmarks that rely on log-probability based evaluation. Prior studies have found this kind of evaluation is not reliable [278, 279] and instead recommend extractive or LLM-as-judge based judging instead as we use in this work. There are comparatively few general LLM benchmarks in the latter category. Finally, we note datasets like FineInstructions can be used to mine for large amounts of task-specific or domain-specific instructions. For example, with 1B+ instructions, even 0.58% of the instructions being math instructions yields 6M+ math instructions. We experiment with building a FAISS index over our set of templates with a semantic similarity embedding [264] and a full-text search index over our generated instructions to allow us to “mine” for relevant instructions efficiently. A search like “Teach me how to write a medical note” yields results for instruction-answer pairs like the one in Figure 6.3. This could enable experiments on training with different mixtures of topics and

task types as well as experiments with increasing the proportion of pre-training data on specific tasks to build a domain-specialist LLM which some prior work has found to be important to downstream performance [280, 281].

6.6 CONCLUSION

This chapter demonstrated how synthetic data can be leveraged to perform pre-training in a supervised, instruction-aligned format rather than through self-supervised next token prediction on pre-training documents. By transforming the learning objective and the structure of pre-training data, this approach introduces inductive biases that better reflect downstream usage patterns and improve knowledge absorption efficiency. This work provides strong evidence that synthetic data can reshape and advance the very foundations of LLM training.

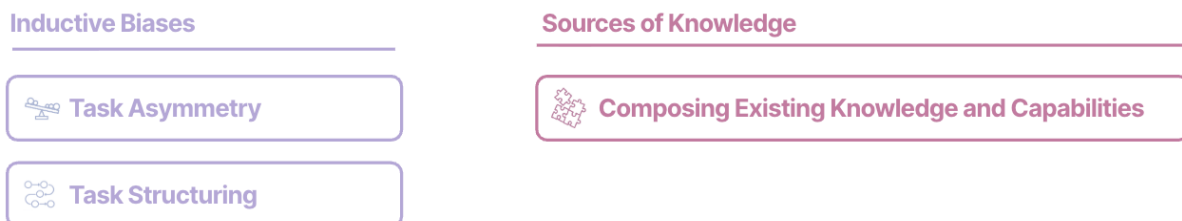


Figure 6.5: Inductive biases and sources of knowledge highlighted by the work on improving the broad general capabilities and performance of a model by applying synthetic data to the pre-training stage of LLM training.

The primary inductive bias applied in this work is task asymmetry. It is relatively easy for a model to perform a task like instantiating an instruction template and producing a grounded response given a document. Our pipeline includes multiple steps to create an effective and diverse dataset of instructions: genericizing queries, matching documents to compatible instruc-

tion templates, and instantiating those instruction templates and selecting relevant excerpts as answers. The pipeline is carefully constructed and represents task structuring by training and orchestrating specialized models distilled from LLMs in a specific manner optimized so that each step can be completed within the existing capabilities of an LLM while producing high-quality, diverse instructions that improve learning performance.

Next, I will conclude this dissertation and discuss promising future synthetic data research directions.

6.7 SUMMARY OF CONTRIBUTIONS

This work is based on a working manuscript “FineInstructions: A Web-Scale Instructions Dataset”. I led the overall research direction, conducted all of the training and evaluation experiments with GPU compute access and assistance from my collaborator Colin Raffel. I am the primary author responsible for writing this chapter and the working manuscript.

7 | CONCLUSION AND FUTURE DIRECTIONS

This dissertation aimed to answer the question of when and why synthetic data works and I proposed reframing synthetic data generated by LLMs as a mechanism by which human practitioners can encode helpful inductive biases as an answer to this question. In Chapter 1, I introduced a structured taxonomy of a broad set of inductive biases and sources of knowledge to aid the understanding of why synthetic data works, when it works, and the understanding of where the knowledge and mechanism for learning comes from when training on synthetic data. And in the subsequent chapters, Chapters 3, 5, and 6, I have systematically applied this taxonomy to my research to make an empirical case in support of the thesis of this dissertation. The research discussed has demonstrated synthetic data applied at various levels of application with LLMs, including in-context learning, fine-tuning, and pre-training. In Chapter 4, I introduced DataDreamer, a framework I developed for synthetic data generation and training, to support current and future research.

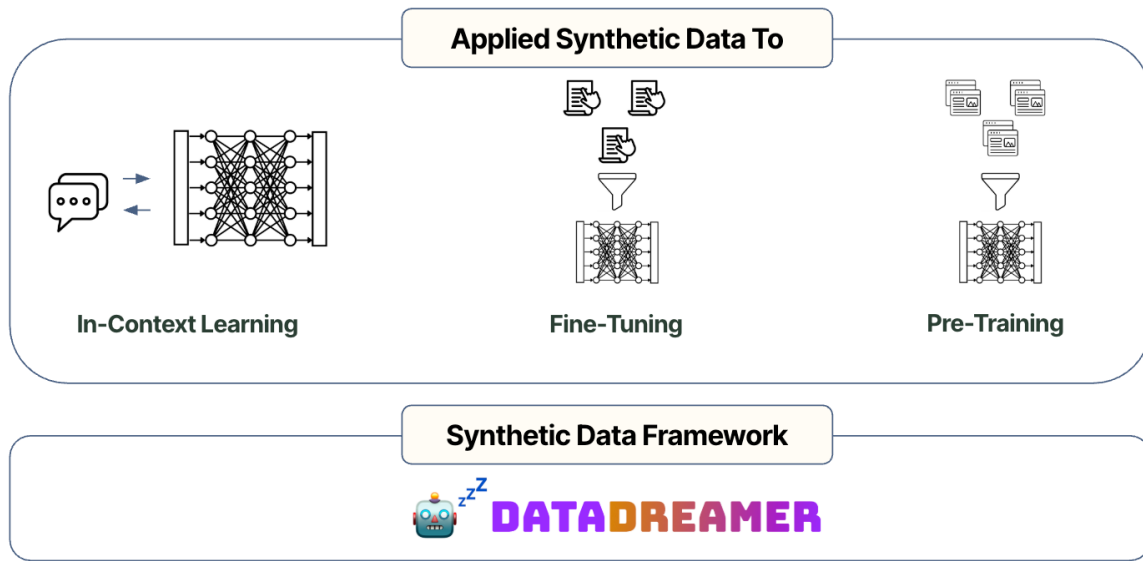


Figure 7.1: The works presented in this dissertation demonstrated applying synthetic data at three levels of LLM learning: 1) In-Context Learning 2) Fine-Tuning 3) Pre-Training.

In Chapter 3, I demonstrated how synthetic data can enhance LLM performance in an in-context learning setting on unsupervised machine translation. By bootstrapping synthetic parallel examples and applying self-consistency filtering I showed that a model can outperform supervised few-shot baselines in zero-shot on low-resource languages. This chapter highlighted how carefully designed learning biases—such as task priming (in-context examples), task asymmetry (self-consistency filtering), and English-centric task structuring—allows synthetic data to surface and refine latent knowledge already present in pre-trained models to improve task-specific performance on machine translation.

Chapter 4 introduced DataDreamer, a framework I developed to make synthetic data generation and training workflows easier to implement, reproduce, and extend. DataDreamer supports multi-stage workflows—including synthetic data generation, fine-tuning, evaluation, and self-improvement—while providing built-in caching, resumability, and reproducibility utilities. The framework enables practitioners to efficiently encode inductive biases at scale, lowering

the barrier to experimenting with complex workflows and fostering open, reproducible LLM research.

In Chapter 5, I showed that fine-tuning on carefully designed synthetic data can allow models to acquire qualitatively new capabilities. Across three case studies, I used DataDreamer-powered workflows to encode task-specific inductive biases at scale. First, StyleDistance incorporated task rules and heuristics by using synthetic parallel paraphrases spanning 40 chosen stylistic attributes to guide contrastive training of content-independent style embeddings that we demonstrated generalized to unseen styles after training. Next, with web agents, our unsupervised self-improvement procedure leveraged task asymmetry, taking advantage of the fact that it is easier for models to verify or critique action sequence trajectories than to generate them, while also surfacing latent world knowledge through model-authored tasks, yielding higher functional correctness, new capability acquisition, and more robust behavior. Lastly, with CoSyn, our generation pipeline applied task structuring and task asymmetry, using code as an intermediate representation along with external tools to produce 400K text-rich synthetic images with instruction data that enabled strong zero-shot performance on text-rich vision language understanding tasks.

Finally, in Chapter 6, DataDreamer was used to generate synthetic data at pre-training scale. In this chapter, I demonstrated how to apply task structuring and task asymmetry to restructure pre-training documents into instruction-answer pairs in order to instruction-tune at pre-training scale instead of performing next token prediction on raw pre-training documents. We found this guides and improves the model in learning general capabilities expected by downstream users in a compute-efficient manner, evaluating our model with a number of LLM benchmarks designed to correlate with human judgments of LLM quality and utility.

My works presented in this dissertation demonstrate leveraging LLMs to generate synthetic datasets allows practitioners to scale their prior knowledge and solve (or make progress on)

tasks that were previously out of reach. Synthetic data as a technique has the potential to broaden applications of machine learning to novel, data-scarce tasks and continue to further the frontier of model capabilities when available model training data is fully exhausted or begins to provide diminishing returns. I end with a brief discussion on central considerations for applying synthetic data and relevant promising future research directions.

7.1 CONSIDERATIONS FOR APPLYING SYNTHETIC DATA

SOURCING DIVERSITY Many of the approaches discussed in this dissertation rely on a seed source of diversity for sampling synthetic data. For example, in Chapter 3, a monolingual source corpus is used to source diversity in machine translation examples, in Section 5.3, personas are used to source diversity in topics and data type for creating synthetic text-rich images, and in Chapter 6, sources of human-written queries are used to create a large bank of instruction templates that are instantiated to create a pre-training scale instruction-answer dataset.

For any given task, finding a source of diversity that is in-distribution with the intended downstream use case is critical to the success of the application of a synthetic data approach. Without introducing such a source of diversity, the dataset is bound to the limited distribution modeled by the generating model and accessible through inference-time sampling which fuels model collapse [25].

IDENTIFYING MODEL WEAKNESSES AND GAPS Synthetic data approaches are successful when the practitioner can identify a model weakness or gap in knowledge and “patch” it with a learning bias. Instead of applying a rule or heuristic as typically would have been done in a classical AI system like an expert system, the “patch” is applied in the form of data as an induc-

tive bias allowing the model to be steered towards desired behavior while still benefiting from its general performance. In Chapter 3, we see a weakness in a general purpose de-noising language model on machine translation improved through being provided bootstrapped in-context examples of the desired task and we see a gap in performance on non-English translation pairs improved with English-centric bootstrapping. In Section 5.1, we see a weakness in prior text-style embeddings conflating content and style representations resolved through highly precise contrastive learning with synthetically constructed triplets. Finally, in Chapter 6, we see a weakness in models pre-trained on next token prediction when applied directly for instruction-following and general task capabilities. We find our procedure of restructuring pre-training documents and training on synthetic instruction-answer pairs better aligns the model with the expected downstream usage.

7.2 FUTURE DIRECTIONS

7.2.1 Synthetic Data for Data-Scarce Tasks

One promising area of synthetic data, where it perhaps has the most utility, is for data-scarce tasks where supervised datasets are non-existent or otherwise difficult to acquire but can be constructed via a sufficiently capable LLM. One example of this is discussed in Section 5.1 where I discuss using LLMs to create synthetic contrastive triplets for training strong, content-independent text style embeddings. While such a supervised dataset does not publicly exist, LLMs can easily create parallel examples of styled text, which when used for fine-tuning generalize to unseen styles.

While machine learning has made steady progress on low-level NLP tasks for which public datasets are widely available such as summarization or question-answering, progress on higher-level tasks more directly applicable to industry application has been slower. These higher-level tasks often involve longer contexts, subjective answers, complex user interactions or environmental feedback making them difficult to source. In some cases, these tasks have datasets which may have access restrictions or are otherwise proprietary. As a result, despite their practical importance, such tasks remain underexplored in academia. Synthetic data offers a path forward by generating high-quality training data or benchmarks for these otherwise inaccessible tasks and can help shift research attention from classical benchmarks toward more challenging and impactful problems.

Some examples of notable data-scarce tasks that similarly may fall into this category where synthetic data may hold potential as a strong approach are tasks like long-term conversational memory [282], search and information retrieval [283], recommendations [284], judging and reward/grading tasks [285], creativity tasks [286, 187], or healthcare tasks that typically have data restrictions [287].

7.2.2 Synthetic Data as Method For Model and Dataset Refinement

Another promising direction for synthetic data lies in its potential to enable models to improve their own performance. A model can generate synthetic training data derived from its own predictions along critiques and/or corrections, and then leverage this data to iteratively refine its abilities. Early work in this area suggests that repeated cycles of this process often lead to diminishing returns, but a few carefully designed rounds of self-training can nonetheless yield measurable gains. For example, methods such as self-critiquing outputs, filtering poor

generations with reward metrics or judging criteria, and applying self-distillation have been shown to enhance task performance in controlled settings [29, 30, 84, 81].

The potential impact of such approaches is significant. Instead of relying solely on external supervision or carefully curated datasets, models could begin to identify their own weaknesses, generate counterexamples or improved responses, and incorporate these into their training distribution as a form of self-correction. While the scope of improvements so far remains modest, the conceptual advance is important: synthetic data may serve not only as a resource for scaling supervised training but also as a mechanism for autonomous model and dataset refinement.

7.2.3 Synthetic Data as Method For Interpretability

Outside of model performance, synthetic data also has the potential to be used as a tool for model explanation and interpretation. Explainability and interpretability are an increasingly important area of research as AI adoption begins to grow. There are a number of works that apply this at the mechanistic level of neural networks to create short descriptions for single neurons in large neural networks by probing activations [288, 289]. Other works explore this at higher levels of explainability and interpretability where by explanations can be generated and used to interpret model predictions or latent space representations [94, 290]. While these methods may help users interpret and analyze model decisions, many of these approaches are still limited by the fact that LLMs typically generate post-hoc explanations [291] that are not faithful [292] to the internal reasoning and decisions used by the model to reach the prediction. Furthering the faithfulness of synthetic explanations remains a remaining challenge for future research.

7.2.4 Automating Synthetic Data Procedures and Workflows

While this dissertation demonstrates the value of synthetic data across a range of applications, all of the presented workflows share an important limitation: they are designed, implemented, tuned, and validated by practitioners (my co-authors and I) in a highly task-specific and iterative manner. Each case study requires careful identification of a model weakness or gap, followed by the manual design of a synthetic data generation strategy, and finally empirical validation and adjustment through multiple rounds of experimentation. This process, although effective, requires substantial domain expertise, engineering effort, and time.

A natural direction for future work is to move beyond practitioner-driven design and toward greater automation of synthetic data pipelines. Early progress in this direction might involve the development of meta-systems that monitor model performance across tasks, detect systematic errors, and propose candidate synthetic data generation procedures tailored to those deficiencies. For example, rather than relying on a practitioner to design contrastive triplets for style embeddings or action sequences for agent models, a higher-level controller could automatically search over data generation strategies, evaluate their downstream effect, and refine them over time. Such systems would not only reduce the reliance on expert intuition but also make synthetic data workflows more reproducible, scalable, and adaptable across domains. By reducing the human effort required to design synthetic data procedures and by enabling models to iteratively refine themselves, future research in this direction has the potential to redefine the role of synthetic data to be a foundation for continual, self-directed model improvement.

Although work towards this direction is nascent, a few notable relevant recent works begin to explore related directions around AutoML [293], automatic dataset selection [294, 295], and ideation, experimentation and validation [296, 297, 298]. My own work such as DataDreamer [16] provides a reasonable reference implementation for the kind of declarative

framework an automated system might use to design and implement such workflows and in Section 5.3 we make some progress towards a generalized data generation pipeline built on-top of DataDreamer to generate in-domain vision language data for any broad vision question-answering task given a short description of the category of synthetic images to be generated. In future years beyond this dissertation, I hope to explore and contribute to this research direction transforming synthetic data from a manual tool of model improvement to into a driver of automated, continual model self-improvement.

BIBLIOGRAPHY

- [1] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep learning.” In: *nature* 521.7553 (2015), pp. 436–444.
- [2] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, June 2019, pp. 4171–4186. DOI: [10 . 18653 / v1 / N19 - 1423](https://doi.org/10.18653/v1/N19-1423). URL: [https : / / aclanthology.org/N19-1423](https://aclanthology.org/N19-1423).
- [3] Colin Raffel et al. “Exploring the limits of transfer learning with a unified text-to-text transformer.” In: *The Journal of Machine Learning Research* 21.1 (2020), pp. 5485–5551.
- [4] Alec Radford et al. “Language Models are Unsupervised Multitask Learners.” In: (2019).
- [5] Tom Brown et al. “Language Models are Few-Shot Learners.” In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle et al. Vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901. URL: <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf>.
- [6] Long Ouyang et al. “Training language models to follow instructions with human feedback.” In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 27730–27744.
- [7] Victor Sanh et al. “Multitask Prompted Training Enables Zero-Shot Task Generalization.” In: *ArXiv abs/2110.08207* (2021). URL: <https://api.semanticscholar.org/CorpusID:239009562>.
- [8] Jason Wei et al. “Finetuned language models are zero-shot learners.” In: *arXiv preprint arXiv:2109.01652* (2021).

- [9] Yue Yu et al. “Large Language Model as Attributed Training Data Generator: A Tale of Diversity and Bias.” In: *Thirty-Seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. 2023.
- [10] Varun Kumar, Ashutosh Choudhary, and Eunah Cho. “Data augmentation using pre-trained transformer models.” In: *arXiv preprint arXiv:2003.02245* (2020).
- [11] Varun Kumar, Ashutosh Choudhary, and Eunah Cho. “Data augmentation using pre-trained transformer models.” In: *arXiv preprint arXiv:2003.02245* (2020).
- [12] Kang Min Yoo et al. “GPT3Mix: Leveraging large-scale language models for text augmentation.” In: *arXiv preprint arXiv:2104.08826* (2021).
- [13] Jesse Michael Han et al. “Unsupervised neural machine translation with generative language models only.” In: *arXiv preprint arXiv:2110.05448* (2021).
- [14] Jiacheng Ye et al. “Zerogen: Efficient zero-shot learning via dataset generation.” In: *arXiv preprint arXiv:2202.07922* (2022).
- [15] Or Honovich et al. “Unnatural instructions: Tuning language models with (almost) no human labor.” In: *arXiv preprint arXiv:2212.09689* (2022).
- [16] Ajay Patel, Colin Raffel, and Chris Callison-Burch. “DataDreamer: A Tool for Synthetic Data Generation and Reproducible LLM Workflows.” In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 3781–3799. DOI: [10.18653/v1/2024.acl-long.208](https://doi.org/10.18653/v1/2024.acl-long.208). URL: <https://aclanthology.org/2024.acl-long.208>.
- [17] Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. “ChatGPT outperforms crowd workers for text-annotation tasks.” In: *Proceedings of the National Academy of Sciences* 120.30 (2023), e2305016120.
- [18] Chris Callison-Burch. “Fast, Cheap, and Creative: Evaluating Translation Quality Using Amazon’s Mechanical Turk.” In: *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*. Ed. by Philipp Koehn and Rada Mihalcea. Singapore: Association for Computational Linguistics, Aug. 2009, pp. 286–295. URL: <https://aclanthology.org/D09-1030>.

- [19] Jesse Michael Han et al. “Unsupervised Neural Machine Translation with Generative Language Models Only.” In: *arXiv preprint arXiv:2110.05448* (2021).
- [20] Rico Sennrich, Barry Haddow, and Alexandra Birch. “Improving Neural Machine Translation Models with Monolingual Data.” In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Katrin Erk and Noah A. Smith. Berlin, Germany: Association for Computational Linguistics, Aug. 2016, pp. 86–96. DOI: [10 . 18653 / v1 / P16 - 1009](https://doi.org/10.18653/v1/P16-1009). URL: [https : //aclanthology.org/P16-1009](https://aclanthology.org/P16-1009).
- [21] Dan C Cireşan et al. “High-performance neural networks for visual object classification.” In: *arXiv preprint arXiv:1102.0183* (2011).
- [22] Suriya Gunasekar et al. “Textbooks Are All You Need.” In: *ArXiv abs/2306.11644* (2023). URL: <https://api.semanticscholar.org/CorpusID:259203998>.
- [23] Hoang Thanh-Tung and Truyen Tran. “Catastrophic forgetting and mode collapse in GANs.” In: *2020 international joint conference on neural networks (ijcnn)*. IEEE. 2020, pp. 1–10.
- [24] Joshua Kazdan et al. “Collapse or Thrive? Perils and Promises of Synthetic Data in a Self-Generating World.” In: *arXiv preprint arXiv:2410.16713* (2024).
- [25] Ilia Shumailov et al. “AI models collapse when trained on recursively generated data.” In: *Nature* 631.8022 (2024), pp. 755–759.
- [26] Jonathan Baxter. “A model of inductive bias learning.” In: *Journal of artificial intelligence research* 12 (2000), pp. 149–198.
- [27] Yann LeCun et al. “Handwritten digit recognition with a back-propagation network.” In: *Advances in neural information processing systems* 2 (1989).
- [28] Alex Wang et al. “Superglue: A stickier benchmark for general-purpose language understanding systems.” In: *Advances in neural information processing systems* 32 (2019).
- [29] Linfeng Zhang et al. *Be Your Own Teacher: Improve the Performance of Convolutional Neural Networks via Self Distillation*. 2019. arXiv: [1905.08094](https://arxiv.org/abs/1905.08094) [cs.LG].
- [30] Minh Pham et al. *Revisiting Self-Distillation*. 2022. arXiv: [2206.08491](https://arxiv.org/abs/2206.08491) [cs.LG].

- [31] Eric Zelikman et al. “STaR: Bootstrapping Reasoning With Reasoning.” In: *Advances in Neural Information Processing Systems*. Ed. by Alice H. Oh et al. 2022. URL: https://openreview.net/forum?id=_3ELRdg2sgI.
- [32] Ian Goodfellow et al. “Generative adversarial networks.” In: *Communications of the ACM* 63.11 (2020), pp. 139–144.
- [33] William Saunders et al. “Self-critiquing models for assisting human evaluators.” In: (2022). arXiv: [2206.05802](https://arxiv.org/abs/2206.05802) [cs.CL].
- [34] Jiaxin Huang et al. *Large Language Models Can Self-Improve*. 2022. arXiv: [2210.11610](https://arxiv.org/abs/2210.11610) [cs.CL].
- [35] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” In: *NeurIPS*. 2022. URL: http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.
- [36] Wojciech Zaremba and Ilya Sutskever. “Learning to execute.” In: *arXiv preprint arXiv:1410.4615* (2014).
- [37] Nicholas Carlini et al. “The secret sharer: evaluating and testing unintended memorization in neural networks.” In: *Proceedings of the 28th USENIX Conference on Security Symposium*. SEC’19. Santa Clara, CA, USA: USENIX Association, 2019, pp. 267–284. ISBN: 9781939133069.
- [38] Ziming Liu et al. “Towards Understanding Grokking: An Effective Theory of Representation Learning.” In: ().
- [39] Xinyi Wang et al. “Generalization vs Memorization: Tracing Language Models’ Capabilities Back to Pretraining Data.” In: *arXiv preprint arXiv:2407.14985* (2024).
- [40] Yizhong Wang et al. “Self-instruct: Aligning language model with self generated instructions.” In: *arXiv preprint arXiv:2212.10560* (2022).
- [41] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023. arXiv: [2210.03629](https://arxiv.org/abs/2210.03629) [cs.CL].
- [42] Bernardino Romera-Paredes et al. “Mathematical discoveries from program search with large language models.” In: *Nature* 625.7995 (2024), pp. 468–475.

- [43] Christopher JCH Watkins and Peter Dayan. “Q-learning.” In: *Machine learning* 8 (1992), pp. 279–292.
- [44] Volodymyr Mnih et al. “Human-level control through deep reinforcement learning.” In: *nature* 518.7540 (2015), pp. 529–533.
- [45] Kalpesh Krishna, John Wieting, and Mohit Iyyer. “Reformulating Unsupervised Style Transfer as Paraphrase Generation.” In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Ed. by Bonnie Webber et al. Online: Association for Computational Linguistics, Nov. 2020, pp. 737–762. DOI: [10.18653/v1/2020.emnlp-main.55](https://doi.org/10.18653/v1/2020.emnlp-main.55). URL: <https://aclanthology.org/2020.emnlp-main.55/>.
- [46] David Yarowsky. “Unsupervised word sense disambiguation rivaling supervised methods.” In: *33rd annual meeting of the association for computational linguistics*. 1995, pp. 189–196.
- [47] Alexander Ratner et al. “Snorkel: Rapid training data creation with weak supervision.” In: *Proceedings of the VLDB endowment. International conference on very large data bases*. Vol. 11. 3. 2017, p. 269.
- [48] Jesse Dodge et al. *Evaluating Prerequisite Qualities for Learning End-to-End Dialog Systems*. 2016. arXiv: [1511.06931](https://arxiv.org/abs/1511.06931) [cs.CL].
- [49] Brenden Lake and Marco Baroni. “Generalization without Systematicity: On the Compositional Skills of Sequence-to-Sequence Recurrent Networks.” In: *Proceedings of the 35th International Conference on Machine Learning*. Ed. by Jennifer Dy and Andreas Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, July 2018, pp. 2873–2882. URL: <https://proceedings.mlr.press/v80/lake18a.html>.
- [50] Koustuv Sinha et al. “CLUTRR: A Diagnostic Benchmark for Inductive Reasoning from Text.” In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Ed. by Kentaro Inui et al. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 4506–4515. DOI: [10.18653/v1/D19-1458](https://doi.org/10.18653/v1/D19-1458). URL: <https://aclanthology.org/D19-1458/>.
- [51] Terry Winograd. *Procedures as a representation for data in a computer program for understanding natural language*. Tech. rep. 1971.

- [52] Marc-Alexandre Côté et al. “Textworld: A learning environment for text-based games.” In: *Workshop on Computer Games*. Springer. 2018, pp. 41–75.
- [53] Mohit Shridhar et al. “Alfworld: Aligning text and embodied environments for interactive learning.” In: *arXiv preprint arXiv:2010.03768* (2020).
- [54] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. “Distilling the Knowledge in a Neural Network.” In: (2015). arXiv: [1503.02531 \[stat.ML\]](#).
- [55] Arnav Gudibande et al. “The false promise of imitating proprietary llms.” In: *arXiv preprint arXiv:2305.15717* (2023).
- [56] Ashish Vaswani et al. “Attention is all you need.” In: *NeurIPS*. 2017.
- [57] Jared Kaplan et al. *Scaling Laws for Neural Language Models*. 2020. arXiv: [2001.08361 \[cs.LG\]](#). URL: <https://arxiv.org/abs/2001.08361>.
- [58] Matthew E. Peters et al. “Deep Contextualized Word Representations.” In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Ed. by Marilyn Walker, Heng Ji, and Amanda Stent. New Orleans, Louisiana: Association for Computational Linguistics, June 2018, pp. 2227–2237. DOI: [10.18653/v1/N18-1202](#). URL: <https://aclanthology.org/N18-1202/>.
- [59] Aakanksha Chowdhery et al. *PaLM: Scaling Language Modeling with Pathways*. 2022. arXiv: [2204.02311 \[cs.CL\]](#). URL: <https://arxiv.org/abs/2204.02311>.
- [60] OpenAI. “GPT-4 technical report.” In: *arXiv preprint arXiv:2303.08774* (2023).
- [61] Ekin Akyürek et al. “What learning algorithm is in-context learning? Investigations with linear models.” In: *The Eleventh International Conference on Learning Representations*.
- [62] Sewon Min et al. *Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?* 2022. arXiv: [2202.12837 \[cs.CL\]](#). URL: <https://arxiv.org/abs/2202.12837>.
- [63] Steve Yadlowsky, Lyric Doshi, and Nilesch Tripuraneni. “Pretraining data mixtures enable narrow model selection capabilities in transformer models.” In: *arXiv preprint arXiv:2311.00871* (2023).

- [64] Jason Wei et al. “Chain-of-thought prompting elicits reasoning in large language models.” In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.
- [65] Patrick Lewis et al. “Retrieval-augmented generation for knowledge-intensive NLP tasks.” In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. NIPS ’20. Vancouver, BC, Canada: Curran Associates Inc., 2020. ISBN: 9781713829546.
- [66] Neil Houlsby et al. *Parameter-Efficient Transfer Learning for NLP*. 2019. arXiv: [1902.00751](https://arxiv.org/abs/1902.00751) [cs.LG]. URL: <https://arxiv.org/abs/1902.00751>.
- [67] Xiang Lisa Li and Percy Liang. *Prefix-Tuning: Optimizing Continuous Prompts for Generation*. 2021. arXiv: [2101.00190](https://arxiv.org/abs/2101.00190) [cs.CL]. URL: <https://arxiv.org/abs/2101.00190>.
- [68] Edward J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021. arXiv: [2106.09685](https://arxiv.org/abs/2106.09685) [cs.CL]. URL: <https://arxiv.org/abs/2106.09685>.
- [69] Paul Christiano et al. *Deep reinforcement learning from human preferences*. 2023. arXiv: [1706.03741](https://arxiv.org/abs/1706.03741) [stat.ML]. URL: <https://arxiv.org/abs/1706.03741>.
- [70] Rafael Rafailov et al. “Direct preference optimization: Your language model is secretly a reward model.” In: *arXiv preprint arXiv:2305.18290* (2023).
- [71] Xuanli He et al. “Generate, Annotate, and Learn: NLP with Synthetic Text.” In: *Transactions of the Association for Computational Linguistics* 10 (2022). Ed. by Brian Roark and Ani Nenkova, pp. 826–842. DOI: [10.1162/tac1-a-00492](https://doi.org/10.1162/tac1-a-00492). URL: <https://aclanthology.org/2022.tac1-1.48/>.
- [72] Alex Wang et al. “GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding.” In: *Proceedings of the 2018 EMNLP Workshop Black-boxNLP: Analyzing and Interpreting Neural Networks for NLP*. Ed. by Tal Linzen, Grzegorz Chrupała, and Afra Alishahi. Brussels, Belgium: Association for Computational Linguistics, Nov. 2018, pp. 353–355. DOI: [10.18653/v1/W18-5446](https://doi.org/10.18653/v1/W18-5446). URL: <https://aclanthology.org/W18-5446/>.
- [73] Or Honovich et al. *Unnatural Instructions: Tuning Language Models with (Almost) No Human Labor*. 2022. arXiv: [2212.09689](https://arxiv.org/abs/2212.09689) [cs.CL]. URL: <https://arxiv.org/abs/2212.09689>.

- [74] Haoran Li et al. *Synthetic Data (Almost) from Scratch: Generalized Instruction Tuning for Language Models*. 2024. arXiv: [2402.13064](https://arxiv.org/abs/2402.13064) [cs.CL]. URL: <https://arxiv.org/abs/2402.13064>.
- [75] Abdullatif Köksal et al. *LongForm: Effective Instruction Tuning with Reverse Instructions*. 2024. arXiv: [2304.08460](https://arxiv.org/abs/2304.08460) [cs.CL]. URL: <https://arxiv.org/abs/2304.08460>.
- [76] Takeshi Kojima et al. *Large Language Models are Zero-Shot Reasoners*. 2023. arXiv: [2205.11916](https://arxiv.org/abs/2205.11916) [cs.CL].
- [77] Zhihong Shao et al. *Synthetic Prompting: Generating Chain-of-Thought Demonstrations for Large Language Models*. 2023. arXiv: [2302.00618](https://arxiv.org/abs/2302.00618) [cs.CL]. URL: <https://arxiv.org/abs/2302.00618>.
- [78] Loubna Ben Allal et al. *Cosmopedia*. Feb. 2024. URL: <https://huggingface.co/datasets/HuggingFaceTB/cosmopedia>.
- [79] Tao Ge et al. *Scaling Synthetic Data Creation with 1,000,000,000 Personas*. 2024. arXiv: [2406.20094](https://arxiv.org/abs/2406.20094) [cs.CL]. URL: <https://arxiv.org/abs/2406.20094>.
- [80] Yiming Zhang et al. *Forcing Diffuse Distributions out of Language Models*. 2024. arXiv: [2404.10859](https://arxiv.org/abs/2404.10859) [cs.CL]. URL: <https://arxiv.org/abs/2404.10859>.
- [81] Jiaxin Huang et al. “Large language models can self-improve.” In: *arXiv preprint arXiv:2210.11610* (2022).
- [82] Xiang Lisa Li et al. “Benchmarking and improving generator-validator consistency of language models.” In: *arXiv preprint arXiv:2310.01846* (2023).
- [83] Zixiang Chen et al. *Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models*. 2024. arXiv: [2401.01335](https://arxiv.org/abs/2401.01335) [cs.LG].
- [84] Weizhe Yuan et al. *Self-Rewarding Language Models*. 2024. arXiv: [2401.10020](https://arxiv.org/abs/2401.10020) [cs.CL].
- [85] Adam Lopez. “Statistical machine translation.” In: *ACM Comput. Surv.* 40.3 (Aug. 2008). ISSN: 0360-0300. DOI: [10.1145/1380584.1380586](https://doi.org/10.1145/1380584.1380586). URL: <https://doi.org/10.1145/1380584.1380586>.

- [86] Philipp Koehn and Rebecca Knowles. “Six Challenges for Neural Machine Translation.” In: *Proceedings of the First Workshop on Neural Machine Translation*. Ed. by Thang Luong et al. Vancouver: Association for Computational Linguistics, Aug. 2017, pp. 28–39. DOI: [10.18653/v1/W17-3204](https://doi.org/10.18653/v1/W17-3204). URL: <https://aclanthology.org/W17-3204/>.
- [87] Linting Xue et al. “mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer.” In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Ed. by Kristina Toutanova et al. Online: Association for Computational Linguistics, June 2021, pp. 483–498. DOI: [10.18653/v1/2021.naacl-main.41](https://doi.org/10.18653/v1/2021.naacl-main.41). URL: <https://aclanthology.org/2021.naacl-main.41>.
- [88] Kangchen Zhu et al. “StyleFlow: Disentangle Latent Representations via Normalizing Flow for Unsupervised Text Style Transfer.” In: *International Conference on Language Resources and Evaluation*. 2022. URL: <https://api.semanticscholar.org/CorpusID:254854477>.
- [89] Ning Dai et al. “Style Transformer: Unpaired Text Style Transfer without Disentangled Latent Representation.” In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 2019, pp. 5997–6007.
- [90] Parker Riley et al. “TextSETTR: Few-Shot Text Style Extraction and Tunable Targeted Restyling.” In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 2021, pp. 3786–3800.
- [91] Julien Hay et al. “Representation learning of writing style.” In: *Proceedings of the 6th Workshop on Noisy User-generated Text (W-NUT 2020)*. Nov. 2020.
- [92] Anna Wegmann, Marijn Schraagen, Dong Nguyen, et al. “Same Author or Just Same Topic? Towards Content-Independent Style Representations.” In: *Proceedings of the 7th Workshop on Representation Learning for NLP*. Association for Computational Linguistics. 2022, p. 249.
- [93] Rafael A Rivera-Soto et al. “Learning universal authorship representations.” In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 2021, pp. 913–919.
- [94] Ajay Patel et al. “Learning Interpretable Style Embeddings via Prompting LLMs.” In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. Ed. by

- Houda Bouamor, Juan Pino, and Kalika Bali. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 15270–15290. DOI: [10.18653/v1/2023.findings-emnlp.1020](https://doi.org/10.18653/v1/2023.findings-emnlp.1020). URL: <https://aclanthology.org/2023.findings-emnlp.1020>.
- [95] Shuyan Zhou et al. “Webarena: A realistic web environment for building autonomous agents.” In: *arXiv preprint arXiv:2307.13854* (2023).
- [96] Konstantinos Bousmalis et al. *RoboCat: A Self-Improving Generalist Agent for Robotic Manipulation*. 2023. arXiv: [2306.11706](https://arxiv.org/abs/2306.11706) [cs.R0].
- [97] Caglar Gulcehre et al. “Reinforced Self-Training (ReST) for Language Modeling.” In: (2023). arXiv: [2308.08998](https://arxiv.org/abs/2308.08998) [cs.CL].
- [98] Yifan Song et al. *Trial and Error: Exploration-Based Trajectory Optimization for LLM Agents*. 2024. arXiv: [2403.02502](https://arxiv.org/abs/2403.02502) [cs.CL].
- [99] Jiayi Pan et al. *Autonomous Evaluation and Refinement of Digital Agents*. 2024. arXiv: [2404.06474](https://arxiv.org/abs/2404.06474) [cs.AI].
- [100] Matthew Johnson-Roberson et al. “Driving in the Matrix: Can virtual worlds replace human-generated annotations for real world tasks?” In: *2017 IEEE International Conference on Robotics and Automation (ICRA)*. Singapore, Singapore: IEEE Press, 2017, pp. 746–753. DOI: [10.1109/ICRA.2017.7989092](https://doi.org/10.1109/ICRA.2017.7989092). URL: <https://doi.org/10.1109/ICRA.2017.7989092>.
- [101] Justin Johnson et al. “Clevr: A diagnostic dataset for compositional language and elementary visual reasoning.” In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 2901–2910.
- [102] Paola Cascante-Bonilla et al. “Simvqa: Exploring simulated environments for visual question answering.” In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 5056–5066.
- [103] Jieyu Zhang et al. “ProVision: Programmatically Scaling Vision-centric Instruction Data for Multimodal Language Models.” In: *arXiv preprint arXiv:2412.07012* (2024).
- [104] Samira Ebrahimi Kahou et al. “FigureQA: An annotated figure dataset for visual reasoning.” In: *arXiv preprint arXiv:1710.07300* (2017).
- [105] Kushal Kafle et al. “DVQA: Understanding data visualizations via question answering.” In: *CVPR*. 2018.

- [106] Nitesh Methani et al. “PlotQA: Reasoning over scientific plots.” In: *WACV*. 2020.
- [107] Hrituraj Singh and Sumit Shekhar. “STL-CQA: Structure-based transformers with localization and encoding for chart question answering.” In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, pp. 3275–3284.
- [108] Shengzhi Li and Nima Tajbakhsh. “Scigraphqa: A large-scale synthetic multi-turn question-answering dataset for scientific graphs.” In: *arXiv preprint arXiv:2308.03349* (2023).
- [109] Victor Carbune et al. “Chart-based reasoning: Transferring capabilities from llms to vlms.” In: *arXiv preprint arXiv:2403.12596* (2024).
- [110] Yucheng Han et al. “Chartllama: A multimodal llm for chart understanding and generation.” In: *arXiv preprint arXiv:2311.16483* (2023).
- [111] Wei He et al. “Distill Visual Chart Reasoning Ability from LLMs to MLLMs.” In: *arXiv preprint arXiv:2410.18798* (2024).
- [112] Risa Shinoda et al. “SBS Figures: Pre-training Figure QA from Stage-by-Stage Synthesized Images.” In: *arXiv preprint arXiv:2412.17606* (2024).
- [113] Renqiu Xia et al. “Chartx & chartvlm: A versatile benchmark and foundation model for complicated chart reasoning.” In: *arXiv preprint arXiv:2402.12185* (2024).
- [114] Suriya Gunasekar et al. *Textbooks Are All You Need*. 2023. arXiv: [2306 . 11644 \[cs.CL\]](#).
- [115] Marah Abdin et al. “Phi-4 technical report.” In: *arXiv preprint arXiv:2412.08905* (2024).
- [116] Kimi Team et al. “Kimi k2: Open agentic intelligence.” In: *arXiv preprint arXiv:2507.20534* (2025).
- [117] Zitong Yang et al. “Synthetic continued pretraining.” In: *The Thirteenth International Conference on Learning Representations*.
- [118] Guilherme Penedo et al. “The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only.” In: *CoRR* (2023).

- [119] Margaret Mitchell Colin Raffel Leandro Von, Werra Thomas Wolf, and Hugging Face. “The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale.” In: ().
- [120] Jeffrey Li et al. “Datacomp-1m: In search of the next generation of training sets for language models.” In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 14200–14282.
- [121] Dan Su et al. “Nemotron-CC: Transforming Common Crawl into a refined long-horizon pretraining dataset.” In: *arXiv preprint arXiv:2412.02595* (2024).
- [122] Alon Albalak et al. “Efficient Online Data Mixing For Language Model Pre-Training.” In: *R0-FoMo: Robustness of Few-shot and Zero-shot Learning in Large Foundation Models*.
- [123] Sang Michael Xie et al. “Doremi: Optimizing data mixtures speeds up language model pretraining.” In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 69798–69818.
- [124] Jiasheng Ye et al. “Data Mixing Laws: Optimizing Data Mixtures by Predicting Language Modeling Performance.” In: *The Thirteenth International Conference on Learning Representations*.
- [125] Alexander Wettig et al. “Organize the Web: Constructing Domains Enhances Pre-Training Data Curation.” In: *arXiv preprint arXiv:2502.10341* (2025).
- [126] Daixuan Cheng et al. “Instruction Pre-Training: Language Models are Supervised Multitask Learners.” In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, pp. 2529–2550.
- [127] Cheng Deng et al. “Learning Foundation Language Models for Geoscience Knowledge Understanding and Utilization.” In: (2023).
- [128] Pratyush Maini et al. “Rephrasing the Web: A Recipe for Compute and Data-Efficient Language Modeling.” In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2024, pp. 14044–14072.
- [129] Thao Nguyen et al. “Recycling the Web: A Method to Enhance Pre-training Data Quality and Quantity for Language Models.” In: *arXiv preprint arXiv:2506.04689* (2025).
- [130] Or Honovich et al. “Unnatural Instructions: Tuning Language Models with (Almost) No Human Labor.” In: (2022). [arXiv: 2212.09689 \[cs.CL\]](https://arxiv.org/abs/2212.09689).

- [131] Rohan Taori et al. *Stanford Alpaca: An Instruction-following LLaMA model*. https://github.com/tatsu-lab/stanford_alpaca. 2023.
- [132] Xian Li et al. “Self-Alignment with Instruction Backtranslation.” In: *The Twelfth International Conference on Learning Representations*.
- [133] Haoran Li et al. “Synthetic data (almost) from scratch: Generalized instruction tuning for language models.” In: *arXiv preprint arXiv:2402.13064* (2024).
- [134] Nathan Lambert et al. “TÜLU 3: Pushing Frontiers in Open Language Model Post-Training.” In: *arXiv preprint arXiv:2411.15124* (2024).
- [135] Subhabrata Mukherjee et al. *Orca: Progressive Learning from Complex Explanation Traces of GPT-4*. 2023. arXiv: [2306.02707](https://arxiv.org/abs/2306.02707) [cs.CL].
- [136] Wing Lian et al. *OpenOrca: An Open Dataset of GPT Augmented FLAN Reasoning Traces*. <https://huggingface.co/Open-Orca/OpenOrca>. 2023.
- [137] Xuezhi Wang et al. “Self-Consistency Improves Chain of Thought Reasoning in Language Models.” In: *The Eleventh International Conference on Learning Representations*.
- [138] Aaron Jaech et al. “Openai o1 system card.” In: *arXiv preprint arXiv:2412.16720* (2024).
- [139] Daya Guo et al. “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning.” In: *arXiv preprint arXiv:2501.12948* (2025).
- [140] Abdullatif Köksal et al. *LongForm: Effective Instruction Tuning with Reverse Instructions*. 2024. arXiv: [2304.08460](https://arxiv.org/abs/2304.08460) [cs.CL].
- [141] Stephen H. Bach et al. *PromptSource: An Integrated Development Environment and Repository for Natural Language Prompts*. 2022. arXiv: [2202.01279](https://arxiv.org/abs/2202.01279) [cs.LG].
- [142] Swaroop Mishra et al. “Cross-task generalization via natural language crowdsourcing instructions.” In: *ACL*. 2022.
- [143] Yizhong Wang et al. “Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ Tasks.” In: *EMNLP*. 2022.

- [144] Avanika Narayan et al. “Cookbook: A framework for improving LLM generative abilities via programmatic data generating templates.” In: *arXiv preprint arXiv:2410.05224* (2024).
- [145] Ajay Patel et al. “Bidirectional language models are also few-shot learners.” In: *arXiv preprint arXiv:2209.14500* (2022).
- [146] Tianyi Zhang et al. “BERTScore: Evaluating Text Generation with BERT.” In: *International Conference on Learning Representations*. 2019.
- [147] Xi Victoria Lin et al. “Few-shot Learning with Multilingual Language Models.” In: *CoRR* abs/2112.10668 (2021). arXiv: [2112.10668](https://arxiv.org/abs/2112.10668). URL: <https://arxiv.org/abs/2112.10668>.
- [148] Naman Goyal et al. “The flores-101 evaluation benchmark for low-resource and multilingual machine translation.” In: *arXiv preprint arXiv:2106.03193* (2021).
- [149] Haokun Liu et al. “Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning.” In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 1950–1965.
- [150] Alexander M Rush. “MiniChain: A Small Library for Coding with Large Language Models.” In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 2023, pp. 311–317.
- [151] Pengfei Liu et al. “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing.” In: *ACM Computing Surveys* 55.9 (2023), pp. 1–35.
- [152] Melanie Sclar et al. “Quantifying Language Models’ Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting.” In: *arXiv preprint arXiv:2310.11324* (2023).
- [153] Ajay Jaiswal et al. “Compressing llms: The truth is rarely pure and never simple.” In: *arXiv preprint arXiv:2310.01382* (2023).
- [154] Ilia Shumailov et al. *The Curse of Recursion: Training on Generated Data Makes Models Forget*. 2023. arXiv: [2305.17493](https://arxiv.org/abs/2305.17493) [cs.LG].
- [155] Lianmin Zheng et al. “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena.” In: *arXiv preprint arXiv:2306.05685* (2023).

- [156] Jinlan Fu et al. “Gptscore: Evaluate as you desire.” In: *arXiv preprint arXiv:2302.04166* (2023).
- [157] Yann Dubois et al. “Alpacafarm: A simulation framework for methods that learn from human feedback.” In: *arXiv preprint arXiv:2305.14387* (2023).
- [158] Cheng-Han Chiang and Hung-yi Lee. “Can Large Language Models Be an Alternative to Human Evaluations?” In: *arXiv preprint arXiv:2305.01937* (2023).
- [159] Jesse Michael Han et al. “Unsupervised neural machine translation with generative language models only.” In: *arXiv preprint arXiv:2110.05448* (2021).
- [160] Haokun Liu et al. “Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning.” In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 1950–1965.
- [161] Cheng-Yu Hsieh et al. “Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes.” In: *arXiv preprint arXiv:2305.02301* (2023).
- [162] Swaroop Mishra et al. “Cross-Task Generalization via Natural Language Crowdsourcing Instructions.” In: *Annual Meeting of the Association for Computational Linguistics*. 2021. URL: <https://api.semanticscholar.org/CorpusID:237421373>.
- [163] Nisan Stiennon et al. “Learning to summarize with human feedback.” In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 3008–3021.
- [164] Yuntao Bai et al. “Constitutional ai: Harmlessness from ai feedback.” In: *arXiv preprint arXiv:2212.08073* (2022).
- [165] Thomas Wolf et al. “HuggingFace’s Transformers: State-of-the-art Natural Language Processing.” In: *CoRR* abs/1910.03771 (2019). arXiv: [1910.03771](https://arxiv.org/abs/1910.03771). URL: <http://arxiv.org/abs/1910.03771>.
- [166] Leandro von Werra et al. *TRL: Transformer Reinforcement Learning*. <https://github.com/huggingface/trl>. 2020.
- [167] Quentin Lhoest et al. “Datasets: A community library for natural language processing.” In: *arXiv preprint arXiv:2109.02846* (2021).

- [168] Mahima Pushkarna, Andrew Zaldivar, and Oddur Kjartansson. “Data cards: Purposeful and transparent dataset documentation for responsible ai.” In: *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*. 2022, pp. 1776–1826.
- [169] Margaret Mitchell et al. “Model cards for model reporting.” In: *Proceedings of the conference on fairness, accountability, and transparency*. 2019, pp. 220–229.
- [170] Tim Dettmers and Luke Zettlemoyer. “The case for 4-bit precision: k-bit inference scaling laws, 2022.” In: URL <https://arxiv.org/abs/2212.09720> ().
- [171] Sourab Mangrulkar et al. *PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods*. <https://github.com/huggingface/peft>. 2022.
- [172] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Nov. 2019. URL: <http://arxiv.org/abs/1908.10084>.
- [173] Adam Paszke et al. “Pytorch: An imperative style, high-performance deep learning library.” In: *Advances in neural information processing systems* 32 (2019).
- [174] Yanli Zhao et al. “Pytorch FSDP: experiences on scaling fully sharded data parallel.” In: *arXiv preprint arXiv:2304.11277* (2023).
- [175] Zhilin Yang et al. “HotpotQA: A dataset for diverse, explainable multi-hop question answering.” In: *arXiv preprint arXiv:1809.09600* (2018).
- [176] Peiyuan Zhang et al. *TinyLlama: An Open-Source Small Language Model*. 2024. arXiv: [2401.02385](https://arxiv.org/abs/2401.02385) [cs.CL].
- [177] Matt Deitke et al. “Molmo and pixmo: Open weights and open data for state-of-the-art multimodal models.” In: *arXiv preprint arXiv:2409.17146* (2024).
- [178] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 3982–3992.
- [179] Jacob Goldberger et al. “Neighbourhood components analysis.” In: *Advances in neural information processing systems* 17 (2004).

- [180] Prannay Khosla et al. “Supervised contrastive learning.” In: *Advances in neural information processing systems* 33 (2020), pp. 18661–18673.
- [181] Florian Schroff, Dmitry Kalenichenko, and James Philbin. “Facenet: A unified embedding for face recognition and clustering.” In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 815–823.
- [182] Di Jin et al. “Deep Learning for Text Style Transfer: A Survey.” In: *Computational Linguistics* 48.1 (Apr. 2022), pp. 155–205. ISSN: 0891-2017. DOI: [10.1162/colia_00426](https://doi.org/10.1162/colia_00426). eprint: https://direct.mit.edu/coli/article-pdf/48/1/155/2006608/coli_a_00426.pdf. URL: https://doi.org/10.1162/coli%5C_a%5C_00426.
- [183] Yla R Tausczik and James W Pennebaker. “The psychological meaning of words: LIWC and computerized text analysis methods.” In: *Journal of language and social psychology* 29.1 (2010), pp. 24–54.
- [184] Dongyeop Kang and Eduard H. Hovy. “xSLUE: A Benchmark and Analysis Platform for Cross-Style Language Understanding and Evaluation.” In: *ArXiv abs/1911.03663* (2019). URL: <https://api.semanticscholar.org/CorpusID:207853312>.
- [185] Anna Wegmann and Dong Nguyen. “Does It Capture STEL? A Modular, Similarity-based Linguistic Style Evaluation Framework.” In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 2021, pp. 7109–7130.
- [186] Colin Raffel et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.” In: *Journal of Machine Learning Research* 21 (2020), 1–6.
- [187] Zachary Horvitz et al. “TinyStyler: Efficient Few-Shot Text Style Transfer with Authorship Embeddings.” In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. Ed. by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 13376–13390. DOI: [10.18653/v1/2024.findings-emnlp.781](https://doi.org/10.18653/v1/2024.findings-emnlp.781). URL: <https://aclanthology.org/2024.findings-emnlp.781/>.
- [188] Sudha Rao and Joel Tetreault. “Dear Sir or Madam, May I Introduce the GYAFC Dataset: Corpus, Benchmarks and Metrics for Formality Style Transfer.” In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. New Orleans, Louisiana: Association for Computational Linguistics, June 2018, pp. 129–140. DOI: [10.18653/v1/N18-1012](https://doi.org/10.18653/v1/N18-1012). URL: <https://aclanthology.org/N18-1012>.

- [189] Ajay Patel et al. “StyleDistance: Stronger Content-Independent Style Embeddings with Synthetic Parallel Examples.” In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Ed. by Luis Chiruzzo, Alan Ritter, and Lu Wang. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 8662–8685. ISBN: 979-8-89176-189-6. DOI: [10.18653/v1/2025.naacl-long.436](https://doi.org/10.18653/v1/2025.naacl-long.436). URL: <https://aclanthology.org/2025.naacl-long.436/>.
- [190] Yixuan Weng et al. “Large language models are better reasoners with self-verification.” In: *arXiv preprint arXiv:2212.09561* (2022).
- [191] Zhibin Gou et al. *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*. 2024. arXiv: [2305.11738](https://arxiv.org/abs/2305.11738) [cs.CL].
- [192] Jesse Michael Han et al. “Unsupervised Neural Machine Translation with Generative Language Models Only.” In: *CoRR* abs/2110.05448 (2021). arXiv: [2110.05448](https://arxiv.org/abs/2110.05448). URL: <https://arxiv.org/abs/2110.05448>.
- [193] Wei-Lin Chiang et al. *Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference*. 2024. arXiv: [2403.04132](https://arxiv.org/abs/2403.04132) [cs.AI].
- [194] Ajay Patel et al. *Bidirectional Language Models Are Also Few-shot Learners*. 2023. arXiv: [2209.14500](https://arxiv.org/abs/2209.14500) [cs.LG].
- [195] Zheng Yuan et al. *Scaling Relationship on Learning Mathematical Reasoning with Large Language Models*. 2023. arXiv: [2308.01825](https://arxiv.org/abs/2308.01825) [cs.CL].
- [196] Jinze Bai et al. *Qwen Technical Report*. 2023. arXiv: [2309.16609](https://arxiv.org/abs/2309.16609) [cs.CL].
- [197] Yinhan Liu et al. “RoBERTa: A Robustly Optimized BERT Pretraining Approach.” In: *CoRR* abs/1907.11692 (2019). arXiv: [1907.11692](https://arxiv.org/abs/1907.11692). URL: <http://arxiv.org/abs/1907.11692>.
- [198] Victor Sanh et al. “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.” In: *CoRR* abs/1910.01108 (2019). arXiv: [1910.01108](https://arxiv.org/abs/1910.01108). URL: <http://arxiv.org/abs/1910.01108>.
- [199] Tim Dettmers et al. *QLoRA: Efficient Finetuning of Quantized LLMs*. 2023. arXiv: [2305.14314](https://arxiv.org/abs/2305.14314) [cs.LG].

- [200] Marius-Constantin Dinu et al. *SymbolicAI: A framework for logic-based approaches combining generative models and solvers*. 2024. arXiv: [2402.00854 \[cs.LG\]](#).
- [201] Kaitao Song et al. “MPNet: Masked and Permuted Pre-training for Language Understanding.” In: *arXiv preprint arXiv:2004.09297* (2020).
- [202] Donald J. Berndt and James Clifford. “Using Dynamic Time Warping to Find Patterns in Time Series.” In: *KDD Workshop*. 1994. URL: <https://api.semanticscholar.org/CorpusID:929893>.
- [203] Yunzhen Feng et al. “A Tale of Tails: Model Collapse as a Change of Scaling Laws.” In: (2024). URL: <https://openreview.net/forum?id=dE8BznvZV>.
- [204] Ajay Patel et al. *Large Language Models Can Self-Improve At Web Agent Tasks*. 2024. arXiv: [2405.20309 \[cs.LG\]](#). URL: <https://arxiv.org/abs/2405.20309>.
- [205] Alec Radford et al. “Learning transferable visual models from natural language supervision.” In: *ICML*. 2021.
- [206] Haotian Liu et al. “Visual Instruction Tuning.” In: *NeurIPS*. 2023.
- [207] Akari Asai et al. “Openscholar: Synthesizing scientific literature with retrieval-augmented lms.” In: *arXiv preprint arXiv:2411.14199* (2024).
- [208] Danna Gurari et al. “Vizwiz grand challenge: Answering visual questions from blind people.” In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 3608–3617.
- [209] Tianbao Xie et al. *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*. 2024. arXiv: [2404.07972 \[cs.AI\]](#).
- [210] Yue Yu et al. *Large Language Model as Attributed Training Data Generator: A Tale of Diversity and Bias*. 2023. arXiv: [2306.15895 \[cs.CL\]](#).
- [211] Ajay Patel, Colin Raffel, and Chris Callison-Burch. “DataDreamer: A Tool for Synthetic Data Generation and Reproducible LLM Workflows.” In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 3781–3799. DOI: [10.18653/v1/2024.acl-long.208](#). URL: <https://aclanthology.org/2024.acl-long.208>.

- [212] Anthropic. *Introducing the next generation of claude*. 2024. URL: <https://www.anthropic.com/news/claude-3-family>.
- [213] Albert Q Jiang et al. “Mistral 7B.” In: *arXiv preprint arXiv:2310.06825* (2023).
- [214] Ahmed Masry et al. “ChartQA: A Benchmark for Question Answering about Charts with Visual and Logical Reasoning.” In: *ACL*. 2022.
- [215] Minesh Mathew, Dimosthenis Karatzas, and CV Jawahar. “DocVQA: A dataset for VQA on document images.” In: *WACV*. 2021.
- [216] Minesh Mathew et al. “InfographicVQA.” In: *WACV*. 2022.
- [217] Yoonsik Kim, Moonbin Yim, and Ka Yeon Song. “TableVQA-Bench: A Visual Question Answering Benchmark on Multiple Table Domains.” In: *arXiv preprint arXiv:2404.19205* (2024).
- [218] Aniruddha Kembhavi et al. “A diagram is worth a dozen images.” In: *ECCV*. 2016.
- [219] Amanpreet Singh et al. “Towards VQA Models That Can Read.” In: *CVPR*. 2019.
- [220] Gilles Baechler et al. *ScreenAI: A Vision-Language Model for UI and Infographics Understanding*. 2024. arXiv: [2402.04615](https://arxiv.org/abs/2402.04615) [cs.CV].
- [221] Yash Goyal et al. “Making the V in VQA Matter: Elevating the Role of Image Understanding in Visual Question Answering.” In: *Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.
- [222] Drew A Hudson and Christopher D Manning. “Gqa: A new dataset for real-world visual reasoning and compositional question answering.” In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 6700–6709.
- [223] Kenneth Marino et al. “OK-VQA: A Visual Question Answering Benchmark Requiring External Knowledge.” In: *CVPR*. 2019.
- [224] Anand Mishra et al. “OCR-VQA: Visual Question Answering by Reading Text in Images.” In: *ICDAR*. 2019.
- [225] Dustin Schwenk et al. “A-OKVQA: A benchmark for visual question answering using world knowledge.” In: *ECCV*. 2022.

- [226] Pan Lu et al. “Learn to explain: Multimodal reasoning via thought chains for science question answering.” In: *NeurIPS*. 2022.
- [227] Pan Lu et al. “Dynamic Prompt Learning via Policy Gradient for Semi-structured Mathematical Reasoning.” In: *ICLR*. 2023.
- [228] Ali Furkan Biten et al. “Scene text visual question answering.” In: *ICCV*. 2019.
- [229] Manoj Acharya, Kushal Kafle, and Christopher Kanan. “TallyQA: Answering complex counting questions.” In: *AAAI*. 2019.
- [230] Lucas Beyer et al. “PaliGemma: A versatile 3B VLM for transfer.” In: *arXiv preprint arXiv:2407.07726* (2024).
- [231] Le Xue et al. “xGen-MM (BLIP-3): A family of open large multimodal models.” In: *arXiv preprint arXiv:2408.08872* (2024).
- [232] Shengbang Tong et al. “Cambrian-1: A fully open, vision-centric exploration of multimodal LLMs.” In: *NeurIPS*. 2024.
- [233] Haotian Liu et al. *LLaVA-NeXT: Improved reasoning, OCR, and world knowledge*. Jan. 2024. URL: <https://llava-vl.github.io/blog/2024-01-30-llava-next/>.
- [234] Bo Li et al. “LLaVA-OneVision: Easy visual task transfer.” In: *arXiv preprint arXiv:2408.03326* (2024).
- [235] Praveesh Agrawal et al. “Pixtral 12B.” In: *arXiv preprint arXiv:2410.07073* (2024).
- [236] Meta. *Llama 3.2: Revolutionizing edge AI and vision with open, customizable models*. 2024. URL: <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>.
- [237] Gemini Team. “Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context.” In: *arXiv preprint arXiv:2403.05530* (2024).
- [238] Anthropic. *The Claude 3 Model Family: Opus, Sonnet, Haiku*. 2024. URL: https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf.
- [239] Colin Raffel et al. “Exploring the limits of transfer learning with a unified text-to-text transformer.” In: *Journal of machine learning research* 21.140 (2020), pp. 1–67.

- [240] Victor Carbune et al. “Chart-based reasoning: Transferring capabilities from llms to vlms.” In: *arXiv preprint arXiv:2403.12596* (2024).
- [241] N Reimers. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” In: *arXiv preprint arXiv:1908.10084* (2019).
- [242] Kanzhi Cheng et al. *SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents*. 2024. arXiv: [2401.10935 \[cs.HC\]](https://arxiv.org/abs/2401.10935).
- [243] Wenyi Hong et al. “Cogagent: A visual language model for gui agents.” In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, pp. 14281–14290.
- [244] Boyu Gou et al. “Navigating the digital world as humans do: Universal visual grounding for gui agents.” In: *arXiv preprint arXiv:2410.05243* (2024).
- [245] Yue Yang et al. “Scaling Text-Rich Image Understanding via Code-Guided Synthetic Multimodal Data Generation.” In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Wanxiang Che et al. Vienna, Austria: Association for Computational Linguistics, July 2025, pp. 17486–17505. ISBN: 979-8-89176-251-0. DOI: [10.18653/v1/2025.acl-long.855](https://doi.org/10.18653/v1/2025.acl-long.855). URL: <https://aclanthology.org/2025.acl-long.855/>.
- [246] Mike Conover et al. *Free Dolly: Introducing the World’s First Truly Open Instruction-Tuned LLM*. 2023. URL: <https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm> (visited on 06/30/2023).
- [247] Nazneen Rajani et al. *No Robots*. https://huggingface.co/datasets/HuggingFaceH4/no_robots. 2023.
- [248] Chunting Zhou et al. “Lima: Less is more for alignment.” In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 55006–55021.
- [249] Sreyan Ghosh et al. “A closer look at the limitations of instruction tuning.” In: *Proceedings of the 41st International Conference on Machine Learning*. 2024, pp. 15559–15589.
- [250] John Hewitt et al. “Instruction following without instruction tuning.” In: *arXiv preprint arXiv:2409.14254* (2024).

- [251] Nathan Lambert et al. “TÜLU 3: Pushing Frontiers in Open Language Model Post-Training.” In: *CoRR* (2024).
- [252] Meta AI. “The llama 3 herd of models.” In: *arXiv preprint arXiv:2407.21783* (2024).
- [253] Wenting Zhao et al. “WildChat: 1M ChatGPT Interaction Logs in the Wild.” In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=Bl8u7ZRlbM>.
- [254] Lianmin Zheng et al. *Judging LLM-as-a-judge with MT-Bench and Chatbot Arena*. 2023. arXiv: [2306.05685](https://arxiv.org/abs/2306.05685) [cs.CL].
- [255] Lianmin Zheng et al. *Judging LLM-as-a-judge with MT-Bench and Chatbot Arena*. 2023. arXiv: [2306.05685](https://arxiv.org/abs/2306.05685) [cs.CL].
- [256] Andreas Köpf et al. “OpenAssistant Conversations-Democratizing Large Language Model Alignment.” In: ().
- [257] Zhilin Wang et al. *HelpSteer: Multi-attribute Helpfulness Dataset for SteerLM*. 2023. arXiv: [2311.09528](https://arxiv.org/abs/2311.09528) [cs.CL].
- [258] Daniel Khashabi et al. “GooAQ: Open Question Answering with Diverse Answer Types.” In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. 2021, pp. 421–433.
- [259] Shachar Don-Yehiya, Leshem Choshen, and Omri Abend. “The ShareLM Collection and Plugin: Contributing Human-Model Chats for the Benefit of the Community.” In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. Ed. by Pushkar Mishra, Smaranda Muresan, and Tao Yu. Vienna, Austria: Association for Computational Linguistics, July 2025, pp. 167–177. ISBN: 979-8-89176-253-4. URL: <https://aclanthology.org/2025.acl-demo.17/>.
- [260] Chaitanya Malaviya et al. “ExpertQA: Expert-Curated Questions and Attributed Answers.” In: *arXiv*. Sept. 2023. URL: <https://arxiv.org/abs/2309.07852>.
- [261] Yunxiang Li et al. “ChatDoctor: A Medical Chat Model Fine-Tuned on a Large Language Model Meta-AI (LLaMA) Using Medical Domain Knowledge.” In: *Cureus* 15.6 (2023).

- [262] Harrison Chase. *LangChain*. Oct. 2022. URL: <https://github.com/langchain-ai/langchain>.
- [263] Todor Markov et al. “A holistic approach to undesired content detection in the real world.” In: *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence*. 2023, pp. 15009–15018.
- [264] Shitao Xiao et al. *C-Pack: Packaged Resources To Advance General Chinese Embedding*. 2023. arXiv: [2309.07597](https://arxiv.org/abs/2309.07597) [cs.CL].
- [265] Matthijs Douze et al. “The faiss library.” In: *IEEE Transactions on Big Data* (2025).
- [266] Flow AI. *Flow Judge: An Open Small Language Model for LLM System Evaluations*. Technical Report. Model: Flow-Judge v0.1 (3.8B parameters); Apache 2.0 license. Flow AI, 2024. URL: <https://www.flow-ai.com/blog/flow-judge>.
- [267] Common Crawl Foundation. *Common Crawl Web Corpus*. <https://commoncrawl.org/>. Accessed: 2025-10-26; dataset release CC-MAIN-2025-08. 2025.
- [268] Mathurin Videau et al. *Meta Lingua: A minimal PyTorch LLM training library*. 2024. URL: <https://github.com/facebookresearch/lingua>.
- [269] Jinjie Ni et al. “MixEval: Deriving Wisdom of the Crowd from LLM Benchmark Mixtures.” In: *arXiv preprint arXiv:2406.06565* (2024).
- [270] Mandar Joshi et al. “TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension.” In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2017, pp. 1601–1611.
- [271] Dan Hendrycks et al. “Measuring Massive Multitask Language Understanding.” In: *International Conference on Learning Representations*.
- [272] Rowan Zellers et al. “HellaSwag: Can a Machine Really Finish Your Sentence?” In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Ed. by Anna Korhonen, David Traum, and Lluís Màrquez. Florence, Italy: Association for Computational Linguistics, July 2019, pp. 4791–4800. DOI: [10.18653/v1/P19-1472](https://doi.org/10.18653/v1/P19-1472). URL: <https://aclanthology.org/P19-1472/>.

- [273] Alon Talmor et al. “CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge.” In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, June 2019, pp. 4149–4158. DOI: [10 . 18653 / v1 / N19 - 1421](https://doi.org/10.18653/v1/N19-1421). URL: <https://aclanthology.org/N19-1421/>.
- [274] OpenAI. *GPT-5 System Card*. Tech. rep. OpenAI, Aug. 2025. URL: <https://cdn.openai.com/gpt-5-system-card.pdf>.
- [275] Ge Bai et al. “MT-Bench-101: A Fine-Grained Benchmark for Evaluating Large Language Models in Multi-Turn Dialogues.” In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2024, pp. 7421–7454.
- [276] Xuechen Li et al. *AlpacaEval: An Automatic Evaluator of Instruction-following Models*. https://github.com/tatsu-lab/alpaca_eval. May 2023.
- [277] Yann Dubois et al. “Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators.” In: *arXiv preprint arXiv:2404.04475* (2024).
- [278] Xinpeng Wang et al. ““My Answer is C”: First-Token Probabilities Do Not Match Text Answers in Instruction-Tuned Language Models.” In: *Findings of the Association for Computational Linguistics: ACL 2024*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 7407–7416. DOI: [10 . 18653 / v1 / 2024 . findings - acl . 441](https://doi.org/10.18653/v1/2024.findings-acl.441). URL: <https://aclanthology.org/2024.findings-acl.441/>.
- [279] Nikhil Chandak et al. “Answer Matching Outperforms Multiple Choice for Language Model Evaluation.” In: *arXiv preprint arXiv:2507.02856* (2025).
- [280] Joel Jang et al. “Exploring the Benefits of Training Expert Language Models over Instruction Tuning.” In: *ICML 2023*. International Machine Learning Society (IMLS). 2023, pp. 14702–14729.
- [281] Chufan Shi et al. “Specialist or Generalist? Instruction Tuning for Specific NLP Tasks.” In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Ed. by Houda Bouamor, Juan Pino, and Kalika Bali. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 15336–15348. DOI: [10 . 18653 /](https://doi.org/10.18653/)

- v1/2023.emnlp-main.947. URL: <https://aclanthology.org/2023.emnlp-main.947/>.
- [282] Adyasha Maharana et al. *Evaluating Very Long-Term Conversational Memory of LLM Agents*. 2024. arXiv: [2402.17753](https://arxiv.org/abs/2402.17753) [cs.CL]. URL: <https://arxiv.org/abs/2402.17753>.
- [283] Rolf Jagerman et al. “Query expansion by prompting large language models.” In: *arXiv preprint arXiv:2305.03653* (2023).
- [284] Kun Su et al. “REGEN: A Dataset and Benchmarks with Natural Language Critiques and Narratives.” In: *arXiv preprint arXiv:2503.11924* (2025).
- [285] Flow AI. *Flow Judge: An Open Small Language Model for LLM System Evaluations*. <https://www.flow-ai.com/blog/flow-judge>. Accessed: September 22, 2025. 2025.
- [286] Ajay Patel, Nicholas Andrews, and Chris Callison-Burch. *Low-Resource Authorship Style Transfer: Can Non-Famous Authors Be Imitated?* 2024. arXiv: [2212.08986](https://arxiv.org/abs/2212.08986) [cs.CL]. URL: <https://arxiv.org/abs/2212.08986>.
- [287] Ruixiang Tang et al. “Does synthetic data generation of LLMs help clinical text mining? 2023.” In: *arXiv preprint arXiv:2303.04360* ().
- [288] Steven Bills et al. “Language models can explain neurons in language models.” In: *OpenAI* (May 2023). Core Research Contributor; Author contributions statement included. Correspondence: interpretability@openai.com. URL: <https://openai.com/research/language-models-can-explain-neurons>.
- [289] Adly Templeton et al. *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*. <https://www.anthropic.com/research/scaling-monosemanticity>. * Core Contributor. Correspondence to henighan@anthropic.com. May 2024.
- [290] Milad Alshomary et al. “Latent Space Interpretation for Stylistic Analysis and Explainable Authorship Attribution.” In: *Proceedings of the 31st International Conference on Computational Linguistics*. Ed. by Owen Rambow et al. Abu Dhabi, UAE: Association for Computational Linguistics, Jan. 2025, pp. 1124–1135. URL: <https://aclanthology.org/2025.coling-main.75/>.
- [291] Nicholas Kroeger et al. “Are large language models post hoc explainers?” In: (2023).

- [292] Andreas Madsen, Sarath Chandar, and Siva Reddy. “Are self-explanations from Large Language Models faithful?” In: *Findings of the Association for Computational Linguistics: ACL 2024*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 295–337. DOI: [10.18653/v1/2024.findings-acl.19](https://doi.org/10.18653/v1/2024.findings-acl.19). URL: <https://aclanthology.org/2024.findings-acl.19/>.
- [293] Patara Trirat, Wonyong Jeong, and Sung Ju Hwang. “Automl-agent: A multi-agent llm framework for full-pipeline automl.” In: *arXiv preprint arXiv:2410.02958* (2024).
- [294] Vijay Viswanathan et al. *Prompt2Model: Generating Deployable Models from Natural Language Instructions*. 2023. arXiv: [2308.12261](https://arxiv.org/abs/2308.12261) [cs.CL].
- [295] Saumya Gandhi et al. “Better synthetic data by retrieving and transforming existing datasets.” In: *arXiv preprint arXiv:2404.14361* (2024).
- [296] Chenglei Si, Diyi Yang, and Tatsunori Hashimoto. “Can llms generate novel research ideas? a large-scale human study with 100+ nlp researchers.” In: *arXiv preprint arXiv:2409.04109* (2024).
- [297] Yutaro Yamada et al. “The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search.” In: *arXiv preprint arXiv:2504.08066* (2025).
- [298] Giulio Starace et al. *PaperBench: Evaluating AI’s Ability to Replicate AI Research*. 2025. arXiv: [2504.01848](https://arxiv.org/abs/2504.01848) [cs.AI]. URL: <https://arxiv.org/abs/2504.01848>.
- [299] noop entry, can be ignored.
- [300] Oana Ignat et al. “A PhD Student’s Perspective on Research in NLP in the Era of Very Large Language Models.” In: *arXiv preprint arXiv:2305.12544* (2023).
- [301] Patrick Haluptzok, Matthew Bowers, and Adam Tauman Kalai. “Language Models Can Teach Themselves to Program Better.” In: *The Eleventh International Conference on Learning Representations*. 2023. URL: <https://openreview.net/forum?id=SaRj2ka1XZ3>.
- [302] Nihal V. Nayak et al. “Learning to Generate Instruction Tuning Datasets for Zero-Shot Task Adaptation.” In: (2024). arXiv: [2402.18334](https://arxiv.org/abs/2402.18334) [cs.CL].
- [303] Avi Singh et al. “Beyond Human Data: Scaling Self-Training for Problem-Solving with Language Models.” In: *Transactions on Machine Learning Research* (2024). Expert

- Certification. ISSN: 2835-8856. URL: <https://openreview.net/forum?id=lNAyUngGFK>.
- [304] Renat Aksitov et al. “ReST meets ReAct: Self-Improvement for Multi-Step Reasoning LLM Agent.” In: *ICLR 2024 Workshop on Large Language Model (LLM) Agents*. 2024. URL: <https://openreview.net/forum?id=7xknRLr7QE>.
 - [305] Hanze Dong et al. “RAFT: Reward rAnked FineTuning for Generative Foundation Model Alignment.” In: *Transactions on Machine Learning Research* (2023). ISSN: 2835-8856. URL: <https://openreview.net/forum?id=m7p507zbly>.
 - [306] Xian Li et al. “Self-Alignment with Instruction Backtranslation.” In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=loijHJBRSrT>.
 - [307] Hung Le et al. “CodeChain: Towards Modular Code Generation Through Chain of Self-revisions with Representative Sub-modules.” In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=vYhg1xSj8j>.
 - [308] Chunting Zhou et al. “LIMA: Less Is More for Alignment.” In: *Thirty-seventh Conference on Neural Information Processing Systems*. 2023. URL: <https://openreview.net/forum?id=KBM0KmX2he>.
 - [309] Yizhong Wang et al. “Self-Instruct: Aligning Language Models with Self-Generated Instructions.” In: (2023). arXiv: [2212.10560](https://arxiv.org/abs/2212.10560) [cs.CL].
 - [310] Can Xu et al. “WizardLM: Empowering Large Pre-Trained Language Models to Follow Complex Instructions.” In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=CfXh93NDgH>.
 - [311] Lichang Chen et al. “AlpaGasus: Training a Better Alpaca with Fewer Data.” In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=FdVXgSJhvz>.
 - [312] Arnav Gudibande et al. “The False Promise of Imitating Proprietary Language Models.” In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=Kz3yckpCN5>.

- [313] Erik Jones et al. “Teaching Language Models to Hallucinate Less with Synthetic Tasks.” In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=xpw7V0P136>.
- [314] Ronen Eldan and Yuanzhi Li. “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?” In: (2023). arXiv: [2305.07759](https://arxiv.org/abs/2305.07759) [cs.CL].
- [315] Suriya Gunasekar et al. “Textbooks Are All You Need.” In: (2023). arXiv: [2306.11644](https://arxiv.org/abs/2306.11644) [cs.CL].
- [316] Yann Dubois et al. “AlpacaFarm: A Simulation Framework for Methods that Learn from Human Feedback.” In: *Thirty-seventh Conference on Neural Information Processing Systems*. 2023. URL: <https://openreview.net/forum?id=4hturzLcKX>.
- [317] Subhabrata Mukherjee et al. “Orca: Progressive Learning from Complex Explanation Traces of GPT-4.” In: (2023). arXiv: [2306.02707](https://arxiv.org/abs/2306.02707) [cs.CL].
- [318] Wenhui Wang et al. “MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers.” In: *NeurIPS*. 2020. URL: <https://proceedings.neurips.cc/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- [319] Baolin Peng et al. “Instruction Tuning with GPT-4.” In: (2023). arXiv: [2304.03277](https://arxiv.org/abs/2304.03277) [cs.CL].
- [320] Asaf Yehudai et al. “Achieving Human Parity in Content-Grounded Datasets Generation.” In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=RjYKTQ0L0W>.
- [321] Zhuyun Dai et al. “Dialog Inpainting: Turning Documents into Dialogs.” In: *Proceedings of the 39th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri et al. Vol. 162. Proceedings of Machine Learning Research. PMLR, July 2022, pp. 4558–4586. URL: <https://proceedings.mlr.press/v162/dai22a.html>.
- [322] Victor Sanh et al. “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.” In: *CoRR* abs/1910.01108 (2019). URL: <http://arxiv.org/abs/1910.01108>.

- [323] Junxian He et al. “Revisiting Self-Training for Neural Sequence Generation.” In: *International Conference on Learning Representations*. 2020. URL: <https://openreview.net/forum?id=SJgdnAVKDH>.
- [324] John Schulman. *Reinforcement learning from human feedback: Progress and challenges*. YouTube. 2023. URL: https://youtu.be/hhiLw5Q_UFg.
- [325] Xuanyu Zhang and Qing Yang. “Self-QA: Unsupervised Knowledge Guided Language Model Alignment.” In: *CoRR* abs/2305.11952 (2023). URL: <https://doi.org/10.48550/arXiv.2305.11952>.
- [326] Aman Madaan et al. “Self-Refine: Iterative Refinement with Self-Feedback.” In: *Thirty-seventh Conference on Neural Information Processing Systems*. 2023. URL: <https://openreview.net/forum?id=S37h0erQLB>.
- [327] Hugo Touvron et al. “LLaMA: Open and Efficient Foundation Language Models.” In: (2023). arXiv: [2302.13971](https://arxiv.org/abs/2302.13971) [cs.CL].
- [328] Noah Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. arXiv: [2303.11366](https://arxiv.org/abs/2303.11366) [cs.AI].
- [329] Baian Chen et al. *FireAct: Toward Language Agent Fine-tuning*. 2023. arXiv: [2310.05915](https://arxiv.org/abs/2310.05915) [cs.CL].
- [330] Caglar Gulcehre et al. *Reinforced Self-Training (ReST) for Language Modeling*. 2023. arXiv: [2308.08998](https://arxiv.org/abs/2308.08998) [cs.CL].
- [331] Yizhong Wang et al. *Self-Instruct: Aligning Language Models with Self-Generated Instructions*. 2023. arXiv: [2212.10560](https://arxiv.org/abs/2212.10560) [cs.CL].
- [332] slaypni. *fastdtw: Fast implementation of the Dynamic Time Warping algorithm*. Accessed: 2024-05-22. 2017. URL: <https://github.com/slaypni/fastdtw>.
- [333] Stan Salvador and Philip Chan. “Toward Accurate Dynamic Time Warping in Linear Time and Space.” In: vol. 11. Jan. 2004, pp. 70–80.
- [334] Paloma Sodhi et al. *SteP: Stacked LLM Policies for Web Actions*. 2024. arXiv: [2310.03720](https://arxiv.org/abs/2310.03720) [cs.LG].
- [335] Hanyu Lai et al. *AutoWebGLM: Bootstrap And Reinforce A Large Language Model-based Web Navigating Agent*. 2024. arXiv: [2404.03648](https://arxiv.org/abs/2404.03648) [cs.CL].

- [336] B. Efron. “Bootstrap Methods: Another Look at the Jackknife.” In: *The Annals of Statistics* 7.1 (Jan. 1979), pp. 1–26. DOI: [10.1214/aos/1176344552](https://doi.org/10.1214/aos/1176344552). URL: <https://doi.org/10.1214%2Faos%2F1176344552>.
- [337] Alfred V. Aho and Jeffrey D. Ullman. *The Theory of Parsing, Translation and Compiling*. Vol. 1. Englewood Cliffs, NJ: Prentice-Hall, 1972.
- [338] American Psychological Association. *Publications Manual*. Washington, DC: American Psychological Association, 1983.
- [339] Ashok K. Chandra, Dexter C. Kozen, and Larry J. Stockmeyer. “Alternation.” In: *Journal of the Association for Computing Machinery* 28.1 (1981), pp. 114–133. DOI: [10.1145/322234.322243](https://doi.org/10.1145/322234.322243).
- [340] Galen Andrew and Jianfeng Gao. “Scalable training of L1-regularized log-linear models.” In: *Proceedings of the 24th International Conference on Machine Learning*. 2007, pp. 33–40.
- [341] Dan Gusfield. *Algorithms on Strings, Trees and Sequences*. Cambridge, UK: Cambridge University Press, 1997.
- [342] Mohammad Sadegh Rasooli and Joel R. Tetreault. “Yara Parser: A Fast and Accurate Dependency Parser.” In: *Computing Research Repository* arXiv:1503.06733 (2015). version 2. URL: <http://arxiv.org/abs/1503.06733>.
- [343] Rie Kubota Ando and Tong Zhang. “A Framework for Learning Predictive Structures from Multiple Tasks and Unlabeled Data.” In: *Journal of Machine Learning Research* 6 (Dec. 2005), pp. 1817–1853. ISSN: 1532-4435.
- [344] Hu Xu et al. “Demystifying CLIP Data.” In: *ICML*.
- [345] Wei Li et al. “On the Effects of Data Scale on Computer Control Agents.” In: *arXiv preprint arXiv:2406.03679* (2024).
- [346] Niklas Muennighoff et al. “OLMoE: Open Mixture-of-Experts Language Models.” In: *arXiv preprint arXiv:2409.02060* (2024).
- [347] Dirk Groeneveld et al. “OLMo: Accelerating the Science of Language Models.” In: *arXiv preprint arXiv:2402.00838* (2024).
- [348] An Yang et al. “Qwen2 Technical Report.” In: *arXiv preprint arXiv:2407.10671* (2024).

- [349] *Chatbot Arena: New models and Elo system update.* <https://lmsys.org/blog/2023-12-07-leaderboard/>. Accessed: 2024-09-24.
- [350] *RealWorldQA.* <https://huggingface.co/datasets/xai-org/RealworldQA>. Accessed: 2024-09-24.
- [351] Xiang Yue et al. “MMMUI: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI.” In: *CVPR*. 2024.
- [352] Bo Li et al. “Otterhd: A high-resolution multi-modality model.” In: *arXiv preprint arXiv:2311.04219* (2023).
- [353] Wenyi Hong et al. “CogVLM2: Visual Language Models for Image and Video Understanding.” In: *arXiv preprint arXiv:2408.16500* (2024).
- [354] Xi Chen et al. “Pali-3 vision language models: Smaller, faster, stronger.” In: *arXiv preprint arXiv:2310.09199* (2023).
- [355] Lin Chen et al. “ShareGPT4V: Improving Large Multi-Modal Models with Better Captions.” In: *arXiv preprint arXiv:2311.12793* (2023).
- [356] OpenAI. “GPT-4o System Card.” In: *arXiv preprint arXiv:2410.21276* (2024).
- [357] OpenAI. *GPT-4o mini System Card*. 2024. URL: <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>.
- [358] 01. AI et al. “Yi: Open Foundation Models by 01.AI.” In: *arXiv preprint arXiv:2403.04652* (2024).
- [359] Mehdi Cherti et al. “Reproducible Scaling Laws for Contrastive Language-Image Learning.” In: *CVPR*. 2023.
- [360] Xiaohua Zhai et al. “Sigmoid loss for language image pre-training.” In: *ICCV*. 2023.
- [361] Alexey Dosovitskiy et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.” In: *ICLR*. 2021.
- [362] Chang Liu, Henghui Ding, and Xudong Jiang. “GRES: Generalized Referring Expression Segmentation.” In: *CVPR*. 2023.

- [363] Yash Goyal et al. “Making the V in VQA matter: Elevating the role of image understanding in visual question answering.” In: *CVPR*. 2017.
- [364] Fuxiao Liu et al. “Mmc: Advancing multimodal chart understanding with large-scale instruction tuning.” In: *arXiv preprint arXiv:2311.10774* (2023).
- [365] Hugo Laurençon, Léo Tronchon, and Victor Sanh. “Unlocking the conversion of Web Screenshots into HTML Code with the WebSight Dataset.” In: *arXiv preprint arXiv:2403.09029* (2024).
- [366] Ziyi Lin et al. “SPHINX: The Joint Mixing of Weights, Tasks, and Visual Embeddings for Multi-modal Large Language Models.” In: *arXiv preprint arXiv:2311.07575* (2023).
- [367] Hugo Laurençon et al. “Building and better understanding vision-language models: insights and future directions.” In: *arXiv preprint arXiv:2408.12637* (2024).
- [368] Hugo Laurençon et al. “What matters when building vision-language models?” In: *NeurIPS*. 2024.
- [369] Yuan Yao et al. “MiniCPM-V: A GPT-4V Level MLLM on Your Phone.” In: *arXiv preprint arXiv:2408.01800* (2024).
- [370] Chaoyou Fu et al. “Vita: Towards open-source interactive omni multimodal llm.” In: *arXiv preprint arXiv:2408.05211* (2024).
- [371] Min Shi et al. “Eagle: Exploring The Design Space for Multimodal LLMs with Mixture of Encoders.” In: *arXiv preprint arXiv:2408.15998* (2024).
- [372] Haozhe Zhao et al. “MMICL: Empowering Vision-language Model with Multi-Modal In-Context Learning.” In: *ICLR*. 2024.
- [373] Ji Lin et al. “VILA: On Pre-training for Visual Language Models.” In: *CVPR*. 2024.
- [374] Yunhao Fang et al. “VILA²: VILA Augmented VILA.” In: *arXiv preprint arXiv:2407.17453* (2024).
- [375] Haotian Zhang et al. “MM1.5: Methods, Analysis & Insights from Multimodal LLM Fine-tuning.” In: *arXiv preprint arXiv:2409.20566* (2024).

- [376] Renrui Zhang et al. “LLaMA-Adapter: Efficient Fine-tuning of Large Language Models with Zero-initialized Attention.” In: *ICLR*. 2024.
- [377] Zhangheng Li et al. “Ferret-UI 2: Mastering Universal User Interface Understanding Across Platforms.” In: *arXiv preprint arXiv:2410.18967* (2024).
- [378] Shraman Pramanick et al. “Jack of All Tasks, Master of Many: Designing General-purpose Coarse-to-Fine Vision-Language Model.” In: *CVPR*. 2023.
- [379] Bo Li et al. “Otter: A Multi-Modal Model with In-Context Instruction Tuning.” In: *arXiv preprint arXiv:2305.03726* (2023).
- [380] Haoxuan You et al. “Ferret: Refer and ground anything anywhere at any granularity.” In: *arXiv preprint arXiv:2310.07704* (2023).
- [381] Haotian Zhang et al. “Ferret-v2: An Improved Baseline for Referring and Grounding with Large Language Models.” In: *COLM*. 2024.
- [382] Jiasen Lu et al. “Unified-IO 2: Scaling Autoregressive Multimodal Models with Vision Language Audio and Action.” In: *CVPR*. 2024.
- [383] Junke Wang et al. “To see is to believe: Prompting gpt-4v for better visual instruction tuning.” In: *arXiv preprint arXiv:2311.07574* (2023).
- [384] Guiming Hardy Chen et al. “Allava: Harnessing gpt4v-synthesized data for a lite vision-language model.” In: *arXiv preprint arXiv:2402.11684* (2024).
- [385] Bo Li et al. “MIMIC-IT: Multi-Modal In-Context Instruction Tuning.” In: *arXiv preprint arXiv:2306.05425* (2023).
- [386] Rahul Thapa et al. “Dragonfly: Multi-Resolution Zoom-In Encoding Enhances Vision-Language Models.” In: *arXiv preprint arXiv:2406.00977* (2024).
- [387] Hanoona Rasheed et al. “GLaMM: Pixel Grounding Large Multimodal Model.” In: *CVPR*. 2024.
- [388] Dongyang Liu et al. “SPHINX-X: Scaling Data and Parameters for a Family of Multi-modal Large Language Models.” In: *ICML*. 2024.
- [389] Dongfu Jiang et al. “MANTIS: Interleaved Multi-Image Instruction Tuning.” In: *arXiv preprint arXiv:2405.01483* (2024).

- [390] Weihang Wang et al. “CogVLM: Visual Expert for Pretrained Language Models.” In: *NeurIPS*. 2024.
- [391] Junbum Cha et al. “Honeybee: Locality-Enhanced Projector for Multimodal LLM.” In: *CVPR*. 2024.
- [392] Bin Lin et al. “MoE-LLaVA: Mixture of Experts for Large Vision-Language Models.” In: *arXiv preprint arXiv:2401.15947* (2024).
- [393] Brandon McKinzie et al. “MM1: Methods, Analysis & Insights from Multimodal LLM Pre-training.” In: *arXiv preprint arXiv:2403.09611* (2024).
- [394] Chunjiang Ge et al. “ConvLLaVA: Hierarchical Backbones as Visual Encoder for Large Multimodal Models.” In: *arXiv preprint arXiv:2405.15738* (2024).
- [395] Bin Xiao et al. “Florence-2: Advancing a unified representation for a variety of vision tasks.” In: *CVPR*. 2024.
- [396] Deyao Zhu et al. “MiniGPT-4: Enhancing Vision-Language Understanding with Advanced Large Language Models.” In: *ICLR*. 2024.
- [397] Zhengqing Yuan et al. “TinyGPT-V: Efficient Multimodal Large Language Model via Small Backbones.” In: *ICML Workshop on Advancing Neural Network Training*. 2024.
- [398] Xiangxiang Chu et al. “MobileVLM : A Fast, Strong and Open Vision Language Assistant for Mobile Devices.” In: *arXiv preprint arXiv:2312.16886* (2023).
- [399] Junyan Li et al. “CoVLM: Composing Visual Entities and Relationships in Large Language Models Via Communicative Decoding.” In: *ICLR*. 2024.
- [400] Xiaoran Fan et al. “MouSi: Poly-Visual-Expert Vision-Language Models.” In: *COLM*. 2024.
- [401] Gen Luo et al. “Cheap and Quick: Efficient Vision-Language Instruction Tuning for Large Language Models.” In: *NeurIPS*. 2023.
- [402] Yiwei Ma et al. “INF-LLaVA: Dual-perspective Perception for High-Resolution Multimodal Large Language Model.” In: *arXiv preprint arXiv:2407.16198* (2024).

- [403] Ranjay Krishna et al. “Visual Genome: Connecting Language and Vision Using Crowd-sourced Dense Image Annotations.” In: *International Journal of Computer Vision* 123 (2016), pp. 32–73.
- [404] Shaohan Huang et al. “Language Is Not All You Need: Aligning Perception with Language Models.” In: *NeurIPS*. 2023.
- [405] Zhiliang Peng et al. “Kosmos-2: Grounding Multimodal Large Language Models to the World.” In: *arXiv preprint arXiv:2306.14824* (2023).
- [406] Wenliang Dai et al. “InstructBLIP: Towards General-purpose Vision-Language Models with Instruction Tuning.” In: *NeurIPS*. 2023.
- [407] Mingze Xu et al. “SlowFast-LLaVA: A Strong Training-Free Baseline for Video Large Language Models.” In: *arXiv preprint arXiv:2407.15841* (2024).
- [408] Kaibing Chen et al. “EVLM: An Efficient Vision-Language Model for Visual Understanding.” In: *arXiv preprint arXiv:2407.14177* (2024).
- [409] Tao Zhang et al. “OMG-LLaVA: Bridging Image-level, Object-level, Pixel-level Reasoning and Understanding.” In: *NeurIPS*. 2024.
- [410] Hai-Long Sun et al. “Parrot: Multilingual Visual Instruction Tuning.” In: *arXiv preprint arXiv:2406.02539* (2024).
- [411] Jinze Bai et al. “Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond.” In: *arXiv preprint arXiv:2308.12966* (2023).
- [412] Licheng Yu et al. “Modeling context in referring expressions.” In: *ECCV*. 2016.
- [413] Chao Jia et al. “Scaling up visual and vision-language representation learning with noisy text supervision.” In: *ICML*. 2021.
- [414] Jason Weston, Samy Bengio, and Nicolas Usunier. “Wsabie: Scaling up to large vocabulary image annotation.” In: *IJCAI*. 2011.
- [415] Andrea Frome et al. “Devise: A deep visual-semantic embedding model.” In: *NeurIPS* (2013).
- [416] Chaitanya Malaviya et al. “ExpertQA: Expert-curated questions and attributed answers.” In: *arXiv preprint arXiv:2309.07852* (2023).

- [417] Siddharth Karamcheti et al. “Prismatic vlms: Investigating the design space of visually-conditioned language models.” In: *arXiv preprint arXiv:2402.07865* (2024).
- [418] Wei-Lin Chiang et al. “Chatbot arena: An open platform for evaluating LLMs by human preference.” In: *ICML*. 2024.
- [419] Rohan Bavishi et al. *Fuyu-8B: A Multimodal Architecture for AI Agents*. 2023. URL: <https://www.adept.ai/blog/fuyu-8b>.
- [420] Wenliang Dai et al. “NVLM: Open frontier-class multimodal LLMs.” In: *arXiv preprint arXiv:2409.11402* (2024).
- [421] Joeri R Hermans, Gerasimos Spanakis, and Rico Möckel. “Accumulated gradient normalization.” In: *Asian Conference on Machine Learning*. PMLR. 2017, pp. 439–454.
- [422] Pan Lu et al. “MathVista: Evaluating mathematical reasoning of foundation models in visual contexts.” In: *ICLR*. 2024.
- [423] Peng Wang et al. “Qwen2-VL: Enhancing Vision-Language Model’s Perception of the World at Any Resolution.” In: *arXiv preprint arXiv:2409.12191* (2024).
- [424] Junnan Li et al. “BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation.” In: *ICML*. 2022.
- [425] Junnan Li et al. “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models.” In: *ICML*. 2023.
- [426] Jean-Baptiste Alayrac et al. “Flamingo: a visual language model for few-shot learning.” In: *NeurIPS*. 2022.
- [427] Angela Fan et al. “ELI5: Long Form Question Answering.” In: *ACL*. 2019.
- [428] Shengbang Tong et al. “Eyes wide shut? exploring the visual shortcomings of multi-modal LLMs.” In: *CVPR*. 2024.
- [429] Xinlei Chen et al. “Microsoft COCO captions: Data collection and evaluation server.” In: *arXiv preprint arXiv:1504.00325* (2015).
- [430] Alec Radford et al. “Improving language understanding by generative pre-training.” In: *OpenAI Blog* (2018).

- [431] Jiabo Ye et al. “UReader: Universal OCR-free visually-situated language understanding with multimodal large language model.” In: *Findings of EMNLP*. 2023.
- [432] Haoyu Lu et al. “Deepseek-vl: towards real-world vision-language understanding.” In: *arXiv preprint arXiv:2403.05525* (2024).
- [433] Pan Zhang et al. “Internlm-xcomposer-2.5: A versatile large vision language model supporting long-contextual input and output.” In: *arXiv preprint arXiv:2407.03320* (2024).
- [434] Xiaoyi Dong et al. “InternLM-XComposer2: Mastering Free-form Text-Image Composition and Comprehension in Vision-Language Large Model.” In: *arXiv preprint arXiv:2401.16420* (2024).
- [435] Xiaoyi Dong et al. “InternLM-XComposer2-4KHD: A Pioneering Large Vision-Language Model Handling Resolutions from 336 Pixels to 4K HD.” In: *arXiv preprint arXiv:2404.06512* (2024).
- [436] Zhe Chen et al. “How far are we to GPT-4V? closing the gap to commercial multimodal models with open-source suites.” In: *arXiv preprint arXiv:2404.16821* (2024).
- [437] Ilya Loshchilov and Frank Hutter. “Sgdr: Stochastic gradient descent with warm restarts.” In: *arXiv preprint arXiv:1608.03983* (2016).
- [438] Haotian Liu et al. “Improved baselines with visual instruction tuning.” In: *CVPR*. 2024.
- [439] Charig Yang, Weidi Xie, and Andrew Zisserman. “It’s About Time: Analog Clock Reading in the Wild.” In: *CVPR*. 2022.
- [440] Diederik P Kingma. “Adam: A method for stochastic optimization.” In: *ICLR*. 2015.
- [441] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization.” In: *ICLR*. 2019.
- [442] Alexander Kirillov et al. “Segment anything.” In: *ICCV*. 2023.
- [443] Jordi Pont-Tuset et al. “Connecting Vision and Language with Localized Narratives.” In: *ECCV*. 2020.
- [444] Xingyi Zhou et al. “Detecting Twenty-thousand Classes using Image-level Supervision.” In: *ECCV*. 2022.

- [445] Huanjin Yao et al. “Dense Connector for MLLMs.” In: *NeurIPS*. 2024.
- [446] Quan Sun et al. “Generative Multimodal Models are In-Context Learners.” In: *CVPR*. 2024.
- [447] Anwen Hu et al. “mPLUG-DocOwl 1.5: Unified Structure Learning for OCR-free Document Understanding.” In: *Findings of EMNLP*. 2024.
- [448] Yuqian Yuan et al. “Osprey: Pixel Understanding with Visual Instruction Tuning.” In: *CVPR*. 2024.
- [449] Maria Tsimpoukelli et al. “Multimodal Few-Shot Learning with Frozen Language Models.” In: *NeurIPS*. 2021.
- [450] Zhang Li et al. “Monkey: Image Resolution and Text Label are Important Things for Large Multi-Modal Models.” In: *CVPR* (2024).
- [451] Marah Abdin et al. “Phi-3 technical report: A highly capable language model locally on your phone.” In: *arXiv preprint arXiv:2404.14219* (2024).
- [452] Maxime Oquab et al. “DINOv2: Learning robust visual features without supervision.” In: *arXiv preprint arXiv:2304.07193* (2023).
- [453] Christoph Schuhmann et al. “LAION-5B: An open large-scale dataset for training next generation image-text models.” In: *arXiv preprint arXiv:2210.08402* (2022).
- [454] OpenGVLab Team. *InternVL2: Better than the Best—Expanding Performance Boundaries of Open-Source Multimodal Models with the Progressive Scaling Strategy*. July 2024. URL: <https://internvl.github.io/blog/2024-07-02-InternVL-2.0/>.
- [455] Yanli Zhao et al. “PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel.” In: *arXiv preprint arXiv:2304.11277* (2023).
- [456] Tri Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.” In: *NeurIPS*. 2022.
- [457] Tri Dao. “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.” In: *ICLR*. 2024.
- [458] Jianlin Su et al. “RoFormer: Enhanced Transformer with Rotary Position Embedding.” In: *Neurocomputing* (2024).

- [459] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. “Layer Normalization.” In: *NeurIPS Deep Learning Symposium*. 2016.
- [460] Dan Hendrycks and Kevin Gimpel. “Gaussian Error Linear Units (GELUs).” In: *arXiv preprint arXiv:1606.08415* (2016).
- [461] Noam Shazeer. “GLU Variants Improve Transformer.” In: *arXiv preprint arXiv:2002.05202* (2020).
- [462] Tao Ge et al. “Scaling synthetic data creation with 1,000,000,000 personas.” In: *arXiv preprint arXiv:2406.20094* (2024).
- [463] Tsung-Yi Lin et al. “Microsoft COCO: Common Objects in Context.” In: *ECCV*. 2014.
- [464] Alina Kuznetsova et al. “The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale.” In: *IJCV* (2020).
- [465] Roy Jonker and Ton Volgenant. “A shortest augmenting path algorithm for dense and sparse linear assignment problems.” In: *Computing* (1987).
- [466] David F Crouse. “On implementing 2D rectangular assignment algorithms.” In: *IEEE Transactions on Aerospace and Electronic Systems* (2016).
- [467] Dan Hendrycks et al. “Measuring Massive Multitask Language Understanding.” In: *ICLR*. 2021.
- [468] Yubo Wang et al. “MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark.” In: *NeurIPS Track on Datasets and Benchmarks*. 2024.
- [469] Karl Cobbe et al. “Training Verifiers to Solve Math Word Problems.” In: *arXiv preprint arXiv:2110.14168* (2021).
- [470] Dan Hendrycks et al. “Measuring Mathematical Problem Solving With the MATH Dataset.” In: *NeurIPS Track on Datasets and Benchmarks*. 2021.
- [471] Peter Clark et al. “Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge.” In: *arXiv preprint arXiv:1803.05457* (2018).
- [472] Mark Chen et al. “Evaluating Large Language Models Trained on Code.” In: *arXiv preprint arXiv:2107.03374* (2021).

- [473] Yizhong Wang et al. “How far can camels go? exploring the state of instruction tuning on open resources.” In: *NeurIPS Track on Datasets and Benchmarks*. 2023.
- [474] Amanpreet Singh et al. *FLAVA: A Foundational Language And Vision Alignment Model*. 2021. eprint: [arXiv:2112.04482](https://arxiv.org/abs/2112.04482).
- [475] Srija Mukhopadhyay et al. “Unraveling the Truth: Do VLMs really Understand Charts? A Deep Dive into Consistency and Robustness.” In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. Ed. by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 16696–16717. DOI: [10.18653/v1/2024.findings-emnlp.973](https://doi.org/10.18653/v1/2024.findings-emnlp.973). URL: <https://aclanthology.org/2024.findings-emnlp.973/>.
- [476] Florian Bordes et al. *An Introduction to Vision-Language Modeling*. 2024. eprint: [arXiv:2405.17247](https://arxiv.org/abs/2405.17247).
- [477] Noah Siegel et al. “Figureseer: Parsing result-figures in research papers.” In: *Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part VII 14*. Springer. 2016, pp. 664–680.
- [478] Jonathan Roberts et al. “SciFIBench: Benchmarking Large Multimodal Models for Scientific Figure Interpretation.” In: *arXiv preprint arXiv:2405.08807* (2024).
- [479] Zhengzhuo Xu et al. “Chartbench: A benchmark for complex visual reasoning in charts.” In: *arXiv preprint arXiv:2312.15915* (2023).
- [480] Maria Tsimpoukelli et al. “Multimodal few-shot learning with frozen language models.” In: *Advances in Neural Information Processing Systems 34* (2021), pp. 200–212.
- [481] Zechen Bai et al. “Hallucination of multimodal large language models: A survey.” In: *arXiv preprint arXiv:2404.18930* (2024).
- [482] Pranav Rajpurkar, Robin Jia, and Percy Liang. “Know What You Don’t Know: Unanswerable Questions for SQuAD.” In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Ed. by Iryna Gurevych and Yusuke Miyao. Melbourne, Australia: Association for Computational Linguistics, July 2018, pp. 784–789. DOI: [10.18653/v1/P18-2124](https://doi.org/10.18653/v1/P18-2124). arXiv: [1806.03822](https://arxiv.org/abs/1806.03822) [cs.CL]. URL: <https://aclanthology.org/P18-2124>.
- [483] Zayne Sprague et al. “To cot or not to cot? chain-of-thought helps mainly on math and symbolic reasoning.” In: *arXiv preprint arXiv:2409.12183* (2024).

- [484] Junnan Li et al. “Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models.” In: *International conference on machine learning*. PMLR. 2023, pp. 19730–19742.
- [485] Subhabrata Mukherjee et al. “Orca: Progressive learning from complex explanation traces of gpt-4.” In: *arXiv preprint arXiv:2306.02707* (2023).
- [486] Jianlv Chen et al. *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. 2024. arXiv: [2402.03216 \[cs.CL\]](#).